Contents

1 Basic 1
2 Graph 2
3 Data Structure 4
4 Flow/Matching 7
5 String 10
6 Math 12
7 Polynomial 16
8 Geometry 17
9 Else 22
10 Python 25

1 Basic

1.1 Default code

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
#define X first
#define Y second
#define SZ(a) ((int)a.size())
#define ALL(v) v.begin(), v.end()
#define pb push_back
```

1.2 Dinic [437335]

Max flow. $O(V^2E)$; unit caps $O(E\sqrt{E})$; bipartite $O(E\sqrt{V})$. cap is int. 0-base.

```
struct MaxFlow { // 0-base
  struct edge { int to, cap, flow, rev; };
  vector<edge> G[MAXN];
  int s, t, dis[MAXN], cur[MAXN], n;
  int dfs(int u, int cap) {
    if (u == t || !cap) return cap;
    for (int &i = cur[u]; i < (int)G[u].size(); ++i) {
      edge &e = G[u][i];
      if (dis[e.to] == dis[u] + 1 && e.flow != e.cap) {
        int df = dfs(e.to, min(e.cap - e.flow, cap));
        if (df) {
          e.flow += df;
          G[e.to][e.rev].flow -= df;
          return df;
        }
      }
    }
    dis[u] = -1;
    return 0;
  }
  bool bfs() {
    fill_n(dis, n, -1);
    queue<int> q; q.push(s); dis[s] = 0;
    while (!q.empty()) {
      int tmp = q.front(); q.pop();
      for (auto &u : G[tmp])
        if (!~dis[u.to] && u.flow != u.cap) {
          q.push(u.to);
          dis[u.to] = dis[tmp] + 1;
        }
    }
    return dis[t] != -1;
  }
  int maxflow(int _s, int _t) {
    // Avoid DFS when s == t:
    //   its base case would otherwise return INF forever.
    if (_s == _t) return 0;
    s = _s; t = _t;
    int flow = 0, df;
    while (bfs()) {
      fill_n(cur, n, 0);
      while ((df = dfs(s, INF))) flow += df;
    }
    return flow;
  }
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i) G[i].clear();
  }
  void reset() {
    for (int i = 0; i < n; ++i)
      for (auto &j : G[i]) j.flow = 0;
  }
  void add_edge(int u, int v, int cap) {
    G[u].pb({v, cap, 0, (int)G[v].size()});
    G[v].pb({u, 0, 0, (int)G[u].size() - 1});
  }
};
MaxFlow mf; mf.init(n); // nodes 0..n-1
mf.add_edge(u, v, c); // directed
int f = mf.maxflow(s, t);
// dis[i]!=-1 => i on s-side of min cut
```

1.3 vimrc

Editor config. The :Hash commands md5 a visual selection so you can check a typed template against the printed hash.

```
" Shell: setxkbmap -option ctrl:nocaps
" ~/.vimrc; :Hash or :2,5 Hash (choose one version below).
se nu ai hls et ru ic is sc cul re=1
se ts=4 sts=4 sw=4 ls=2 mouse=a
syntax on|hi cursorline cterm=none ctermbg=89|set bg=dark
inoremap {<CR> {<CR>}<Esc>ko<tab>
ca Hash w !cpp -dD -P -fpreprocessed \|
  \ tr -d '[:space:]' \| md5sum \| cut -c-6
ca Hash w !cpp -dD -P \| sed "/^\#.*$/d" \|
  \ tr -d '[:space:]' \| md5sum \| cut -c-6
ca Hash w !g++ -E - \| sed "/^\#.*$/d" \|
  \ tr -d '[:space:]' \| md5sum \| cut -c-6
```

1.4 readchar [9c8bd3]

Buffered getchar via fread_unlocked, amortised $O(1)$. Returns a byte or EOF; store in int. Do not mix input readers.

```
// Private stdin buffer; don't mix readers; result is int.
inline int readchar() { // byte or EOF; amortized O(1)
  static char buf[65536], *p = buf, *e = buf;
  if (p == e) p = buf,
    e = buf + fread_unlocked(buf, 1, sizeof buf, stdin);
  return p == e ? EOF : (unsigned char)*p++;
}
int x = 0, c = readchar(); // int preserves EOF
while (c != EOF && (c < '0' || c > '9')) c = readchar();
while (c >= '0' && c <= '9')
  x = x * 10 + c - '0', c = readchar();
```

1.5 BigIntIO [de590a]

Read/print signed __int128. read(x) fails on EOF/overflow, leaving x unchanged; read() returns 0 on failure. Handles sign.

```
// Unlocked stdio, one thread; skips junk; adjacent sign.
bool read(__int128 &value) { // EOF/overflow: unchanged
  int ch = getchar_unlocked(), prev = 0;
  bool overflow = false;
  while (ch != EOF && !isdigit(ch))
    prev = ch, ch = getchar_unlocked();
  if (ch == EOF) return false; bool neg = prev == '-';
  __uint128_t x = 0, lim = ((__uint128_t)1 << 127) - !neg;
  do { unsigned d = ch - '0';
    if (x > (lim - d) / 10) overflow = true;
    else x = x * 10 + d;
  } while
      (isdigit(ch = getchar_unlocked())); // eats delim
  if (overflow) return false;
  value = !neg ? (__int128)x : x >> 127 ?
    -(__int128)(x - 1) - 1 : -(__int128)x;
  return true;
}
__int128 read() { __int128 x = 0; read(x); return x; }
void print_unsigned(__uint128_t x) { // 64-bit if possible
  auto q = x >> 64 ? x / 10 : (uint64_t)x / 10;
  if (q) print_unsigned(q);
  putchar_unlocked(x - q * 10 + '0');
}
void print(__int128 x) {
  if (x < 0) putchar_unlocked('-');
  print_unsigned(x < 0 ? (__uint128_t)(-(x + 1)) + 1 : x);
}
bool cmp(__int128 x, __int128 y) { return x > y; }
__int128 a = read(), b = read();
print(a * b); putchar('\n');
```

1.6 Black Magic [a66634]

GNU extensions: mt19937; pb_ds red-black tree with find_by_order/order_of_key $O(\log n)$; mergeable heap join; persistent rope. Uses the default preamble; join empties its distinct source heap.

```
#include <bits/extc++.h>
#include <ext/rope>
using namespace __gnu_pbds; using namespace __gnu_cxx;
typedef tree<int, null_type, std::less<int>, rb_tree_tag,
  tree_order_statistics_node_update> tree_set;
typedef cc_hash_table<int, int> umap;
typedef __gnu_pbds::priority_queue<int> heap;
#ifndef BLACK_MAGIC_NO_MAIN
int main() {
  mt19937 rng(chrono::steady_clock::now()
    .time_since_epoch().count());
  auto get_rand = [&](int l, int r) { // l<=r
    return uniform_int_distribution<int>(l, r)(rng);
  };
  vector<int> v = {1, 2, 3}; get_rand(0, 10);
  shuffle(v.begin(), v.end(), rng); tree_set s;
  s.insert(71), s.insert(22); // O(log n), 0-based rank
  assert(*s.find_by_order(0)==22 &&
    *s.find_by_order(1)==71);
  assert(s.order_of_key(22)==0 && s.order_of_key(71)==1);
  s.erase(22);
```

```
  assert
      (*s.find_by_order(0)==71 && s.order_of_key(71)==0);
  // Meldable heap: distinct heaps; join empties b.
  heap a, b; a.push(1), b.push(2), a.join(b);
  rope<char> r[2]; r[1] = r[0]; // persistent copy
  r[1].insert(0, "abc"), r[1].erase(1, 1);
  std::cout << r[1].substr(0, 2) << std::endl;
}
#endif
```

1.7 Pragma Optimization [a37289]

Aggressive GCC optimisation and SIMD targets; Ofast relaxes FP semantics. Call apply_pragmas() for x86 DAZ/FTZ. SIGILL on unsupported CPUs.

```
// GCC, CPU supports targets; Ofast relaxes FP rules.
#pragma GCC optimize("Ofast,no-stack-protector")
#pragma GCC optimize("no-math-errno,unroll-loops")
#if defined(__x86_64__) || defined(__i386__)
#pragma GCC target("sse,sse2,sse3,ssse3,sse4")
#pragma GCC target("popcnt,abm,mmx,avx,arch=skylake")
inline void apply_pragmas() { __builtin_ia32_ldmxcsr(
    __builtin_ia32_stmxcsr() | 0x8040); }
#else
inline void apply_pragmas() {}
#endif
```

1.8 Bitset [1fd88d]

std::bitset API reference. reset() clears to 0 and set() fills with 1.

```
#ifndef BITSET_NO_MAIN
int main() {
  bitset<4> bit;
  bit.all(), bit.any(), bit.none(); // all / any / none
  bit.count(), bit.to_string('0', '1'); // count / digits
  bit.reset(), bit.set(); // all 0 / all 1
  bitset<8> b3{0}, b4{42}; hash<bitset<8>> hash_fn8;
  hash_fn8(b3), hash_fn8(b4);
}
#endif
```

2 Graph

2.1 BCC Vertex* [81836f]

Vertex biconnected components and articulation points, $O(V+E)$. Block-cut forest has **up to** $\max(n, 2n-2)$ **nodes**. Call block_cut_tree() once after solve(). 0-base.

```
struct BCC { // 0-base; O(V+E); multi-edges; ignore loops
  int n, dft, ecnt, nbcc;
  vector<int> low, dfn, bln, stk, is_ap, cir;
  vector<vector<pii>> G; vector<vector<int>> bcc, nG;
  void make_bcc(int u) { bcc.emplace_back();
    auto &b = bcc.back(); b.reserve(2), b.pb(u);
    for (; stk.back() != u; stk.pop_back())
      bln[stk.back()] = nbcc, b.pb(stk.back());
    stk.pop_back(), bln[u] = nbcc++; }
  void dfs(int u, int pe) {
    int child = 0; low[u] = dfn[u] = ++dft, stk.pb(u);
    for (auto [v, e] : G[u])
      if (!dfn[v]) {
        dfs(v, e), ++child, low[u] = min(low[u], low[v]);
        if (dfn[u] <= low[v]) {
          is_ap[u] = 1, bln[u] = nbcc;
          make_bcc(v), bcc.back().pb(u);
        }
      } else if (e != pe && dfn[v] < dfn[u])
        low[u] = min(low[u], dfn[v]);
    if (pe == -1 && child < 2) is_ap[u] = 0;
    if (pe == -1 && child == 0) make_bcc(u);
  }
  BCC(int _n): n(_n), dft(), ecnt(), nbcc(),
    // n>=0; counts fit int
    low(n), dfn(n), bln(n), is_ap(n), G(n) {}
  void add_edge(int u, int v) {
    G[u].pb({v, ecnt}), G[v].pb({u, ecnt++}); }
  void solve() { // edges first; IDs [0,n); enough stack
    for (int i = 0; i < n; ++i) if (!dfn[i]) dfs(i, -1); }
  void block_cut_tree() { // once after solve; nbcc/bln
    int base = nbcc; cir.assign(base, 1);
    for (int i = 0; i < n; ++i)
      if (is_ap[i]) bln[i] = nbcc++;
    nG.assign(nbcc, {});
    for (int i = 0; i < base; ++i) for (int j : bcc[i])
      if (is_ap[j]) nG[i].pb(bln[j]), nG[bln[j]].pb(i);
  } // <= max(n, 2*n-2) nodes; bln[i] maps vertex i
};
BCC b(n); b.add_edge(u, v); b.solve();
b.bcc[i]; // vertices of i-th block
b.is_ap[u]; b.bln[u];
b.block_cut_tree(); // nG, <= max(n, 2n-2) nodes
```

2.2 Bridge* [e040b0]

Bridges and 2-edge-connected components, $O(V+E)$. Uses edge ids rather than the parent vertex, so **multi-edges are handled correctly**. 0-base.

```
struct ECC { // 0-base; O(V+E); edge ids handle multiedges
  int n, dft, ecnt, necc;
  vector<int> low, dfn, bln, is_bridge, stk;
  vector<vector<pii>> G;
  void dfs(int u, int f) {
    dfn[u] = low[u] = ++dft, stk.pb(u);
    for (auto [v, e] : G[u])
      if (!
          dfn[v]) dfs(v, e), low[u] = min(low[u], low[v]);
      else if (e != f) low[u] = min(low[u], dfn[v]);
    if (low[u] == dfn[u]) {
      if (f != -1) is_bridge[f] = 1;
      for (; stk.back() != u; stk.pop_back())
        bln[stk.back()] = necc;
      bln[u] = necc++, stk.pop_back();
    }
  }
  ECC(int _n): n(_n), dft(), ecnt(), necc(), // n>=0
    low(n), dfn(n), bln(n), G(n) {} // Counts fit int.
  void add_edge(int u, int v) {
    G[u].pb({v, ecnt}), G[v].pb({u, ecnt++}); }
  void solve() { is_bridge.resize(ecnt);
    for (int i = 0; i < n; ++i) if (!dfn[i]) dfs(i, -1); }
}; // bln: ECC ids; vertices [0,n); DFS stack must fit
// Add all edges before solve.
ECC e(n); e.add_edge(u, v); e.solve();
e.is_bridge[edge_id]; // Edge ids handle multiedges
e.bln[u]; // 2-edge-connected comp id
```

2.3 SCC* [3e1afd]

Tarjan SCC, $O(V+E)$. bln ids come out in **reverse topological order** — 2SAT relies on this. 0-base.

```
struct SCC { // 0-base; O(V+E); reverse topological IDs
  int n, dft, nscc;
  vector<int> low, dfn, bln, instack, stk;
  vector<vector<int>> G;
  void dfs(int u) {
    low[u] = dfn[u] = ++dft, instack[u] = 1, stk.pb(u);
    for (int v : G[u])
      if (!dfn[v]) dfs(v), low[u] = min(low[u], low[v]);
      else if (instack[v]) low[u] = min(low[u], dfn[v]);
    if (low[u] == dfn[u]) {
      for (; stk.back() != u; stk.pop_back())
        bln[stk.back()] = nscc, instack[stk.back()] = 0;
      instack[u] = 0, bln[u] = nscc++, stk.pop_back();
    }
  }
  SCC(int _n): n(_n), dft(), nscc(), low(n), // n>=0
    dfn(n), bln(n), instack(n), G(n) {}
  void add_edge(int u, int v) { G[u].pb(v); }
  // Add edges first; IDs [0,n); enough DFS stack.
  void solve() {
    for (int i = 0; i < n; ++i) if (!dfn[i]) dfs(i); }
}; // scc_id(i): bln[i]
SCC s(n); s.add_edge(u, v); s.solve();
s.bln[u]; // component id
// ids are in REVERSE topological order
```

2.4 2SAT* [3eaa42]

$O(V+E)$ via implication graph + SCC. Variable i is node i (true) / $i+n$ (false). Requires SCC.cpp. Use istrue only if solve() succeeds; add all clauses first. 0-base.

```
struct SAT { // 0-base true=i,false=i+n; O(V+E)
  int n; vector<bool> istrue; // n>=0; 2*n fits int
  SCC scc; // Reverse-topological SCC; enough DFS stack.
  SAT(int _n): n(_n), istrue(n + n), scc(n + n) {}
  // Literals [0,2*n).
  int rv(int a) { return a >= n ? a - n : a + n; }
  void add_clause(int a, int b) {
    scc.add_edge(rv(a), b), scc.add_edge(rv(b), a); }
  bool solve() { // Clauses first; istrue only on success
    scc.solve(); for (int i = 0; i < n; ++i)
      if (scc.bln[i] == scc.bln[i + n]) return false;
      else istrue[i] = scc.bln[i] < scc.bln[i + n],
        istrue[i + n] = !istrue[i];
    return true; }
};
SAT sat(n); // var i true=i, false=i+n
sat.add_clause(a, b); // (a or b)
if (sat.solve()) use sat.istrue[i];
```

2.5 MinimumMeanCycle* [dd2d0b]

Karp, $O(VE+V^2)$: extracts finite matrix edges once. Absent entries: INF; absolute walk sums ($\le n$ edges) < INF, DP differences fit ll. Returns a reduced mean fraction, $(-1,-1)$ if acyclic.

```
// N>0; ll INF>0; |walk sums, <=n edges|<INF.
ll road[N][N]; // Fill active entries; absent=INF.
struct MinimumMeanCycle { // init 0<=n<=N.
  ll dp[N + 1][N]; int n; // Needs ample/static storage.
  static bool less_frac(pll a, pll b) { // Denominators>0.
```

```
    return (__int128)a.first * b.second <
           (__int128)b.first * a.second; }
  pll solve() {
    vector<vector<pair<int, ll>>> edges(n);
    for (
        int u = 0; u < n; ++u) for (int v = 0; v < n; ++v)
      if (road[u][v] < INF)
        edges[u].emplace_back(v, road[u][v]);
    fill_n(dp[0], n, 0); // every vertex may be the start
    for (int k = 1; k <= n; ++k) {
      fill_n(dp[k], n, INF);
      for (int u = 0; u < n; ++u)
        if (ll d = dp[k - 1][u]; d < INF)
          for (auto [v, w] : edges[u])
            dp[k][v] = min(dp[k][v], d + w);
    }
    vector<pll> high(n); vector<int> active;
    for (int v = 0; v < n; ++v) if (dp[n][v] < INF)
      high[v] = {dp[n][v], n}, active.push_back(v);
    for (int k = 1; k < n; ++k) for (int v : active) {
      pll cur = {dp[n][v] - dp[k][v], n - k};
      if (less_frac(high[v], cur)) high[v] = cur;
    }
    pll ans
        = {-1, -1}; // O(VE+V^2); dp differences fit ll
    for (int v : active)
      if (ans.second < 0 || less_frac(high[v], ans))
        ans = high[v];
    if (ans.second < 0) return ans;
    auto [p, q] = ans; ll g = gcd(p, q);
    return {p / g, q / g}; }
  void init(int _n) { n = _n; }
};
// fill road[i][j], INF if absent
static MinimumMeanCycle mmc; mmc.init(n);
auto [p, q] = mmc.solve(); // p/q reduced
// (-1,-1) if no cycle
// O(VE+V^2); builds edges from road, then Karp DP.
```

2.6 Virtual Tree* [e8dfa8]

Compresses k keys into an $O(k)$ tree in $O(k \log k + kT_{\mathrm{LCA}})$. Needs LCA(), preorderdfn,dep. Resetbeforerebuilding;buildsortskeysandreturns−1ifempty.

```
// Rooted tree: LCA, preorder dfn, dep; IDs [0,N).
vector<int> vG[N]; int top, st[N];
void insert(int u) { // top=-1 first; unique preorder keys
  if (top == -1) { st[++top] = u; return; }
  int p = LCA(st[top], u);
  while (top >= 1 && dep[st[top - 1]] >= dep[p])
    vG[st[top - 1]].pb(st[top]), --top;
  if (st[top] != p) vG[p].pb(st[top]), st[top] = p;
  st[++top] = u;
}
void reset(int u) { // valid root; enough recursion stack
  for (int i : vG[u]) reset(i); vG[u].clear();
}
// Clear previous vG first; build O(k log k+k*T_LCA).
int build(vector<int> &v) {
  if (v.empty()) return -1;
  top = -1;
  sort(ALL(v), [&](int a, int b) {
    return dfn[a] < dfn[b]; });
  v.erase(unique(ALL(v)), v.end());
  for (int i : v) insert(i);
  while (top > 0) vG[st[top - 1]].pb(st[top]), --top;
  return st[0];
}
void solve(vector<int> &v) {
  int root = build(v); if (root == -1) return;
  // do something
  reset(root);
}
// needs LCA(), dfn[], dep[] prepared
solve(keys); // builds vG, then resets
// do the DP where marked in the file
```

2.7 Maximum Clique Dyn* [4f0549]

Max clique by branch and bound with dynamic colouring. Exponential, but fast for $n \le 100$. Simple undirected graph, $0 \le n \le N$; GCC/libstdc++. 0-base.

```
struct MaxClique { // 0-base; 0<=n<=N, N>0; simple graph
  // Undirected; libstdc++ bitset search; exponential.
  bitset<N> G[N], cs[N + 2];
  vector<int> next[N + 1], color[N + 1];
  int ans, sol[N], q, cur[N], d[N], n;
  void init(int _n) { n = _n;
    for (int i = 0; i < n; ++i) G[i].reset(); }
  void add_edge(int u, int v) { G[u][v] = G[v][u] = 1; }
  void pre_dfs(vector<int> &r, int l, bitset<N> mask) {
    if (l < 4) {
      for (int i : r) d[i] = (G[i] & mask).count();
      sort(ALL(r), [&](int x, int y) {
        return d[x] > d[y]; });
    }
    auto &c = color[q]; c.assign(SZ(r), 0); // by depth
    int lft = max(ans - q + 1, 1), rgt = 1, tp = 0;
    cs[1].reset(), cs[2].reset();
    for (int p : r) {
      int k = 1;
      while ((cs[k] & G[p]).any()) ++k;
      if (k > rgt) cs[++rgt + 1].reset();
      cs[k][p] = 1;
      if (k < lft) r[tp++] = p;
    }
    for (int k = lft; k <= rgt; ++k)
      for (int p = cs[k]._Find_first(); p < N;
           p = cs[k]._Find_next(p))
        r[tp] = p, c[tp++] = k;
    dfs(r, c, l + 1, mask);
  }
  void dfs(vector<int> &r, vector<int> &c, int l,
           bitset<N> mask) {
    while (!r.empty()) {
      int p = r.back(); r.pop_back(), mask[p] = 0;
      if (q + c.back() <= ans) return;
      cur[q++] = p;
      auto
          &nr = next[q]; nr.clear(); nr.reserve(r.size());
      for (int i : r) if (G[p][i]) nr.pb(i);
      if (!nr.empty()) pre_dfs(nr, l, mask & G[p]);
      else if (q > ans) ans = q, copy_n(cur, q, sol);
      c.pop_back(), --q;
    }
  }
  int solve() {
    vector<int> r(n); ans = q = 0, iota(ALL(r), 0);
    pre_dfs(r, 0, bitset<N>().set()); return ans;
  }
};
static MaxClique mc; mc.init(n); mc.add_edge(u, v);
int k = mc.solve(); // sol[0..k) = clique
// fast for n <= 100
```

2.8 Minimum Steiner Tree* [c630e3]

$O(V^3 + V3^T + V^2 2^T)$, T distinct terminals. Nonnegative costs, vcst for vertices; directed common sink. Cost capped at INF. Solve once/init; no pre-call shortest_path. 0-base.

```
struct SteinerTree { // 0-base, 0 <= n <= N, t <= T < 31
  // 0 <= edge/vertex costs < INF <= INT_MAX; INF > 0
  // Directed common sink; solve once; no shortest_path
  int n, dst[N][N], dp[1 << T][N], tdst[N], vcst[N];
  void init(int _n) {
    n = _n; for (int i = 0; i < n; ++i)
      fill_n(dst[i], n, INF), dst[i][i] = vcst[i] = 0;
  }
  void chmin(int &x, ll val) { x = min<ll>(x, val); }
  void add_edge(int u, int v, int w) {
    chmin(dst[u][v], w); }
  void shortest_path() { // entered vertex cost; once/init
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j)
        if (i != j) dst[i][j] =
          min<ll>(INF, (ll)dst[i][j] + vcst[j]);
    for (int k = 0; k < n; ++k)
      for (int i = 0; i < n; ++i) if (dst[i][k] < INF)
        for (int j = 0; j < n; ++j)
          chmin(dst[i][j], (ll)dst[i][k] + dst[k][j]);
  }
  int solve(const vector<int>& ter) { // <= T distinct
    vector<int> ts = ter; sort(ALL(ts));
    ts.erase(unique(ALL(ts)), ts.end());
    if (ts.empty()) return 0;
    shortest_path();
    int t = SZ(ts), full = (1 << t) - 1;
    copy_n(vcst, n, dp[0]);
    for (int msk = 1; msk <= full; ++msk) {
      fill_n(dp[msk], n, INF); int rest = msk & (msk - 1);
      if (!rest) { int u = ts[__lg(msk)];
        dp[msk][u] = vcst[u]; } // singleton, then close
      for (int sub = rest; sub; sub = (sub - 1) & rest)
        for (int i = 0; i < n; ++i) // low bit in msk^sub
          chmin(dp[msk][i], (ll)dp[sub][i] +
                dp[msk ^ sub][i] - vcst[i]);
      fill_n(tdst, n, INF);
      for (int j = 0; j < n; ++j) if (dp[msk][j] < INF)
        for (int i = 0; i < n; ++i)
          chmin(tdst[i], (ll)dp[msk][j] + dst[j][i]);
      copy_n(tdst, n, dp[msk]);
    }
    return *min_element(dp[full], dp[full] + n);
  }
}; // O(V^3+V 3^T+V^2 2^T); min(cost, INF), empty: 0
static SteinerTree st; st.init(n); st.vcst[i] = nodeCost;
```

```
st.add_edge
    (u, v, w); // directed; 0<=edge/vertex costs<INF
int c
    = st.solve(terminals); // once/init; no shortest_path
// c==INF: disconnected or cost>=INF; empty terminals: 0.
```

2.9 Dominator Tree* [119173]

Lengauer-Tarjan, $O((V+E)\log V)$. Only vertices reachable from root matter. tree: vertex ids; idom: DFS ids. init before edges. **1-base.**

```
struct dominator_tree { // 1-base; O((V+E) log V)
  // init before edges; 1<=n<N; root/endpoints in [1,n].
  // Reachable only; enough DFS/eval stack.
  vector<int> G[N], rG[N], tree[N]; // tree: vertex IDs
  int n, pa[N], dfn[N], id[N], Time, semi[N];
  int idom[N], best[N]; // idom: DFS IDs
  void init(int _n) {
    n = _n; for (int i = 1; i <= n; ++i)
      G[i].clear(), rG[i].clear();
  }
  void add_edge(int u, int v) { G[u].pb(v), rG[v].pb(u); }
  void dfs(int u) {
    id[dfn[u] = ++Time] = u;
    for (int v : G[u]) if (!dfn[v])
      dfs(v), pa[dfn[v]] = dfn[u];
  }
  int find(int y, int x) {
    if (y <= x) return y;
    int tmp = find(pa[y], x);
    if (semi[best[y]] > semi[best[pa[y]]])
      best[y] = best[pa[y]];
    return pa[y] = tmp;
  }
  void tarjan(int root) {
    Time = 0; for (int i = 1; i <= n; ++i)
      dfn[i] = idom[i] = 0, tree[i].clear(),
      best[i] = semi[i] = i;
    dfs(root);
    for (int i = Time; i > 1; --i) {
      for (int v : rG[id[i]]) if (int w = dfn[v])
        find(w, i), semi[i] = min(semi[i], semi[best[w]]);
      tree[semi[i]].pb(i);
      for (int v : tree[pa[i]]) {
        find(v, pa[i]);
        idom[
            v] = semi[best[v]] == pa[i] ? pa[i] : best[v];
      }
      tree[pa[i]].clear();
    }
    for (int i = 2; i <= Time; ++i) {
      if (idom[i] != semi[i]) idom[i] = idom[idom[i]];
      tree[id[idom[i]]].pb(id[i]);
    }
  }
};
static dominator_tree dt; dt.init(n);
dt.add_edge(u, v); dt.tarjan(root); // 1-base
dt.tree[u]; // Children in dominator tree
```

2.10 Minimum Clique Cover* [1abb3f]

$O(n2^n)$ cover count modulo 2^{32}; cancellation may overestimate. Needs OR-zeta fwt() and 32-bit unsigned; simple undirected graph, $0 \le n \le N < 31$, with feasible 2^N arrays.

```
struct Clique_Cover { // 0-base; O(n2^n).
  // Simple undirected; valid IDs; 0<=n<=N<31; arrays fit.
  int co[1 << N], n, E[N];
  unsigned
      dp[1 << N]; // 32-bit; mod 2^32 may overestimate
  void init(int _n) { n = _n; fill_n(E, n, 0); }
  void add_edge(int u, int v) {
    E[u] |= 1 << v, E[v] |= 1 << u; }
  int solve() {
    int m = 1 << n;
    co[0] = 1, dp[0] = (n & 1) ? 1 : -1u;
    for (int i = 1; i < m; ++i) {
      int t = i & -i, s = i ^ t;
      dp[i] = -dp[s];
      co[i] = co[s] && (E[__builtin_ctz(t)] & s) == s;
    }
    fwt(co, m, 1); // OR-zeta fwt(int*,int,int).
    for (int ans = 1; ans < n; ++ans) {
      unsigned sum = 0;
      for (int i = 0; i < m; ++i) sum += (dp[i] *= co[i]);
      if (sum) return ans;
    }
    return n;
  }
};
static Clique_Cover cc; cc.init(n); cc.add_edge(u, v);
int k = cc.solve(); // O(n 2^n)
// Mod 2^32 may overestimate; needs OR-zeta fwt().
```

2.11 NumberofMaximalClique* [8bef6c]

Bron-Kerbosch with pivot, $O(n\lceil N/w\rceil 3^{n/3})$ (w: word bits). Count fits int. Simple undirected; GCC/libstdc++, $n < N$. 1-base.

```
struct
    BronKerbosch { // 1-base, 0 <= n < N; count fits int
  // Simple undirected; enough stack; GCC/libstdc++.
  // n=0: empty clique; dfs: init S; active P/X disjoint.
  int n, S; bitset<N> g[N];
  void init(int _n) {
    n = _n; for (int i = 1; i <= n; ++i) g[i].reset();
  }
  void add_edge(int u, int v) { g[u][v] = g[v][u] = 1; }
  void dfs(bitset<N> P, bitset<N> X) {
    if (P.none()) { if (X.none()) ++S; return; }
    auto U = P | X; int pivot = 0, best = -1;
    int bound = P.count() - X.none(); // Loopless bound.
    for (int u = U._Find_first(); u < N;
         u = U._Find_next(u)) {
      int score = (P & g[u]).count();
      if (score > best) best = score, pivot = u;
      if (score == bound) break;
    }
    auto cand = P & ~g[pivot];
    for (int v = cand._Find_first(); v < N;
         v = cand._Find_next(v)) {
      dfs(P & g[v], X & g[v]); P[v] = 0, X[v] = 1;
    }
  }
  void dfs_vec(vector<int> P, vector<int> X) { // As dfs.
    bitset<N> p, x; for (int v : P) p[v] = 1;
    for (int v : X) x[v] = 1; dfs(p, x);
  }
  int solve() {
    bitset<N> P; for (int i = 1; i <= n; ++i) P[i] = 1;
    S = 0, dfs(P, {}); return S;
  }
}; // O(n ceil(N/word_bits) 3^(n/3)), incl. bitset work.
// bitset<N> rows; needs GCC/libstdc++ bitset search.
static BronKerbosch bk; bk.init(n);
bk.add_edge(u, v); // 1-base
int c = bk.solve(); // Number of maximal cliques
```

3 Data Structure

3.1 BIT kth* [35445d]

k-th smallest from a 1-based value-domain BIT in $O(\log N)$. Any $N > 0$; frequencies nonnegative, $1 \le k \le$ total; sums fit int.

```
int bit[N + 1]; // 1-base, N>0, freq>=0; O(log N)
int query_kth(int k) { // 1<=k<=total; sums fit int
  int res = 0;
  for (int i = 1 << __lg(N); i; i >>= 1)
    if (res+i <= N && bit[res+i] < k) k -= bit[res += i];
  return res + 1;
}
// bit[1..N], N > 0; frequencies >= 0
int v = query_kth(k); // 1 <= k <= total
// O(log N); arbitrary N
```

3.2 Interval Container* [89e0ad]

Maintains nonempty disjoint, nonadjacent half-open $[L, R)$ intervals with auto-merge; amortised $O(\log n)$. Updates require $L \le R$.

```
// Nonempty disjoint, nonadjacent [L,R); require L<=R.
// Update O(log s + erasures), amortized O(log s).
set<pii>::iterator addInterval(
    set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.upper_bound({L, INT_MAX}), before = it;
  if (it != is.begin() && prev(it)->Y >= R)
    return prev(it);
  while (it != is.end() && it->X <= R)
    R = max(R, it->Y), before = it = is.erase(it);
  if (it != is.begin() && (--it)->Y >= L)
    L = min(L, it->X), R = max(R, it->Y), is.erase(it);
  return is.insert(before, {L, R}); }
void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = is.lower_bound({L, INT_MIN});
  if (it != is.begin() && prev(it)->Y > L) --it;
  while (it != is.end() && it->X < R) {
    auto [l, r] = *it; it = is.erase(it);
    if (l < L) is.emplace_hint(it, l, L);
    if (r > R) { is.emplace_hint(it, R, r); break; }
  }
}
set<pii> s; // disjoint [L, R)
addInterval(s, 3, 7); // merges overlaps
removeInterval(s, 4, 5); // punches a hole
```

3.3 Leftist Tree [dc6b3a]

Mergeable *max*-heap, merge/pop $O(\log n)$, keeps size/sum. Merge disjoint, uniquely owned new nodes; pop deletes the nonempty root. No lazy tags.

```
struct
     node { // max-heap; merge/pop O(log n); pop nonempty
  // Disjoint new nodes, unique ownership; sum ops fit ll.
  ll v, data, sz, sum; node *l, *r;
  node(ll k) : v(0), data(k), sz(1), sum(k), l(0), r(0) {}
};
ll sz(node *p) { return p ? p->sz : 0; }
ll V(node *p) { return p ? p->v : -1; }
ll sum(node *p) { return p ? p->sum : 0; }
node *merge(node *a, node *b) {
  if (!a || !b) return a ? a : b;
  if (a->data < b->data) swap(a, b);
  a->r = merge(a->r, b);
  if (V(a->r) > V(a->l)) swap(a->r, a->l);
  a->v = V(a->r) + 1, a->sz = sz(a->l) + sz(a->r) + 1;
  a->sum = sum(a->l) + sum(a->r) + a->data; return a;
}
void
.. pop(node *&o) { delete exchange(o,merge(o->l,o->r)); }
node *a = new node(x), *b = new node(y); // MAX-heap
a = merge(a, b); // O(log n)
ll top = a->data; pop(a);
// a->sz, a->sum kept per subtree
```

3.4 Dynamic 2D Segment Tree [b634b6]

Rectangle add / rectangle sum, initially zero. 0-base, half-open coordinates; $O(\log n \log m)$ per operation. Both dimensions allocate nodes dynamically. sum includes updates intersecting the node; lazy records updates covering its full x interval. **Uses int: large sums may overflow.**

```
// n,m>=0; rectangle add/sum, initially zero.
// 0-base, half-open in-domain ranges; O(log n log m).
// All int intermediates fit; free nodes manually.
// Helpers use node [l,r); apply/push require l<r.
struct Seg { // 1D range add/sum; query allocates no nodes
  struct Node {
    int sum = 0, lazy = 0;
    Node *l = nullptr, *r = nullptr;
  };
  int n; Node *root = nullptr;
  Seg(int n = 0) : n(n) {}
  void apply(Node *&idx, int l, int r, int v) {
    if (!idx) idx = new Node();
    idx->sum += v * (r - l), idx->lazy += v;
  }
  void push(Node *idx, int l, int r) {
    if (!idx || !idx->lazy || r - l == 1) return;
    int m = l + (r - l) / 2;
    apply(idx->l, l, m, idx->lazy);
    apply(idx->r, m, r, idx->lazy), idx->lazy = 0;
  }
  void update(Node *&idx, int l, int r,
              int ql, int qr, int v) {
    if (!v || ql >= qr || qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) return apply(idx, l, r, v);
    if (!idx) idx = new Node();
    idx->sum += v * (min(r, qr) - max(l, ql));
    int m = l + (r - l) / 2;
    if (ql < m) update(idx->l, l, m, ql, qr, v);
    if (qr > m) update(idx->r, m, r, ql, qr, v);
  }
  int query(Node *idx, int l, int r, int ql, int qr) {
    if (!idx || ql >= qr || qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return idx->sum;
    int m = l + (r - l) / 2;
    // Children exclude lazy; count the queried overlap.
    return idx->lazy * (min(r, qr) - max(l, ql))
         + (ql < m ? query(idx->l, l, m, ql, qr) : 0)
         + (qr > m ? query(idx->r, m, r, ql, qr) : 0);
  }
  void update(int l, int r, int v) {
    update(root, 0, n, l, r, v); }
  int query(int l, int r) {
    return query(root, 0, n, l, r); }
};
struct Seg2D {
  struct Node {
    Seg sum, lazy;
    Node *l = nullptr, *r = nullptr;
    Node(int m) : sum(m), lazy(m) {}
  };
  int n, m; Node *root = nullptr;
  Seg2D(int n, int m) : n(n), m(m) {}
  void update(Node *&idx, int l, int r,
              int xl, int xr, int yl, int yr, int v) {
    if (!v || xl >= xr || yl >= yr || xr <= l || r <= xl)
      return;
    if (!idx) idx = new Node(m);
    // sum
        counts all updates intersecting this x interval.
    idx->
        sum.update(yl, yr, v * (min(r, xr) - max(l, xl)));
    if (xl <= l && r <= xr)
      return idx->lazy.update(yl, yr, v);
    int md = l + (r - l) / 2;
    if (xl < md) update(idx->l, l, md, xl, xr, yl, yr, v);
    if (xr > md) update(idx->r, md, r, xl, xr, yl, yr, v);
  }
  int query(Node *idx, int l, int r,
            int xl, int xr, int yl, int yr) {
    if (!
        idx || xl >= xr || yl >= yr || xr <= l || r <= xl)
      return 0;
    if (xl <= l && r <= xr) return idx->sum.query(yl, yr);
    int md = l + (r - l) / 2;
    // Children
        exclude lazy, which covers this x interval.
    return idx->lazy.query(yl, yr)
         * (min(r, xr) - max(l, xl))
    + (xl < md ? query(idx->l, l, md, xl, xr, yl, yr) : 0)
    + (xr
        > md ? query(idx->r, md, r, xl, xr, yl, yr) : 0);
  }
  void update(int xl, int xr, int yl, int yr, int v) {
    update(root, 0, n, xl, xr, yl, yr, v);
  }
  int query(int xl, int xr, int yl, int yr) {
    return query(root, 0, n, xl, xr, yl, yr);
  }
};
// A: rectangle add + rectangle sum; initially all zero.
Seg2D st(n, m); // 0-based, half-open coordinates
st.update(xl, xr, yl, yr, v); // rectangle add
int sum = st.query(xl, xr, yl, yr); // rectangle sum
```

3.5 Heavy light Decomposition* [f99cac]

Vertex path sums in $O(\log^2 n)$ using Seg from the preceding template. Connected tree; set all val[u] before build(). Vertices 1-base, root 1; flattened positions half-open, 0-base.

```
struct Heavy_light_Decomposition { // 1-base, root 1
  // Needs
       Seg, N; connected tree; 1<=n<N; stack for depth.
  // init, edges, all val, build, query; int math fits.
  // Build O(n), path sum O(log^2 n); positions 0-base.
  int n, ulink[N], deep[N], mxson[N], w[N], pa[N];
  int t, pl[N], val[N]; // val: vertex data
  Seg seg; vector<int> G[N];
  void init(int _n) {
    n = _n, w[0] = 0;
    for (int i = 1; i <= n; ++i)
      G[i].clear(), mxson[i] = 0;
  }
  void add_edge(int a, int b) { G[a].pb(b), G[b].pb(a); }
  void dfs(int u, int f, int d) {
    w[u] = 1, pa[u] = f, deep[u] = d;
    for (int i : G[u]) if (i != f) {
      dfs(i, u, d + 1), w[u] += w[i];
      if (w[i] > w[mxson[u]]) mxson[u] = i;
    }
  }
  void cut_impl(int u, int link, bool values) {
    pl[u] = t++, ulink[u] = link;
    if (values) seg.update(pl[u], pl[u] + 1, val[u]);
    if (!mxson[u]) return;
    cut_impl(mxson[u], link, values);
    for (int i : G[u]) if (i != pa[u] && i != mxson[u])
      cut_impl(i, i, values);
  }
  void cut(int u, int link) { cut_impl(u, link, true); }
  void build() {
    t = 0, seg = Seg(n), dfs(1, 1, 1);
    cut_impl(1, 1, false);
    vector<int> a(n);
    for (int u = 1; u <= n; ++u) a[pl[u]] = val[u];
    auto make = [&](auto &&self, int l, int r)
        -> Seg::Node* {
      if (r - l == 1)
        return a[l] ? new Seg::Node{a[l], a[l]} : nullptr;
      int m = l + (r - l) / 2;
      auto x = self(self, l, m), y = self(self, m, r);
      if (!x && !y) return nullptr;
      return new Seg::Node{
        (x ? x->sum : 0) + (y ? y->sum : 0), 0, x, y};
    }; seg.root = make(make, 0, n);
  }
  int query(int a, int b) {
    int ta = ulink[a], tb = ulink[b], res = 0;
    while (ta != tb) {
      if (deep[ta] > deep[tb]) swap(ta, tb), swap(a, b);
      res += seg.query(pl[tb], pl[b] + 1);
      tb = ulink[b = pa[tb]];
    }
```

```
    if (pl[a] > pl[b]) swap(a, b);
    return res + seg.query(pl[a], pl[b] + 1);
  }
};
static Heavy_light_Decomposition hld; hld.init(n);
hld.add_edge(a, b); hld.val[u] = w; hld.build(); // 1-base
hld.query(a, b); // O(log^2 n)
```

3.6 Centroid Decomposition* [c919f6]

Builds a centroid tree in $O(n \log n)$; modify/query are $O(\log n)$. upinfo removes same-branch overcount. Connected tree; modify marks a copy, query sums path weights to marked copies. 1-base.

```
struct Cent_Dec { // 1-base; build O(n log n)
  // Connected tree; int weights; 1<=n<N; supply N.
  // Stack for tree depth; all ll intermediates fit.
  // init, edges, build, then modify/query O(log n).
  vector<pii> G[N];
  // (sum distance,count); upinfo subtracts same branch.
  pll info[N], upinfo[N];
  int n, pa[N], layer[N], sz[N], done[N];
  ll dis[__lg(N) + 1][N];
  void init(int _n) {
    n = _n, layer[0] = -1;
    fill_n(pa + 1, n, 0), fill_n(done + 1, n, 0);
    fill_n(info, n+1, pll()), fill_n(upinfo, n+1, pll());
    for (int i = 1; i <= n; ++i) G[i].clear();
  }
  void add_edge(int a, int b, int w) {
    G[a].pb({b, w}), G[b].pb({a, w}); }
  void get_cent(int u, int f, int &mx, int &c, int num) {
    int mxsz = 0; sz[u] = 1;
    for (auto [v, w] : G[u]) if (!done[v] && v != f) {
      get_cent(v, u, mx, c, num);
      sz[u] += sz[v], mxsz = max(mxsz, sz[v]);
    }
    if (mx > (mxsz = max(mxsz, num-sz[u])))
      mx = mxsz, c = u;
  }
  void dfs(int u, int f, ll d, int org) {
    dis[layer[org]][u] = d;
    for (auto [v, w] : G[u])
      if (!done[v] && v != f) dfs(v, u, d + w, org);
  }
  int cut(int u, int f, int num) {
    int mx = num + 1, c = 0;
    get_cent(u, f, mx, c, num);
    done[c] = 1, pa[c] = f, layer[c] = layer[f] + 1;
    dfs(c, f, 0, c); // Distances before child cuts.
    for (auto [v, w] : G[c]) if (!done[v])
      upinfo[cut(v, c,
        sz[v] > sz[c] ? num-sz[c] : sz[v])] = pll();
    return done[c] = 0, c;
  }
  void build() { cut(1, 0, n); }
  void modify(int u) { // mark one copy of u
    for (int a = u, ly = layer[a]; a; a = pa[a], --ly) {
      info[a].X += dis[ly][u], ++info[a].Y;
      if (pa[a])
        upinfo[a].X += dis[ly-1][u], ++upinfo[a].Y;
    }
  }
  ll query
      (int u) { // Sum path weights to marks; signed OK
    ll rt = 0;
    for (int a = u, ly = layer[a]; a; a = pa[a], --ly) {
      rt += info[a].X + info[a].Y * dis[ly][u];
      if (pa[a])
        rt -= upinfo[a].X + upinfo[a].Y * dis[ly-1][u];
    }
    return rt; }
};
static Cent_Dec cd; cd.init(n); cd.add_edge(a, b, w);
cd.build(); // 1-base
cd.modify(u); // mark u
ll s = cd.query(u); // sum dist to marked
```

3.7 LiChaoST* [f0c9cd]

Line container for integer $x \in [0,n)$, $O(\log n)$. Keeps the **maximum**; negate for minimum. IDs $\neq -1$; line values within $\pm$INF, arithmetic fits ll. For $n \leq 0$, insert is ignored, query returns -INF.

```
// Max on integer x in [0,n); O(log n); inserted id != -1.
// INF>0; values in [-INF,INF]; m*x and m*x+k fit ll.
// 4*n fits int/memory; n<=0: no insert, query -INF.
struct L {
  ll m, k, id; L() : id(-1) {}
  L(ll a, ll b, ll c) : m(a), k(b), id(c) {}
  ll at(ll x) { return m * x + k; }
};
class LiChao {
  int n; vector<L> nodes;
  void insert(int l, int r, int rt, L ln) {
    int m = (l + r) >> 1; L &o = nodes[rt];
    if (o.id == -1) return o = ln, void();
    if (o.at(m) < ln.at(m)) swap(o, ln);
    if (r - l == 1 || o.m == ln.m) return;
    if (ln.m < o.m) { // can win only left
      if (ln.at(l) > o.at(l)) insert(l, m, rt << 1, ln);
    } else if (ln.at(r - 1) > o.at(r - 1))
      insert(m, r, rt << 1 | 1, ln);
  } public:
  LiChao(int n_) : n(max(n_, 0)), nodes(n * 4) {}
  void insert(L ln) { if (n) insert(0, n, 1, ln); }
  ll query(ll x) {
    ll ret = -INF; int l = 0, r = n, rt = 1;
    while (l < r && nodes[rt].id != -1) {
      ret = max(ret, nodes[rt].at(x));
      if (r - l == 1) break;
      int m = (l + r) >> 1; rt = rt << 1 | (x >= m);
      if (x < m) r = m; else l = m;
    } return ret;
  }
};
LiChao lc(n); // x domain [0, n)
lc.insert(L(m, k, id)); // y = m*x + k
ll best = lc.query(x); // MAXIMUM
// for min: insert negated, negate result
```

3.8 XOR Basis [c60757]

Inserts a 64-bit unsigned integer into a linear basis in $O(64)$. basis[i] has highest set bit i; a dependent value is discarded. Zero basis before each instance.

```
// 64-bit unsigned long long; libstdc++ __lg.
unsigned long long basis[64]; // basis[i]: highest bit i.
void ins(unsigned long long x) { // O(64); zero each test
  while (x) { int i = __lg(x);
    if (!basis[i]) return void(basis[i] = x);
    x ^= basis[i];
  }
}
ins(x); // add x to the XOR basis
```

3.9 Treap [37350c]

Rotation-free treap, expected $O(\log n)$. Split key $\leq k$; split2: first k. interval(l,r): 1-base start, length. Merge disjoint ordered trees. No lazy; propagate in kth/Rank/erase too.

```
// Expected O(log n); int sizes fit, total<=RAND_MAX.
// Stack for height; uniquely owned new'ed nodes.
// merge: disjoint trees, all a keys <= all b keys.
// Split outputs distinct, may alias root.
// split: <=k | >k; split2: first k, 0<=k<=size.
// interval: 1-base l, length r; 1<=l<=size+1,
// 0<=r<=size-l+1.
struct node {
  int data, sz, priority; node *l, *r;
  node(int k) : data(k), sz(1), priority(rand()),
    l(0), r(0) {}
  void
      up() { sz = 1 + (l ? l->sz : 0) + (r ? r->sz : 0); }
  void down() {} // Lazy: also down before kth/Rank/erase
};
int sz(node *a) { return a ? a->sz : 0; }
node *merge(node *a, node *b) {
  if (!a || !b) return a ? a : b;
  if (a->priority < b->priority)
    return a->down(), a->r = merge(a->r, b), a->up(), a;
  return b->down(), b->l = merge(a, b->l), b->up(), b;
}
void split(node *o, node *&a, node *&b, int k) {
  if (!o) return a = b = 0, void();
  o->down();
  if (o->data <= k)
    a = o, split(o->r, a->r, b, k), a->up();
  else b = o, split(o->l, a, b->l, k), b->up();
}
void split2(node *o, node *&a, node *&b, int k) {
  if (sz(o) <= k) return a = o, b = 0, void();
  o->down();
  if (sz(o->l) < k)
    a = o, split2(o->r, a->r, b, k - sz(o->l) - 1);
  else b = o, split2(o->l, a, b->l, k);
  o->up();
}
node *kth(node *o, int k) { // 1 <= k <= size
  int s = sz(o->l) + 1; if (k == s) return o;
  return k < s ? kth(o->l, k) : kth(o->r, k - s);
}
int Rank(node *o, int key) { // count keys < key
  if (!o) return 0;
  return o->data < key ? sz(o->l) + 1 + Rank(o->r, key)
                       : Rank(o->l, key);
}
bool erase(node *&o, int k) { // erase one copy
  if (!o) return 0;
```

```
  if (o->data == k) { node *t = o;
    o->down(), o = merge(o->l, o->r);
    delete t; return 1;
  }
  node *&t = k < o->data ? o->l : o->r;
  return erase(t, k) ? o->up(), 1 : 0;
}
void insert_node(node *&o, node *p) { // fresh singleton
  if (!o || p->priority < o->priority)
    return split(o, p->l, p->r, p->data), o = p,
      o->up(), void();
  o->down();
  insert_node
      (p->data < o->data ? o->l : o->r, p); o->up();
}
void insert(node *&o, int k) {
  insert_node(o, new node(k)); }
void interval(node *&o, int l, int r) { // start l, len r
  node *a, *b, *c;
  split2(o, a, b, l - 1), split2(b, b, c, r);
  o = merge(a, merge(b, c)); // modify b before merging
}
node *root = nullptr; insert(root, k);
// Or insert_node(root, new node(k)).
node *t = kth(root, i); // i-th, 1-base
int r = Rank(root, key); // # less than key
interval(root, l, len); // 1-base start, length len
```

4 Flow/Matching

4.1 Bipartite Matching* [f8a01b]

Max bipartite matching, Hopcroft-Karp. $O((E+V)\sqrt{V})$. 0-base; unmatched: mp=-1, mq=l.

```
struct Bipartite_Matching { // 0-base; 0<=l,r<=N
  // O((E+V)sqrt(V)); valid endpoints; DFS stack for path.
  int mp[N], mq[N], dis[N + 1], cur[N], l, r;
  vector<int> G[N + 1];
  bool dfs(int u) {
    for (int &i = cur[u]; i < SZ(G[u]); ++i) {
      int e = G[u][i];
      if (dis[mq[e]] == dis[u] + 1 &&
          (mq[e] == l || dfs(mq[e])))
        return mp[mq[e] = u] = e, 1;
    } return dis[u] = -1, 0;
  }
  bool bfs() {
    fill_n(dis, l + 1, -1); queue<int> q;
    for (int i = 0; i < l; ++i)
      if (!~mp[i]) q.push(i), dis[i] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      if (~dis[l] && dis[u] >= dis[l]) break;
      for (int e : G[u]) if (!~dis[mq[e]])
        q.push(mq[e]), dis[mq[e]] = dis[u] + 1;
    } return dis[l] != -1;
  }
  int matching() {
    int res = 0; fill_n(mp, l, -1), fill_n(mq, r, l);
    while (bfs()) {
      fill_n(cur, l, 0);
      for (
          int i = 0; i < l; ++i) res += !~mp[i] && dfs(i);
    } return res;
  }
  void add_edge(int s, int t) { G[s].pb(t); }
  void init(int _l, int _r) { l = _l, r = _r;
    for (int i = 0; i <= l; ++i) G[i].clear(); }
};
static Bipartite_Matching bm; bm.init(nl, nr); // 0-base
bm.add_edge(l, r);
int m = bm.matching();
// pair (i, bm.mp[i]) when bm.mp[i] != -1
```

4.2 Kuhn Munkres* [3f9429]

Max weight *perfect* matching on a square bipartite graph, $O(V^3)$. Must exist; missing edges $= -\text{INF}, \text{INF} > 2n\max|w|$. 0-base.

```
struct KM { // Max weight perfect matching; O(n^3)
  // 0<=n<=N; 0-based; missing=-INF; arithmetic fits ll.
  // INF > 2*n*max|real w|; a perfect matching must exist.
  ll w[N][N], hl[N], hr[N], slk[N];
  int fl[N], fr[N], pre[N], qu[N], ql, qr, n;
  bool vl[N], vr[N];
  void init(int _n) { n = _n;
    for (int i = 0; i < n; ++i) fill_n(w[i], n, -INF);
  }
  void add_edge(int a, int b, ll wei) { w[a][b] = wei; }
  bool Check(int y) { // right y; queue stores left
    if (vr
        [y] = 1, ~fr[y]) return vl[qu[qr++] = fr[y]] = 1;
    while (~y) swap(y, fl[fr[y] = pre[y]]); return 0;
  }
  void bfs(int s) { // unmatched left root
    fill_n(slk, n, INF);
    fill_n(vl, n, 0), fill_n(vr, n, 0);
    ql = qr = 0, qu[qr++] = s, vl[s] = 1;
    for (ll d;;) {
      while (ql < qr)
        for (int y = 0, x = qu[ql++]; y < n; ++y)
          if (!vr[y] &&
              slk[y] >= (d = hl[x] + hr[y] - w[x][y])) {
            if (pre[y] = x, d) slk[y] = d;
            else if (!Check(y)) return;
          } d = INF;
      for (int x = 0; x < n; ++x)
        if (!vr[x]) d = min(d, slk[x]);
      for (int x = 0; x < n; ++x) {
        if (vr[x]) hr[x] += d;
        else slk[x] -= d;
        if (vl[x]) hl[x] -= d;
      } for (int x = 0; x < n; ++x)
        if (!vr[x] && !slk[x] && !Check(x)) return;
    }
  }
  ll solve() {
    fill_n
        (fl, n, -1), fill_n(fr, n, -1), fill_n(hl, n, 0);
    fill_n(hr, n, -INF);
    for (
        int i = 0; i < n; ++i) for (int j = 0; j < n; ++j)
      hr[j] = max(hr[j], w[i][j]);
    for (int i = 0; i < n; ++i) bfs(i);
    ll res = 0;
    for (int i = 0; i < n; ++i) res += w[i][fl[i]];
    return res;
  }
};
static KM km; km.init(n); // square; perfect matching
km.add_edge(i, j, w); // min: use -w; missing=-INF
ll best = km.solve();
// km.fl: left -> right; km.fr: right -> left
```

4.3 MincostMaxflow* [767b3c]

Min cost max flow, initial SPFA + Johnson potentials + Dijkstra (handles negative-cost edges). For A augmentations, $O(VE + A(E+V)\log(E+V))$. For s != t, the entire residual graph must have no negative-cost cycle; each solve returns additional flow/cost (the degenerate s==t query returns flow=cost=0). Nonnegative capacities; distances < INF, arithmetic fits ll. 0-base.

```
// Init 0<=n<=N; IDs [0,n); cap>=0.
// s!=t: no negative residual cycles anywhere.
// INF>0; ll math fits; finite BF/reduced distances < INF.
// A augmentations: O(VE + A(E+V)log(E+V)).
struct MinCostMaxFlow {
  struct Edge { int from, to; ll cap, flow, cost;
    int rev; } *past[N];
  vector<Edge> G[N];
  bool inq[N]; int n, s, t;
  ll dis[N], up[N], pot[N];
  bool BellmanFord() {
    fill_n(dis, n, INF), fill_n(inq, n, 0); queue<int> q;
    dis[s] = 0, up[s] = LLONG_MAX, past[s] = nullptr;
    q.push(s), inq[s] = 1;
    while (!q.empty()) {
      int u = q.front(); q.pop(), inq[u] = 0;
      for (auto &e : G[u])
        if (e.cap > e.flow && dis[e.to] > dis[u]+e.cost) {
          dis[e.to] = dis[u] + e.cost, past[e.to] = &e;
          up[e.to] = min(up[u], e.cap - e.flow);
          if (!inq[e.to]) q.push(e.to), inq[e.to] = 1;
        }
    } return dis[t] != INF;
  }
  bool Dijkstra() {
    fill_n(dis, n, INF); using State = pair<ll, int>;
    priority_queue
        <State, vector<State>, greater<State>> q;
    dis[s] = 0, up[s] = LLONG_MAX, past[s] = nullptr;
    q.push({0, s});
    while (!q.empty()) {
      auto [d, u] = q.top(); q.pop();
      if (d != dis[u]) continue;
      for (auto &e : G[u]) if (e.cap > e.flow) {
        ll nd = d + e.cost + pot[u] - pot[e.to];
        if (nd < dis[e.to]) {
          dis[e.to] = nd, past[e.to] = &e;
          up[e.to] = min(up[u], e.cap - e.flow);
          q.push({nd, e.to});
        }
      }
    } return dis[t] != INF;
  }
  void solve(int _s, int _t, ll &flow, ll &cost,
```

```
    bool neg = true) {
    s = _s, t = _t, flow = cost = 0; // added flow/cost
    if (s == t) return; // zero-length flow
    // Rebuild potentials each call, including neg=false.
    fill_n(pot, n, 0); bool need_bf = neg;
    if (!neg) for (int u = 0; u < n; ++u)
      for (auto &e : G[u])
        need_bf |= e.cap > e.flow && e.cost < 0;
    if (need_bf ? !BellmanFord() : !Dijkstra()) return;
    do { // Reuse the initial shortest path.
      flow += up[t];
      cost += up[t] * (dis[t] + pot[t] - pot[s]);
      for (int i = 0; i < n; ++i)
        if (dis[i] != INF) pot[i] += dis[i];
      for (int i = t; past[i]; i = past[i]->from) {
        auto &e = *past[i]; e.flow += up[t];
        G[e.to][e.rev].flow -= up[t];
      }
    } while (Dijkstra());
  }
  void init(int _n) {
    n = _n, fill_n(pot, n, 0);
    for (int i = 0; i < n; ++i) G[i].clear();
  }
  void add_edge(ll a, ll b, ll cap, ll cost) {
    G[a].pb({int(a), int(b), cap, 0, cost,
      SZ(G[b]) + (a == b)});
    G[b].pb({int(b), int(a), 0, 0, -cost, SZ(G[a]) - 1});
  }
};
static MinCostMaxFlow mcmf; mcmf.init(n);
mcmf.add_edge(u, v, cap, cost);
ll flow, cost;
mcmf.solve(s, t, flow, cost); // SPFA + Johnson + Dijkstra
// s!=t: no negative residual cycles anywhere.
// Additional flow/cost per call; s==t returns both zero.
// neg=false skips SPFA only if active residual costs >=0.
```

4.4 Maximum Simple Graph Matching* [08e0b6]

Max matching on a loopless undirected graph (blossom). $O(V^3)$. solve() returns added pairs; unmatched is n. 0-base.

```
struct Matching { // Undirected, loopless; 0-base; free=n
  // n>=0; int math fits; O(n^3) with O(n^2) entries.
  queue<int> q; int n;
  vector<int> fa, s, vis, pre, match;
  vector<vector<int>> G;
  int Find(int u) { return u == fa[u] ? u :
    fa[u] = Find(fa[u]); }
  int LCA(int x, int y) {
    static int tk = 0; tk++; x = Find(x); y = Find(y);
    for (;; swap(x, y)) if (x != n) {
      if (vis[x] == tk) return x;
      vis[x] = tk, x = Find(pre[match[x]]);
    }
  }
  void Blossom(int x, int y, int l) {
    for (; Find(x) != l; x = pre[y]) {
      pre[x] = y, y = match[x];
      if (s[y] == 1) q.push(y), s[y] = 0;
      for (int z: {x, y}) if (fa[z] == z) fa[z] = l;
    }
  }
  bool Bfs(int r) { if (G[r].empty()) return false;
    for (int u : G[r]) if (match[u] == n)
      return match[r] = u, match[u] = r, true;
    iota(ALL(fa), 0); fill(ALL(s), -1);
    q = {}; q.push(r); s[r] = 0;
    for (; !q.empty(); q.pop()) {
      int x = q.front(); for (int u : G[x])
        if (s[u] == -1) {
          if (pre[u] = x, s[u] = 1, match[u] == n) {
            for (int a = u, b = x, last; b != n;
                 a = last, b = pre[a])
              last = match[b], match[b] = a, match[a] = b;
            return true;
          } q.push(match[u]); s[match[u]] = 0;
        } else if (!s[u] && Find(u) != Find(x)) {
          int l = LCA(u, x);
          Blossom(x, u, l); Blossom(u, x, l);
        }
    } return false;
  }
  Matching(int _n) : n(_n), fa(n + 1), s(n + 1),
    vis(n + 1), pre(n + 1, n), match(n + 1, n), G(n) {}
  void add_edge(int u, int v) { G[u].pb(v), G[v].pb(u); }
  int solve() { // Valid matching -> added pairs.
    int ans = 0;
    for (int x = 0; x < n; ++x)
      if (match[x] == n) ans += Bfs(x);
    return ans;
  }
};
Matching M(n); // general graph
M.add_edge(u, v);
int sz = M.solve();
// M.match[x] == n  =>  x is unmatched
```

4.5 Maximum Weight Matching* [705e62]

Weighted blossom, $O(V^3)$, 1-base; loopless, uses $w > 0$ only. int arithmetic must fit. Returns (weight, pairs); ties need not maximise pairs.

```
struct
    WeightGraph { // Loopless undirected; 1-base; O(n^3)
  // n>=0; w>0 only; all int arithmetic (incl. 2*w) fits.
  struct edge { int u, v, w; }; int n, nx;
  vector<int> lab, slk, match, st, pa, S, vis;
  vector<vector<edge>> g;
  vector<vector<int>> flo, flo_from; queue<int> q;
  WeightGraph(int n_) : n(n_), nx(n * 2), lab(nx + 1),
    slk(nx + 1), g(nx + 1, vector<edge>(nx + 1)),
    flo(nx + 1), flo_from(nx + 1, vector(n + 1, 0)) {
    match = st = pa = S = vis = slk;
    for (int u = 1; u <= n; ++u)
      for (int v = 1; v <= n; ++v)
        g[u][v] = {u, v, 0};
  }
  int E(edge e) {
    return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
  void update_slk(int u, int x, int &s) {
    if (!s || E(g[u][x]) < E(g[s][x])) s = u; }
  void set_slk(int x) {
    slk[x] = 0;
    for (int u = 1; u <= n; ++u)
      if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
        update_slk(u, x, slk[x]);
  }
  void q_push(int x) {
    if (x <= n) q.push(x);
    else for (int y : flo[x]) q_push(y); }
  void set_st(int x, int b) {
    st[x] = b;
    if (x > n) for (int y : flo[x]) set_st(y, b); }
  vector<int> split_flo(vector<int> &f, int xr) {
    auto it = find(ALL(f), xr);
    if (auto pr = it - f.begin(); pr % 2 == 1)
      reverse(1 + ALL(f)), it = f.end() - pr;
    auto res = vector(f.begin(), it);
    return f.erase(f.begin(), it), res;
  }
  void set_match(int u, int v) {
    match[u] = g[u][v].v;
    if (u <= n) return;
    int xr = flo_from[u][g[u][v].u];
    auto &f = flo[u], z = split_flo(f, xr);
    for (int i = 0; i < SZ(z); ++i)
      set_match(z[i], z[i ^ 1]);
    set_match(xr, v); f.insert(f.end(), ALL(z));
  }
  void augment(int u, int v) {
    for (;;) {
      int xnv = st[match[u]]; set_match(u, v);
      if (!xnv) return;
      set_match(v = xnv, u = st[pa[xnv]]); }
  }
  int lca(int u, int v) {
    static int t = 0;
    for (++t; u || v; swap(u, v)) if (u) {
      if (vis[u] == t) return u;
      vis[u] = t, u = st[match[u]];
      if (u) u = st[pa[u]];
    } return 0;
  }
  void add_blossom(int u, int o, int v) {
    int b = find(n + 1 + ALL(st), 0) - begin(st);
    lab[b] = 0, S[b] = 0, match[b] = match[o];
    auto &f = flo[b]; f.clear(); f.pb(o);
    for (int t : {u, v}) {
      reverse(1 + ALL(f));
      for (int x = t, y; x != o; x = st[pa[y]])
        f.pb(x), f.pb(y = st[match[x]]), q_push(y);
    } set_st(b, b);
    for (int x = 1; x <= nx; ++x)
      g[b][x].w = g[x][b].w = 0;
    fill(ALL(flo_from[b]), 0);
    for (int xs : flo[b]) {
      for (int x = 1; x <= nx; ++x)
        if (g[b][x].w == 0 || E(g[xs][x]) < E(g[b][x]))
          g[b][x] = g[xs][x], g[x][b] = g[x][xs];
      for (int x = 1; x <= n; ++x)
        if (flo_from[xs][x]) flo_from[b][x] = xs;
    } set_slk(b);
```

```
  }
  void expand_blossom(int b) {
    for (int x : flo[b]) set_st(x, x);
    int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
    for (int x : split_flo(flo[b], xr)) {
      if (xs == -1) { xs = x; continue; }
      pa[xs] = g[x][xs].u, S[xs] = 1, S[x] = 0;
      slk[xs] = 0, set_slk(x), q_push(x), xs = -1;
    } for (int x : flo[b])
      if (x == xr) S[x] = 1, pa[x] = pa[b];
      else S[x] = -1, set_slk(x);
    st[b] = 0;
  }
  bool on_found_edge(const edge &e) {
    if (int u = st[e.u], v = st[e.v]; S[v] == -1) {
      int nu = st[match[v]]; pa[v] = e.u, S[v] = 1;
      slk[v] = slk[nu] = S[nu] = 0; q_push(nu);
    } else if (S[v] == 0) {
      if (int o = lca(u, v)) add_blossom(u, o, v);
      else return augment(u, v), augment(v, u), true;
    } return false;
  }
  bool matching() {
    fill(ALL(S), -1), fill(ALL(slk), 0); q = {};
    for (int x = 1; x <= nx; ++x)
      if (st[x] == x && !match[x])
        pa[x] = S[x] = 0, q_push(x);
    if (q.empty()) return false;
    for (;;) {
      while (!q.empty()) {
        int u = q.front(); q.pop();
        if (S[st[u]] == 1) continue;
        for (int v = 1; v <= n; ++v)
          if (g[u][v].w > 0 && st[u] != st[v]) {
            if (E(g[u][v]))
              update_slk(u, st[v], slk[st[v]]);
            else if (on_found_edge(g[u][v])) return true;
          }
      } int d = 0x3f3f3f3f;
      for (int b = n + 1; b <= nx; ++b)
        if (st[b] == b && S[b] == 1)
          d = min(d, lab[b] / 2);
      for (int x = 1; x <= nx; ++x)
        if (int s = slk[x]; st[x] == x && s && S[x] <= 0)
          d = min(d, E(g[s][x]) / (S[x] + 2));
      for (int u = 1; u <= n; ++u)
        if (S[st[u]] == 1) lab[u] += d;
        else if (S[st[u]] == 0) {
          if (lab[u] <= d) return false;
          lab[u] -= d;
        }
      for (int b = n + 1; b <= nx; ++b)
        if (st[b] == b && S[b] >= 0)
          lab[b] += d * (2 - 4 * S[b]);
      for (int x = 1; x <= nx; ++x)
        if (int s = slk[x]; st[x] == x && s && st[s] != x
            && E(g[s][x]) == 0 && on_found_edge(g[s][x]))
          return true;
      for (int b = n + 1; b <= nx; ++b)
        if (st[b] == b && S[b] == 1 && lab[b] == 0)
          expand_blossom(b);
    }
  }
  pair<
      ll, int> solve() { // Rebuild; ties: any pair count.
    fill(ALL(match), 0), fill(ALL(st), 0); // Reset.
    for (int u = 0; u <= n; ++u)
      st[u] = flo_from[u][u] = u;
    int w_max = 0, n_matches = 0; ll tot_weight = 0;
    for (int u = 1; u <= n; ++u)
      for (int v = 1; v <= u; ++v)
        w_max = max(w_max, g[u][v].w);
    fill(
        ALL(lab), w_max); // Tight seeds; n/2 are optimal.
    for (int u = 1; u <= n; ++u) if (w_max && !match[u])
      for (int v = u + 1; v <= n; ++v)
        if (!match[v] && g[u][v].w == w_max) {
          match
              [u] = v, match[v] = u, ++n_matches; break; }
    if (n_matches < n / 2) while (matching()) ++n_matches;
    for (int u = 1; u <= n; ++u)
      if (match[u] && match[u] < u)
        tot_weight += g[u][match[u]].w;
    return {tot_weight, n_matches};
  }
  void add_edge(int u, int v, int w) {
    g[u][v].w = g[v][u].w = w; }
};
WeightGraph g(n); // 1-based
g.add_edge(u, v, w);
auto [sum, cnt] = g.solve();
// total weight, number of matched pairs
// solve
    () rebuilds from the current weights on every call.
```

4.6 SW-mincut [df8117]

Global min cut, Stoer-Wagner. $O(V^3)$, adjacency matrix, undirected nonnegative weights. Sums fit int; solve mutates the matrix. 0-base.

```
struct SW { // 0-base, undirected, weights>=0. O(V^3)
  // 0<=
      n<=MXN; cuts fit int; loops ignored; solve mutates.
  static const int MXN = 514, INF = 2147483647;
  int vst[MXN], edge[MXN][MXN], wei[MXN];
  void init(int n) { for (int i = 0; i < n; ++i)
    fill_n(edge[i], n, 0); }
  void addEdge(int u, int v, int w) {
    if (u != v) edge[u][v] += w, edge[v][u] += w; }
  int search(int &s, int &t, int n) { // n>=1; one phase
    fill_n(vst, n, 0), fill_n(wei, n, 0);
    s = t = -1; int mx, cur;
    for (int j = 0; j < n; ++j) {
      cur =
            max_element(wei, wei + n) - wei, mx = wei[cur];
      vst[cur] = 1, wei[cur] = -1, s = t, t = cur;
      for (int i = 0; i < n; ++i)
        wei[i] += !vst[i] * edge[cur][i];
    } return mx;
  }
  int solve(int n) {
    int res = n <= 1 ? 0 : INF; // empty cut for n <= 1
    for (int x, y; n > 1; --n) {
      res = min(res, search(x, y, n)); if (!res) break;
      for (int i = 0; i < n; ++i)
        if (i != x && i != y)
          edge[i][x] = (edge[x][i] += edge[y][i]);
      for (int i = 0; i < n; ++i)
        edge[y][i]=edge[n-1][i], edge[i][y]=edge[i][n-1];
      edge[y][y] = 0;
    } return res;
  }
} sw;
sw.init(n); // undirected, adj matrix
sw.addEdge(u, v, w); // weights accumulate
int cut = sw.solve(n); // global min cut
```

4.7 BoundedFlow*(Dinic*) [ab42c0]

Flow with $0 \le lo \le hi$; circulation or max nonnegative flow (-1 if infeasible). Reuses Dinic's MaxFlow; MAXN$\ge n+3$. INF bounds capacities/positive demand sum; int arithmetic fits. Solve once; mutates graph. 0-base.

```
// Needs MaxFlow (Dinic.cpp); MAXN>= n+3; 0 <= lo <= hi.
// int math fits; INF >= capacity and total positive cnt.
struct BoundedFlow : MaxFlow { // 0-base; n>=0; solve once
  int vn, cnt[MAXN];
  void add_edge(int u, int v, int cap) { // lower bound 0
    int id = SZ(G[u]); MaxFlow::add_edge(u, v, cap);
    G[u][
        id].rev += u == v; // Fix self-loop reverse index.
  }
  void init(int _n) {
    vn = _n, MaxFlow::init(vn + 3); fill_n(cnt, vn, 0);
  }
  void add_edge(int u, int v, int lo, int hi) {
    cnt[u] -= lo, cnt[v] += lo; int id = SZ(G[u]);
    add_edge(u, v, hi); G[u][id].flow = lo;
  }
  bool solve() { // Mutates graph, even on failure.
    int sum = 0;
    for (int i = 0; i < vn; ++i)
      if (cnt[i] > 0)
        add_edge(vn + 1, i, cnt[i]), sum += cnt[i];
      else if (cnt[i] < 0) add_edge(i, vn + 2, -cnt[i]);
    if (sum != maxflow(vn + 1, vn + 2)) sum = -1;
    for (int i = 0; i < vn; ++i)
      if (cnt[i]) G[i].pop_back();
    G[vn + 1].clear(), G[vn + 2].clear(); // Demand arcs.
    return sum != -1;
  }
  // Max nonnegative flow; -1 if infeasible.
  int solve(int _s, int _t) {
    if (_s == _t) return solve() ? 0 : -1; // Circulation.
    add_edge(_t, _s, INF);
    if (!solve()) return -1;
    int x = G[_t].back().flow; // Drop balancing arc.
    G[_t].pop_back(), G[_s].pop_back();
    return x + maxflow(_s, _t);
  }
};
// Include Dinic first; MAXN>= n+3.
static BoundedFlow bf; bf.init(n);
bf.add_edge(u, v, lo, hi); // lower bound lo
```

```
bool feasible = bf.solve(); // Circulation; solve once.
// Or bf.solve(s, t): max nonnegative flow; -1 on failure.
```

4.8 Gomory Hu tree* [7a78ec]

All-pairs min cut as a tree ($n-1$ max-flow runs, Gusfield). Needs a MaxFlow named Dinic, built with symmetric nonnegative capacities. 0-base.

```
MaxFlow Dinic;
int g[MAXN], gh_w[MAXN];
// 0-base; 0<=n=Dinic.n<=MAXN; n-1 flows if n>0.
// Symmetric caps>=0; INF>0; int math and DFS stack fit.
void GomoryHu(int n) {
  fill_n(g, n, 0), fill_n(gh_w, n, 0);
  for (int i = 1; i < n; ++i) {
    Dinic.reset();
    int p = g[i], cut = gh_w[i] = Dinic.maxflow(i, p);
    for (int j = i + 1; j < n; ++j)
      if (g[j] == p && ~Dinic.dis[j]) g[j] = i;
    if (p && ~Dinic.dis[g[p]])
      g[i] = g[p], g[p] = i,
      gh_w[i] = gh_w[p], gh_w[p] = cut;
  }
}
Dinic.init(n); // build graph in Dinic first
GomoryHu(n); // g[i]: parent, gh_w[i]: weight to g[i]
// mincut(u,v) = min gh_w[x] on the tree path u-v
```

4.9 Minimum Cost Circulation* [576bf7]

Min cost circulation via capacity scaling; cancels negative cycles. $O(V(E+1)^2\log(C+2))$. Call solve(mxlg) once: highest bit of max capacity (0 if all zero). 0-base.

```
// 0-base; solve once; O(V(E+1)^2 log(C+2)).
struct MinCostCirculation {
  // 0<=n<=N; valid endpoints, cap>=0; adj sizes fit int.
  // Finite dis<INF; all arithmetic (even -cost) fits ll.
  struct Edge { int from, to;
    ll cap, fcap, flow, cost; int rev; };
  Edge *past[N]; vector<Edge> G[N];
  ll dis[N]; bool inq[N]; int n;
  void BellmanFord(int s) {
    fill_n(dis, n, INF), fill_n(inq, n, 0); queue<int> q;
    dis[s] = 0, past[s] = nullptr, q.push(s), inq[s] = 1;
    while (!q.empty()) {
      int u = q.front(); q.pop(), inq[u] = 0;
      for (auto &e : G[u])
        if (e.cap > e.flow && dis[e.to] > dis[u]+e.cost) {
          dis[e.to] = dis[u] + e.cost, past[e.to] = &e;
          if (!inq[e.to]) q.push(e.to), inq[e.to] = 1;
        }
    }
  }
  void try_edge(Edge &cur) {
    if (cur.cap > cur.flow) return ++cur.cap, void();
    if (cur.from == cur.to) // Empty return path.
      dis[cur.from] = 0, past[cur.from] = nullptr;
    else BellmanFord(cur.to);
    if (dis[cur.from] != INF &&
        dis[cur.from] + cur.cost < 0) {
      ++cur.flow, --G[cur.to][cur.rev].flow;
      for (int i = cur.from; past[i]; i = past[i]->from) {
        auto &e = *past[i];
        ++e.flow, --G[e.to][e.rev].flow;
      }
    } ++cur.cap;
  }
  void solve(int mxlg) { // max cap top bit; 0 if all zero
    for (int b = mxlg; b >= 0; --b) {
      for (int i = 0; i < n; ++i)
        for (auto &e : G[i]) e.cap *= 2, e.flow *= 2;
      for (int i = 0; i < n; ++i)
        for (auto &e : G[i])
          if (e.fcap >> b & 1) try_edge(e);
    }
  }
  void init(int _n) {
    n = _n; for (int i = 0; i < n; ++i) G[i].clear(); }
  void add_edge(ll a, ll b, ll cap, ll cost) {
    G[a].pb({int(a), int(b), 0, cap, 0, cost,
             SZ(G[b]) + (a == b)});
    G[b].pb({int(b), int(a), 0, 0, 0, -cost, SZ(G[a])-1});
  }
} mcmf;
mcmf.init(n); // negative costs allowed
mcmf.add_edge(u, v, cap, cost);
mcmf.solve(mxlg); // once after building the graph
// mxlg = max capacity's highest set bit; 0 if all zero
```

4.10 matching

- 若圖無孤立點，最大匹配數+最小邊覆蓋數$=|V|$。
- 任意有限圖皆有最大獨立集大小+最小點覆蓋大小$=|V|$。
- 在二分圖中，最大匹配數=最小點覆蓋大小（Kőnig定理）。
- 對DAG的最小頂點不交路徑覆蓋，建二分圖；每條$u\to v$加邊$u_L\to v_R$，數量$=|V|-$最大匹配數。

5 String

5.1 KMP [ee9e1e]

Pattern search, $O(|A|+|B|)$. Prints every 1-based occurrence; an empty pattern matches all $|A|+1$ boundaries. nxt is the prefix-function table.

```
// 1-base; empty t: n+1 matches; |s|+1, |t| fit int.
int n=s.size(), m=t.size(); vector<int> nxt(m); // O(n+m)
if (!
    m) for (int i = 0; i <= n; ++i) cout << i + 1 << '\n';
else {
  for (int i = 1, j = 0; i < m; i++) {
    while (j && t[i] != t[j]) j = nxt[j - 1];
    nxt[i] = (j += t[i] == t[j]);
  } for (int i = 0, j = 0; i < n; i++) {
    while (j && s[i] != t[j]) j = nxt[j - 1];
    j += s[i] == t[j];
    if (j == m) cout << i - m + 2 << '\n', j = nxt[j - 1];
  }
}
string s = "ababa", t = "aba";
// run the listing: prints 1 and 3
// nxt[] is the prefix-function table
```

5.2 Z-value* [7baf18]

$z[i]=\text{lcp}(s,s[i..])$ in $O(n)$. **$z[0]$ is left unset.**

```
int z[MAXn]; // O(n); z[0] kept; |s|<=min(MAXn,INT_MAX)
void make_z(const string &s) {
  for (int i = 1, l = 0, r = 0; i < SZ(s); ++i) {
    z[i] = i < r ? min(r - i, z[i - l]) : 0;
    while (i + z[i] < SZ(s) && s[i + z[i]] == s[z[i]])
      ++z[i];
    if (i + z[i] > r) l = i, r = i + z[i]; } }
make_z(s); // z[i] = lcp(s, s[i..])
// z[0] is NOT set; assign SZ(s) if needed
```

5.3 Manacher* [35a947]

All palindromic radii in $O(n)$ by interleaving %. Need MAXN$\geq 2n+1$. 0-base centre i: $z[2i+1]/2$; gap $i,i+1$: $z[2i+2]/2$.

```
int z[MAXN]; // O(n); 0-base; MAXN >= 2*|tmp|+1
// Center i: z[2*i+1]/2; gap i,i+1: z[2*i+2]/2.
// Index arithmetic fits int; aba, abba have radius 2.
void Manacher(const string &tmp) {
  string s(2 * SZ(tmp) + 1, '%');
  for (int i = 0; i < SZ(tmp); ++i) s[2 * i + 1] = tmp[i];
  for (int i = 0, l = 0, r = 0; i < SZ(s); ++i) {
    z[i] = r > i ? min(z[2 * l - i], r - i) : 1;
    while (i - z[i] >= 0 && i + z[i] < SZ(s) &&
      s[i + z[i]] == s[i - z[i]]) ++z[i];
    if (z[i] + i > r) r = z[i] + i, l = i; }
}
Manacher(s); // const string&; need MAXN >= 2*SZ(s)+1
// centre i       -> radius z[i*2+1]/2
// centre i,i+1   -> radius z[i*2+2]/2
```

5.4 SAIS* [fe1cef]

Suffix array plus Kasai height, $O(n+A)$, $A=1+\max s$. **Needs C++20**; positive ints (unsigned bytes for strings). Direct sais: final 0 sentinel.

```
// sais: nonempty positive random-access ints + final 0.
// C++20; size, A=1+max(s) fit int; O(n+A) time/space.
// sa
//   : suffix order; ra: inverse; hi[i]: LCP(sa[i],sa[i-1])
auto sais(const auto &s) {
  int n = SZ(s); if (n == 1) return vector{0};
  int z = ranges::max(s) + 1;
  vector<int> c(z), sa(n), lms, q(n/2+1);
  vector<bool> t(n, true);
  for (int x : s) ++c[x]; partial_sum(ALL(c), c.begin());
  for (int i = n - 2; i >= 0; --i)
    t[i] = s[i] == s[i+1] ? t[i+1] : s[i] < s[i+1];
  auto is_lms = views::filter([&](int i) {
    return i && t[i] && !t[i-1]; });
  auto induce = [&] {
    for (auto x = c; int y : sa)
      if (y-- && !t[y]) sa[x[s[y]-1]++] = y;
    for (auto x = c; int y : sa | views::reverse)
      if (y-- && t[y]) sa[--x[s[y]]] = y;
  }; lms.reserve(n/2); // nonadjacent LMS: q uses i/2
  for (auto x = c; int i : views::iota(0, n) | is_lms)
    q[i/2] = SZ(lms), lms.pb(sa[--x[s[i]]] = i);
  induce(); int nz = 0; vector<int> ns(SZ(lms)), nsa;
  for (int j = -1; int i : sa | is_lms) {
    if (j >= 0) {
      int len = min({n-i, n-j, lms[q[i/2]+1]-i});
      nz += lexicographical_compare(
        begin(s)+j, begin(s)+j+len,
        begin(s)+i, begin(s)+i+len);
    } ns[q[i/2]] = nz, j = i; }
  if (nz + 1 == SZ(lms)) { // Unique names: invert.
    nsa.resize(SZ(lms));
    for (int i = 0; i < SZ(lms); ++i) nsa[ns[i]] = i;
  } else nsa = sais(ns); fill(ALL(sa), 0);
  for (auto x = c; int y : nsa | views::reverse)
```

```
    y = lms[y], sa[--x[s[y]]] = y;
  return induce(), sa; }
struct Suffix { // _s[0,n)>0; 0<=n<INT_MAX; empty OK
  int n; vector<int> sa, hi, ra;
  Suffix(const auto &_s, int _n) : n(_n), hi(n), ra(n) {
    vector<int> s(n+1); copy_n(_s, n, s.begin());
    sa = sais(s), sa.erase(sa.begin());
    for (int i = 0; i < n; ++i) ra[sa[i]] = i;
    for (int i = 0, h = 0; i < n; ++i) {
      if (!ra[i]) { h = 0; continue; }
      for (int j = sa[ra[i]-1]; max(i,j)+h < n &&
           s[i+h] == s[j+h];) ++h;
      hi[ra[i]] = h ? h-- : 0; }
  }
};
vector<unsigned char> a(s.begin(), s.end()); // no 0 bytes
Suffix sf(a.data(), a.size());
sf.sa[i]; // i-th smallest suffix
sf.ra[i]; // rank of suffix i
sf.hi[i]; // lcp(sa[i], sa[i-1]); needs C++20
```

5.5 Aho-Corasick Automatan* [eca65f]

Multi-pattern matching. Build $O(\sum|s_i|\sigma)$, scan $O(|T|)$. Use rnx for $O(1)$ transitions; root is node 1, chars are c-'A'. Solve once after scan; empty patterns: seed cnt[1]=1 before scan.

```
// Root 1; len>=2, sigma>=1; nodes [1,top), top<=len.
// c-'A' in [0,sigma); total visits fit int.
// init, insert all, make_fl, scan via rnx, solve once.
// Build O(top*sigma), scan O(|T|).
struct AC_Automatan {
  int nx[len][sigma], fl[len], cnt[len], ord[len], top;
  int rnx[len][sigma]; // complete transitions
  int newnode() {
    fill_n
        (nx[top], sigma, -1), fill_n(rnx[top], sigma, 0);
    fl[top] = cnt[top] = ord[top] = 0; return top++;
  }
  void init() { top = 1, newnode(); }
  int input(string &s) {
    int x = 1; for (char c : s) {
      int &v = nx[x][c - 'A']; if (v < 0) v = newnode();
      x = v;
    } return x; // empty pattern: cnt[1]=1 before scan
  }
  void make_fl() {
    ord[0] = 1; for (int h = 0, t = 1; h < t; ++h) {
      int r = ord[h]; for (int i = 0; i < sigma; ++i) {
        int v = nx[r][i], f = r == 1 ? 1 : rnx[fl[r]][i];
        rnx[r][i] = v < 0 ? f : v;
        if (v >= 0) fl[v] = f, ord[t++] = v;
      }
    }
  }
  void solve() { // propagate visits once, after scanning
    for (int i = top - 2; i > 0; --i)
      cnt[fl[ord[i]]] += cnt[ord[i]];
  }
} ac;
ac.init(); vector<int> ends;
for (auto &pat : pats) ends.pb(ac.input(pat));
ac.make_fl(); int x = 1;
for (char c : text)
  x = ac.rnx[x][c - 'A'], ++ac.cnt[x];
ac.solve(); // propagate visits once
ac.cnt[ends[i]]; // occurrences of pats[i]
```

5.6 Smallest Rotation [5b1a96]

Lexicographically smallest rotation (Booth/min-notation) in $O(n)$.

```
string mcp(string s) { // O(n); unsigned bytes; empty OK
  int n = SZ(s), i = 0, j = 1; s += s; // 2*n fits int
  while (i < n && j < n) { int k = 0;
    while (k < n && s[i + k] == s[j + k]) ++k;
    if ((unsigned char)s[i+k] <= (unsigned char)s[j+k])
      j += k+1;
    else i += k+1; if (i == j) ++j;
  } return s.substr(min(i, j), n);
}
string t = mcp(s); // lexicographically smallest rotation
```

5.7 De Bruijn sequence* [c6067b]

Sequence of length $N+K-1$ with distinct length-N windows ($1 \le K \le C^N$). $C \ge 1, 0 \le N <$ MAXC·MAXN; allocate $N+K-1$ outputs.

```
constexpr int MAXC = 10, MAXN = 1e5 + 10;
struct DBSeq { // FKM; C>=1, N>=0, 1<=K<=C^N, N<MAXC*MAXN
  // Valid
       out[0,L), disjoint from *this; L=N+K-1 fits int.
  // K>1,N>1 need O(N) stack.
  int C, N, K, L, buf[MAXC * MAXN];
  void dfs(int *out, int t, int p, int &ptr) {
    if (ptr >= L) return; if (t > N) {
      if (N % p) return; int k = min(p, L-ptr);
      copy_n(buf+1, k, out+ptr), ptr += k;
    } else { buf[t] = buf[t - p], dfs(out, t + 1, p, ptr);
      for (int j = buf[t - p] + 1; j < C && ptr < L; ++j)
        buf[t] = j, dfs(out, t + 1, t, ptr);
    }
  }
  void solve(int _c, int _n, int _k, int *out) {
    int p = 0; C = _c, N = _n, K = _k;
    L = N + (K-1), buf[0] = 0;
    if (N == 1) { iota(out, out + L, 0); return; }
    if (K == 1) { fill_n(out, L, 0); return; }
    dfs(out, 1, 1, p);
    if (p < L) fill(out + p, out + L, 0);
  }
} dbs;
dbs.solve(C, N, K, out); // C>=1, N>=0, 1<=K<=C^N
// N+K-1 symbols; K distinct length-N windows
```

5.8 Extended SAM* [712322]

Generalised suffix automaton over many strings; cnt gives occurrences per state. Chars are c-'a'. Insert all, build once, solve once; clones must fit $2N$ states.

```
struct exSAM { // init, insert all, build/solve once
  // Root=0; N,CNUM>=1; states incl. clones fit 2*N.
  // 2*N, total length fit int; 0<=c-'a'<CNUM.
  int len[N * 2], link[N * 2], next[N * 2][CNUM], tot;
  int lenSorted[N * 2], cnt[N * 2]; // order, counts
  int newnode() { fill_n(next[tot], CNUM, 0);
    len[tot] = cnt[tot] = link[tot] = 0; return tot++;
  }
  void init() { tot = 0, newnode(), link[0] = -1; }
  int insertSAM(int last, int c) {
    int cur = next[last][c], p = link[last];
    len[cur] = len[last] + 1;
    while (p != -1 && !next[p][c])
      next[p][c] = cur, p = link[p];
    if (p == -1) return link[cur] = 0, cur;
    int q = next[p][c];
    if (len[p] + 1 == len[q]) return link[cur] = q, cur;
    int clone = newnode();
    for (int i = 0; i < CNUM; ++i) // built edges only
      next[clone][i] = len[next[q][i]] ? next[q][i] : 0;
    len[clone] = len[p] + 1, link[clone] = link[q];
    while (p != -1 && next[p][c] == q)
      next[p][c] = clone, p = link[p];
    link[cur] = link[q] = clone; return cur;
  }
  void insert(const string &s) {
    int cur = 0; for (char ch : s) {
      int &v = next[cur][ch - 'a']; if (!v) v = newnode();
      ++cnt[cur = v];
    }
  }
  void build() { // O(total input length * CNUM)
    int t = 1; lenSorted[0] = 0; // BFS, then length order
    for (int h = 0; h < t; ++h) { int u = lenSorted[h];
      for (int i = 0; i < CNUM; ++i)
        if (next[u][i]) lenSorted[t++] = insertSAM(u, i);
    } vector<int> lc(len[lenSorted[t-1]]+1); // max depth
    for (int i = 1; i < tot; ++i) ++lc[len[i]];
    partial_sum(ALL(lc), lc.begin());
    for (int i = 1; i < tot; ++i)
      lenSorted[--lc[len[i]]] = i;
  }
  void solve() {
    for (int i = tot - 2; i >= 0; --i)
      cnt[link[lenSorted[i]]] += cnt[lenSorted[i]];
  }
};
static exSAM ex; ex.init();
for (auto &s : strs) ex.insert(s); // trie
ex.build(); ex.solve(); // cnt = occurrences per state
```

5.9 PalTree* [a2e235]

Eertree for lowercase letters: distinct palindromic substrings and occurrences in $O(n\sigma)$. add takes char codes; count() once after all add.

```
struct
    palindromic_tree { // 'a'..'z'; n+2 fits int; O(26n)
  // Roots 0:0,1:-1; add takes char codes.
  struct node {
    int next[26] = {}, fail = 0, len, cnt = 0, num = 0;
    // cnt: occurrences; num: palindromic suffix count
    node(int l = 0) : len(l) {}
  };
  vector<node> St;
  vector<char> s;
  int last, n;
  palindromic_tree() { St.reserve(2); clear(); }
  void clear() {
    St.clear(), s.clear(), last = 1, n = 0;
    St.pb(0), St.pb(-1), St[0].fail = 1, s.pb(-1);
  }
  int get_fail(int x) {
```

```
    while (s[n - St[x].len - 1] != s[n]) x = St[x].fail;
    return x;
  }
  void add(int c) {
    s.pb(c -= 'a'), ++n; int cur = get_fail(last);
    if (!St[cur].next[c]) { int now = SZ(St);
      St.pb(St[cur].len + 2);
      St[now].fail = St[get_fail(St[cur].fail)].next[c];
      St[cur].next[c] = now;
      St[now].num = St[St[now].fail].num + 1;
    } last = St[cur].next[c], ++St[last].cnt;
  }
  void count() { // call once after all add()
    for (int i = SZ(St) - 1; i >= 0; --i)
      St[St[i].fail].cnt += St[i].cnt;
  }
  int size
      () { return SZ(St) - 2; } // distinct palindromes
};
palindromic_tree pt; for (char c : s) pt.add(c);
pt.count(); // fill cnt (occurrences)
pt.size(); // # distinct palindromes
```

5.10 Main Lorentz [2f8901]

All tandem repeats in $O(n \log n + kN)$, stored as inclusive 0-based start-range per period. Supports every byte value; main_lorentz clears rep. Require $\lfloor n/2 \rfloor < kN$.

```
// rep[l]: inclusive 0-based starts (+sft) of l-squares.
// Period=l; n/2<kN (integer division).
// n
    and sft+p (0<=p<n) fit int; empty/all bytes OK; C++17.
vector<pair<int, int>> rep[kN];
void main_lorentz_impl(const string &s, int sft) {
  string a; vector<int> z, rz; // Shared after children.
  auto rec = [&](auto &&self, string_view s,
                 int sft) -> void {
    int n=s.size(), nu=n/2, nv=n-nu; if (n<=1) return;
    if (n == 2) {
      if (s[0] == s[1]) rep[1].push_back({sft, sft});
      return; }
    string_view u = s.substr(0, nu), v = s.substr(nu);
    self(self, u, sft), self(self, v, sft+nu);
    auto zalgo = [](const string &a, vector<int> &z) {
      int n = a.size(); z.assign(n, 0);
      for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r-i+1, z[i-l]);
        while (i+z[i] < n && a[z[i]] == a[i+z[i]]) ++z[i];
        if (i+z[i]-1 > r) l = i, r = i+z[i]-1;
      } };
    // |u|<=|v|; two Z arrays capped at half boundaries.
    a.assign(v); a += u; zalgo(a, z); reverse(ALL(a));
    zalgo(a, rz); for (int c = 0; c < n; ++c) {
      bool left = c < nu; int l, k1, k2;
      if (left
          ) l = nu-c, k1 = min(c, rz[l]), k2 = z[nv+c];
      else l = c-nu+1, k1 = min(nu, rz[n-l]),
        k2 = min(nv-l, z[l]);
      int L = max(1, l-k2), R = min(l-left, k1);
      int d = left ? 0 : l-1;
      if (L<=R) rep[l].emplace_back(sft+c-R-d, sft+c-L-d);
    }
  }; rec(rec, s, sft); }
void main_lorentz(const string &s, int sft = 0) { // Reset
  for (auto &v : rep) v.clear();
  main_lorentz_impl(s, sft); }
// O(n log n+kN); local p=start-sft: s[p,p+l)==s[p+l,p+2l)
main_lorentz(s); // resets and fills rep[length]
// [L,R] in rep[l]: every p has s[p,p+l)==s[p+l,p+2l)
```

6 Math

6.1 ax+by=gcd(only exgcd *) [b91042]

Extended Euclid for nonnegative operands; returns Bezout coefficients, $O(\log \min(a,b))$. The trailing shift recipe needs $b > 0$, $\gcd(a,b) \mid res$ and fitting ll arithmetic.

```
pll exgcd(ll a, ll b) { // a,b>=0; ll arithmetic must fit.
  if (!b) return {1, 0};
  auto p = exgcd(b, a % b);
  return {p.Y, p.X - p.Y * (a / b)};
}
// ax+by=res, a>=0,b>0; g=gcd(a,b) must divide res.
// (x,y)=exgcd(a,b)*res/g; x'=x mod (b/g) in [0,b/g).
// y' = y - (x'-x)/(b/g)*(a/g).
```

6.2 Floor and Ceil [160df3]

Floor/ceil division that is correct for **negative** operands ($C++$ truncates toward zero). Requires $b \neq 0$; exclude INT_MIN/-1.

```
int floor(int a, int b) { // b!=0; exclude INT_MIN/-1.
  return a/b - (a%b && (a<0) != (b<0)); }
int ceil(int a, int b) {
  return a/b + (a%b && (a<0) == (b<0)); }
```

6.3 Floor Enumeration [c0114b]

Enumerates the $O(\sqrt{n})$ distinct values of $\lfloor n/i \rfloor$ with their index ranges. Requires $0 \le n <$ INT_MAX.

```
// 0<=n<INT_MAX; floor(n/i)=x on [l,r], O(sqrt(n)) blocks.
// l<=x gives r=l; next l>=q gives next quotient q-1.
for (int l=1,r,q=n; l<=n; l=r+1,q=l<q ? n/l : q-1) {
  int x=q; r=l<=x ? l : n/x; }
```

6.4 Modular Binomial / Lucas [ecce1e]

Requires ModArithmetic, prime p, and $0 \le N < p$. Precompute factorials / inverse factorials in $O(N + \log p)$, query $\binom{n}{k}$ in $O(1)$ for $n \le N$. Invalid k returns 0. Lucas handles larger n in $O(\log_p n)$, but needs the full $N = p-1$ table: use only small primes.

```
// ModArithmetic; prime p; 0<=N<min(p,INT_MAX).
// O(N+log p) init; O(1) C.
struct ModBinomial { ModArithmetic mod;
  vector<ll> fac, ifac;
  ModBinomial(int N, ll p) : mod(p) {
    assert(p >= 2 && N >= 0 && N < p);
    fac.resize(N + 1), ifac.resize(N + 1), fac[0] = 1;
    auto mul = [p](ll x, int i) { return p < (1LL << 31) ?
      x*i%p : (ll)((__int128)x*i%p); };
    for (
        int i = 1; i <= N; ++i) fac[i] = mul(fac[i-1], i);
    ifac[N] = mod.pow(fac[N], p - 2);
    for (int i = N; i; --i) ifac[i - 1] = mul(ifac[i], i);
  }
  ll C(int n, int k) const { // invalid: 0; else n<=N.
    if (n < 0 || k < 0 || k > n) return 0;
    assert(n < (int)fac.size());
    if (mod.m < (1LL << 31))
      return fac[n]*ifac[k]%mod.m*ifac[n-k]%mod.m;
    return (__int128)fac[n]*ifac[k]%mod.m*ifac[n-k]%mod.m;
  }
  ll lucas(ll n, ll k) const { // N=p-1; O(log_p n).
    if (n < 0 || k < 0 || k > n) return 0;
    assert((ll)fac.size() == mod.m);
    ll r
        = 1; for (; r && (n || k); n /= mod.m, k /= mod.m)
      r = r*C(n % mod.m, k % mod.m)%mod.m; // p<=INT_MAX
    return r;
  }
};
ModBinomial comb(1000000, 1000000007);
comb.C(5, 2);   // 10; invalid k returns 0
comb.fac[5];    // 120
comb.ifac[5];   // inverse of 120 modulo p
ModBinomial small(6, 7);
small.lucas(10, 3); // 120 mod 7 = 1
```

6.5 Linear Congruence [284b48]

All solutions of $ax \equiv b \pmod n$ in $[0,n)$, sorted. Requires exgcd, $n > 0$; signed a,b are normalized. Empty if $\gcd(a,n) \nmid b$; otherwise exactly $\gcd(a,n)$ solutions. $O(\log n + \gcd(a,n))$ time; output can be large.

```
// Sorted ax=b mod n in [0,n); n>0; exgcd; O(log n+g).
vector
    <ll> mod_leq(ll a, ll b, ll n) { // g answers fit RAM
  a %= n; if (a < 0) a += n;
  b %= n; if (b < 0) b += n;
  auto [u, v] = exgcd(a, n);
  ll g = (__int128)a * u + (__int128)n * v, step = n / g;
  if (b % g) return {};
  ll x = (__int128)u * (b / g) % step;
  if (x < 0) x += step;
  vector<ll> rt(g);
  for (ll i = 0; i < g; ++i) rt[i] = x + i * step;
  return rt;
}
auto xs = mod_leq(4, 2, 6); // {2,5}: 4*x == 2 (mod 6)
```

6.6 Mod Min [17b844]

Smallest $k \ge 0$ with $l \le ak \bmod m \le r$, $O(\log m)$; -1 if none. Requires $m > 0$, $0 \le a < m$, $0 \le l \le r < m$, and all intermediate products/sums fitting ll.

```
// 0<=a<m, 0<=l<=r<m; min k: l<=ak%m<=r, or -1. O(log m).
// m>0; all evaluated ll products/sums must fit.
ll mod_min(ll a, ll m, ll l, ll r) {
  if (!a) return l ? -1 : 0; ll k = (l + a - 1) / a;
  if (k * a <= r) return k; // d: gap to next a-multiple
  ll c = m % a, d = k*a - l;
  ll y = mod_min(c, a, d - (r-l), d);
  return y == -1 ? -1 : (l + y*c + a - 1) / a + y*(m/a);
}
ll k = mod_min(3, 7, 4, 5); // 4: 3*4 mod 7 = 5
```

6.7 Linear Mod Inverse [232adb]

All inverses $1..N$ in $O(N)$. Function-body snippet: supply N, mod, and inv first. **Requires prime** int mod **and** $1 \le N <$ mod. For an arbitrary modulus or one inverse, use ModArithmetic::inv.

```
// Prime int mod, 1<=N<mod; allocate inv[0..N]. O(N).
inv[1] = 1; for (int i=2; i<=N; ++i)
  inv[i] = 1LL * (mod-mod/i) * inv[mod%i] % mod;
int N = 10, mod = 1000000007; // prime, 1<=N<mod
vector<int> inv(N + 1); // Run template here.
ll quotient = 1LL * 7 * inv[3] % mod; // inv[3]=333333336
```

6.8 Linear Filter Mu [1301d8]

Mobius function by linear sieve, $O(n)$; also leaves the prime list in p. Require $n \geq 1$; zero flg, tot first; p is 1-base.

```
void getMu() { // 1<=n<INT_MAX; zero flg,tot; O(n).
  // mu, flg[0..n]; p[1..prime count]; enough capacity.
  mu[1] = 1; for (int i = 2; i <= n; ++i) {
    if (!flg[i]) p[++tot] = i, mu[i] = -1;
    for (int j = 1; j <= tot && 1LL*i*p[j] <= n; ++j) {
      int x = i * p[j]; flg[x] = 1;
      if (i % p[j] == 0) { mu[x] = 0; break; }
      mu[x] = -mu[i];
    } }
}
```

6.9 Gaussian integer gcd [7447af]

GCD over $\mathbb{Z}[i]$; local rnd rounds the quotient to the nearest Gaussian integer. Requires $b \neq 0$ and fitting ll arithmetic.

```
// b!=0;
    evaluated ll arithmetic (incl. rounding) must fit.
complex<ll> gaussian_gcd(complex<ll> a, complex<ll> b) {
  ll c = a.real()*b.real() + a.imag()*b.imag(),
     d = a.imag()*b.real() - a.real()*b.imag(),
     r = b.real()*b.real() + b.imag()*b.imag();
  auto rnd = [&](ll x) {
    return (2*x + (x >= 0 ? r : -r))/(2*r); };
  if (c % r == 0 && d % r == 0) return b;
  return
      gaussian_gcd(b, a - complex<ll>(rnd(c),rnd(d))*b);
}
```

6.10 GaussElimination [360a03]

Gauss-Jordan on $n \times w$ doubles, $n \leq w$, $O(n^2 w)$. Max-abs pivots in first n columns, cutoff 10^{-12}. Reduces matrix only; no solution/status. Absolute epsilon is scale-sensitive; use fractions for exactness.

```
// Rectangular: 0<=n<=w<=INT_MAX; pivot cols <n. O(n^2*w).
// Max-abs pivot; eps=1e-12 scale-sensitive; finite math.
void GAS(vector<vector<double>> &vc) {
  int len = vc.size(), row = 0;
  for (int col = 0; col < len && row < len; ++col) {
    int idx = row;
    for (int j = row + 1; j < len; ++j)
      if (fabs(vc[j][col]) > fabs(vc[idx][col])) idx = j;
    if (fabs(vc[idx][col]) < 1e-12) continue;
    swap(vc[idx], vc[row]); double pivot = vc[row][col];
    for (double &x : vc[row]) x /= pivot;
    for (int j = 0; j < len; ++j) {
      if (j == row || fabs(vc[j][col]) < 1e-12) continue;
      double mul = vc[j][col];
      for (int k = 0; k < (int)vc[j].size(); ++k)
        vc[j][k] -= mul * vc[row][k];
    } ++row;
  }
}
```

6.11 floor sum* [36d26d]

$\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$ in logarithmic time. Requires $n \geq 0, m > 0, a, b \geq 0$ and all intermediate arithmetic fitting ll; this version does not handle negative a, b. Returns the integer sum, not a residue.

```
// Sum floor((a*i+b)/m), 0<=i<n; n,a,b>=0,m>0; fit ll.
ll floor_sum(ll n, ll m, ll a, ll b) {
  ll ans = 0;
  if (a >= m) ans += (n - 1) * n * (a / m) / 2, a %= m;
  if (b >= m) ans += n * (b / m), b %= m;
  ll y = a * n + b;
  return
      ans + (y < m ? 0 : floor_sum(y / m, a, m, y % m));
}
ll s = floor_sum(4, 5, 3, 2); // 0+1+1+2 = 4
```

6.12 Miller Rabin* [80bc1f]

Deterministic primality via 7 witnesses; the supplied wrapper handles $n < 2$. This ll interface covers $n \leq$ LLONG_MAX. $O(\log n)$ per witness. Use the example's 128-bit mul and prime wrapper.

```
// Witness a>=0; exact mul for 0<=x,y<n; use prime().
bool Miller_Rabin(ll a, ll n) {
  if (n < 2) return 0;
  if ((a %= n) == 0) return 1;
  if (n % 2 == 0) return n == 2;
  ll d = n - 1, x = a; int s = __builtin_ctzll(d);
  // d>>s is odd; x=a consumes its low bit.
  for (d >>= s + 1; d; d >>= 1) { a = mul(a, a, n);
    if (d & 1) x = mul(x, a, n); }
  if (x == 1 || x == n - 1) return 1;
  while (--s) if ((x = mul(x, x, n)) == n - 1) return 1;
  return 0;
}
// Signed ll domain (through LLONG_MAX).
// Place mul() before Miller_Rabin.cpp; prime() after it.
ll mul(ll a, ll b, ll m) { // 0 <= a,b < m, m > 0
  return (ll)((__int128)a * b % m);
}
bool prime(ll n) { // negatives/0/1 are not prime
  if (n < 2) return false;
  for (
      ll a : {2,325,9375,28178,450775,9780504,1795265022})
    if (!Miller_Rabin(a, n)) return false;
  return true;
}
```

6.13 Fraction [a92cb1]

Exact rational with auto-reduction. ll **numerator/denominator overflow easily** after a few operations; no divide-by-zero check. Exclude LLONG_MIN and zero denominators/divisors.

```
struct fraction { // No LLONG_MIN or ll overflow.
  ll n, d; // Nonzero denominators/divisors.
  fraction(const ll &a=0, const ll &b=1): n(a), d(b) {
    ll g = d == 1 ? 1 : gcd(n, d); n /= g, d /= g;
    if (d < 0) n = -n, d = -d;
  }
  fraction operator-() const { return {-n, d}; }
  fraction operator+(const fraction &b) const {
    return {n*b.d + b.n*d, d*b.d}; }
  fraction operator-(const fraction &b) const {
    return {n*b.d - b.n*d, d*b.d}; }
  fraction operator*(const fraction &b) const {
    return {n*b.n, d*b.d}; }
  fraction operator/(const fraction &b) const {
    return {n*b.d, d*b.n}; }
  void
      print() { cout << n; if (d != 1) cout << "/" << d; }
};
```

6.14 Simultaneous Equations [8e3811]

Exact Gauss-Jordan over fraction; returns -1 if inconsistent, else the rank. Requires Fraction.cpp. Solution in sol, free variables zero; overwrites M.

```
struct matrix { // 0<=n,m<=MAXN
  // Intermediates obey Fraction.cpp preconditions.
  int n, m;
  fraction M[MAXN][MAXN + 1], sol[MAXN];
  int solve
      () { // -1: inconsistent; else rank, free vars=0
    int rank = 0, piv[MAXN];
    for (int col = 0; col < m && rank < n; ++col) {
      int row = rank;
      while (row < n && !M[row][col].n) ++row;
      if (row == n) continue;
      for (int k = col; k <= m; ++k)
        swap(M[rank][k], M[row][k]);
      for (int j = 0; j < n; ++j)
        if (j != rank && M[j][col].n) {
          fraction t = -M[j][col] / M[rank][col];
          M[j][col] = {};
          for (int k = col + 1; k <= m; ++k)
            if (M[rank][k].n)
              M[j][k] = t*M[rank][k]+M[j][k];
        }
      piv[rank++] = col;
    }
    for (int i = rank; i < n; ++i)
      if (M[i][m].n) return -1;
    fill_n(sol, m, fraction());
    for (int i = 0; i < rank; ++i)
      sol[piv[i]] = M[i][m] / M[i][piv[i]];
    return rank;
  }
};
matrix eq{}; eq.n = eq.m = 2;
eq.M[0][0]=1; eq.M[0][1]=1;  eq.M[0][2]=3;
eq.M[1][0]=2; eq.M[1][1]=-1; eq.M[1][2]=0;
int rank=eq.solve(); // sol={1,2}; -1: inconsistent
```

6.15 Pollard Rho* [e5ac1c]

Factorises positive ll integers, heuristic $O(n^{1/4})$. Requires prime and 128-bit mul from the Miller–Rabin example. Factors accumulate in cnt; clear it before each new number.

```
map<ll, int> cnt; // Clear per input; exact mul(a,b,n).
void
    PollardRho(ll n) { // 1<=n<=LLONG_MAX; correct prime()
  if (int e = __builtin_ctzll(n)) cnt[2] += e, n >>= e;
  if (n == 1) return;
  if (prime(n)) return ++cnt[n], void();
  ll x=0, y=0, q=1, d=1, seed=1; int batch=1;
  auto f = [&](ll z) { ll t = mul(z,z,n);
    return t >= n-seed ? t-(n-seed) : t+seed; };
  while (d == 1) {
    for (int i = 0; i < batch; ++i) {
      if (x == y) { x = seed = (seed + 1) % n; y = f(x); }
      ll t = mul(q, abs(x - y), n);
      if (t) q = t; // q!=0 ensures gcd(q,n)<n.
      x = f(x), y = f(f(y));
    } d = gcd(q, n); batch = min(64, batch * 2);
```

```
  }
  PollardRho(d), PollardRho(n / d);
}
cnt.clear(); PollardRho(n); // cnt accumulates otherwise
for (auto [p, e] : cnt) // n = product(p^e)
  use_factor(p, e);
```

6.16 Simplex Algorithm [b7f3e8]

maxcx s.t. $Ax \le b, x \ge 0$; -1 means unbounded/infeasible or a valid optimum. n constraints, m variables, 0-indexed; $n+2 \le$ MAXN, $m+2 \le$ MAXM. x holds the solution on success.

```
const int MAXN = 11000, MAXM = 405;
const double eps = 1E-10;
double a[MAXN][MAXM], d[MAXN][MAXM];
double b[MAXN], c[MAXM], x[MAXM]; int ix[MAXN + MAXM];
// max cx: Ax<=b, x>=0; 0-base; n rows, m variables.
// n,m>=0; n+2<=MAXN,m+2<=MAXM; x optimal on success.
// -1: infeasible/unbounded or a valid optimum.
// Finite, well-scaled arithmetic; absolute eps.
// O((n+1)(m+1))/pivot, exponential worst case.
double simplex(int n, int m) {
  ++m; fill_n(d[n], m + 1, 0), fill_n(d[n + 1], m + 1, 0);
  iota(ix, ix + n + m, 0); int r = n, s = m - 1;
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m - 1; ++j) d[i][j] = -a[i][j];
    d[i][m - 1] = 1, d[i][m] = b[i];
    if (d[r][m] > d[i][m]) r = i;
  }
  copy_n(c, m - 1, d[n]); d[n + 1][m - 1] = -1;
  for (double dd;;) {
    if (r < n) {
      swap(ix[s], ix[r + m]); d[r][s] = 1 / d[r][s];
      for (int j = 0; j <= m; ++j)
        if (j != s) d[r][j] *= -d[r][s];
      for (int i = 0; i <= n + 1; ++i)
        if (i != r && d[i][s]) {
          double v = d[i][s]; d[i][s] = 0;
          for (int j = 0; j <= m; ++j)
            d[i][j] += d[r][j] * v;
        }
    }
    r = s = -1; // Bland's rule: smallest variable index.
    for (
        int j = 0; j < m; ++j) if (s < 0 || ix[s] > ix[j])
      if (d[n + 1][j] > eps ||
          (d[n + 1][j] > -eps && d[n][j] > eps)) s = j;
    if (s < 0) break;
    for (int i = 0; i < n; ++i) if (d[i][s] < -eps)
      if (r < 0 || (dd = d[r][m] / d[r][s] -
          d[i][m] / d[i][s]) < -eps ||
          (dd < eps && ix[r + m] > ix[i + m])) r = i;
    if (r < 0) return -1;
  }
  if (d[n + 1][m] < -eps) return -1;
  double ans = 0; fill_n(x, m, 0);
  for (int i = m; i < n + m; ++i) if (ix[i] < m - 1)
    ans += d[i - m][m] * c[ix[i]], x[ix[i]] = d[i - m][m];
  return ans;
}
int n=3, m=2; // max 3x+2y: x+y<=4, x<=2, y<=3
a[0][0]=1; a[0][1]=1; b[0]=4;
a[1][0]=1; b[1]=2; a[2][1]=1; b[2]=3;
c[0]=3; c[1]=2;
double value=simplex(n,m); // value=10, x={2,2}
```

6.17 chineseRemainder [1239a1]

Merge $x \equiv x_1 \pmod{m_1}$ and $x \equiv x_2 \pmod{m_2}$, including non-coprime moduli. Requires exgcd, $m_1, m_2 > 0$, and $\text{lcm}(m_1, m_2) \le$ LLONG_MAX. Returns the least nonnegative solution, or -1 if inconsistent. Uses 128-bit intermediates. $O(\log\min(m_1,m_2))$; iterate to merge more equations.

```
ll solve(ll x1, ll m1, ll x2, ll m2) {
  ll g = gcd(m1, m2); __int128 d = (__int128)x2 - x1;
  if (d % g) return -1; // m1,m2>0; lcm fits ll.
  m2 /= g; pll p = exgcd(m1 / g, m2);
  __int128 t = m2 == 1 ? 0 : (__int128)p.first*(d/g)%m2;
  ll lcm = m1 * m2; __int128 res = x1 + (__int128)m1 * t;
  res %= lcm; return (ll)(res < 0 ? res + lcm : res);
}
ll x = solve(2, 6, 5, 9); // x=14 (mod lcm=18)
```

6.18 Factorial without prime factor* [ab4a85]

$n!$ with all factors p removed, modulo p^k. Requires $n \ge 0$, prime p, $k \ge 1$, $p^k <$ min(MAXP, INT_MAX), and products fitting ll. $O(p^k + \log_p n)$ time, O(MAXP) space. This returns only the unit part; account for the exponent of p separately when computing a factorial or binomial modulo p^k.

```
// n!
//   without p factors mod pk=p^k; p prime; O(pk+log_p n).
// n>=0, k>=1, pk<min(MAXP,INT_MAX).
ll prod[MAXP];
ll fac_no_p(ll n, ll p, ll pk) {
  prod[0] = 1; for (int i = 1; i <= pk; ++i)
    prod[i] = i % p ? prod[i - 1] * i % pk : prod[i - 1];
  ll rt
      = 1; for (; n; n /= p) { // Unit block squared = 1.
    if ((n / pk) & 1) rt = rt * prod[pk] % pk;
    rt = rt * prod[n % pk] % pk;
  } return rt;
}
ll value=fac_no_p(10,3,27);
// value=7: remove every factor 3 from 10!, then mod 27
```

6.19 Jacobi / Modular Square Root [d6fe05]

Jacobi(a,m) needs positive odd m; returns $-1,0,1$ in $O(\log m)$. For composite m, $+1$ does not guarantee a square. QuadraticResidue(a,p) uses Cipolla: prime $2 \le p \le$ INT_MAX, expected $O(\log p)$; returns one root in $[0,p)$ or -1. Signed input is normalized.

```
// Jacobi: positive odd modulus; O(log m).
int Jacobi(int a, int m) {
  a %= m; if (a < 0) a += m; int s = 1;
  for (; m > 1;) {
    a %= m; if (!a) return 0;
    int r = __builtin_ctz((unsigned)a);
    if ((r & 1) && ((m & 7) == 3 || (m & 7) == 5)) s = -s;
    a >>= r; if (a & m & 2) s = -s;
    swap(a, m);
  } return s;
}
// Prime 2<=p<=INT_MAX; root or -1; expected O(log p).
int QuadraticResidue(int a, int p) {
  a %= p; if (a < 0) a += p; if (p == 2) return a;
  if (p % 4 == 3) { ll r = 1, b = a; // a^((p+1)/4).
    for (int e = (1LL+p)/4; e; e >>= 1, b = b*b%p)
      if (e & 1) r = r*b%p;
    return r*r%p == a ? r : -1; }
  int jc = Jacobi(a, p), b, d;
  if (jc <= 0) return jc;
  do b = rand() % p, d = (1LL * b * b + p - a) % p;
  while (Jacobi(d, p) != -1);
  auto mul = [&](pair<ll, ll> x, pair<ll, ll> y) {
    auto [u, v] = x; auto [s, t] = y;
    return pair{(u*s + d*(v*t%p))%p, (u*t + v*s)%p}; };
  pair<ll, ll> f{b, 1}, g{1, 0}; // x + y*sqrt(d).
  for (int e = (1LL + p) >> 1; e; e >>= 1, f = mul(f, f))
    if (e & 1) g = mul(g, f);
  return g.first; }
Jacobi(2, 15); // 1; composite m: +1 need not be a square
int r = QuadraticResidue(10, 13); // 6 or 7
if (r != -1) { // prime p only; -1 means no square root
  int other = (13 - r) % 13;
}
```

6.20 Modular Determinant [a2b291]

Euclidean elimination without modular inverses, $O(n^3 \max(1, \log P))$; composite moduli work too. Supply MAXN, positive ll modulus P, and a square matrix with entries in $[0,P)$. Returns the determinant modulo P and overwrites the matrix. 128-bit products.

```
// 0<=n=m<=MAXN, entries in [0,P), P>0; overwrites M.
struct Matrix {
  int n, m; ll M[MAXN][MAXN];
  int row_swap(int i, int j) { if (i == j) return 0;
    swap_ranges(M[i], M[i] + m, M[j]); return 1;
  }
  ll det() { // Composite P OK; O(n^3 max(1,log P)).
    ll rt = 1 % P;
    for (int i = 0; i < n; ++i) {
      for (int j = i + 1; j < n; ++j) while (M[j][i]) {
        ll q = M[i][i] / M[j][i]; // q=1: no remainder.
        if (q) for (int k = i; k < m; ++k)
          M[i][k] = q == 1 ? (M[i][k] >= M[j][k] ?
            M[i][k] - M[j][k] : P - M[j][k] + M[i][k]) :
            P <= INT_MAX ? (M[j][k]*(P-q) + M[i][k]) % P :
            ((__int128)M[j][k] * (P-q) + M[i][k]) % P;
        // Earlier columns in both rows are zero.
        swap_ranges(M[i]+i, M[i]+m, M[j]+i),
          rt = rt ? P - rt : 0;
      }
      if (!M[i][i]) return 0;
      rt = (__int128)rt * M[i][i] % P;
    }
    return rt; }
};
Matrix a; a.n = a.m = 2;
a.M[0][0] = 1 % P; a.M[0][1] = 2 % P;
a.M[1][0] = 3 % P; a.M[1][1] = 4 % P;
ll d = a.det(); // -2 modulo P; matrix a is overwritten
```

6.21 PiCount* [5b27ac]

$\pi(n)$ for $n \le 10^{13}$; $n \le 1$ returns 0. Conservative $O(n^{3/4})$ time bound, $O(\sqrt{n})$ space.

```
ll PrimeCount
    (ll n) { // n<=10^13; n<=1:0; O(sqrt n) space.
  if (n <= 1) return 0;
  int v = sqrt(n), s = (v + 1) / 2, pc = 0;
```

```
  vector
      <int> smalls(s), roughs(s); // smalls: bound 2*j+1.
  vector<ll> larges(s); vector<char> skip(v + 1);
  for (int i = 1; i < s; ++i) smalls[i] = i + 1;
  for (int i = 0; i < s; ++i)
    roughs
        [i] = 2*i+1, larges[i] = (n / roughs[i] + 1) / 2;
  for (int p = 3; p <= v; p += 2) {
    if (skip[p]) continue; int q = p * p; ++pc;
    if (1LL * q * q > n) break; skip[p] = 1;
    for (int i = q; i <= v; i += 2 * p) skip[i] = 1;
    int ns = 0; for (int k = 0; k < s; ++k) {
      int i = roughs[k]; if (skip[i]) continue;
      ll d = 1LL * i * p;
      larges[ns] = larges[k] - (d <= v ?
        larges[smalls[d / 2] - pc] :
        smalls[(n / d - 1) / 2]) + pc;
      roughs[ns++] = i;
    } s = ns;
    for (int j = (v / p - 1) / 2; j >= p / 2; --j) {
      int c = smalls[j] - pc;
      int e = min((j + 1) * p + p / 2, (v + 1) / 2);
      for (int i = j * p + p / 2; i < e; ++i)
        smalls[i] -= c;
    }
  }
  for (int k = 1; k < s; ++k) {
    ll m = n / roughs[k], t = larges[k] - (pc + k - 1);
    for (int l = 1;
         l < k && 1LL * roughs[l] * roughs[l] <= m; ++l)
      t -= smalls[(m / roughs[l] - 1) / 2] - (pc + l - 1);
    larges[0] -= t;
  } return larges[0];
}
```

6.22 Discrete Log* [261f30]

Call the 3-argument version: smallest $k \geq 0$ with $x^k \equiv y \pmod m$, or -1. Expected $O(\sqrt{m})$ time/space. Requires positive int m and $x, y \in [0, m)$; non-coprime bases work. Supply fpow(x,k,m) for verification. The 4-argument version is internal.

```
// m>0; x,y in [0,m); supply fpow(x,k,m); use 3-arg API.
// Expected O(sqrt(m)) time/space; 4-arg helper internal.
int DiscreteLog(int s, int x, int y, int m) {
  int step
      = sqrt(m), b = 1; if (1LL*step*step < m) ++step;
  unordered_map<int, int> p; p.reserve(2 * step);
  for (int i = 0; i < step; ++i)
    p[y] = i, y = 1LL * y * x % m, b = 1LL * b * x % m;
  for (long long i = 0; i < 1LL * m + 10; i += step) {
    s = 1LL * s * b % m;
    if (auto it = p.find(s); it != p.end()) {
      long long ans = i + step - it->second;
      return ans <= INT_MAX ? (int)ans : -1;
    }
  } return -1;
}
int DiscreteLog(int x, int y, int m) {
  if (m == 1 || y == 1) return 0;
  if (x <= 1) return y == x ? 1 : -1; int s = 1;
  for (int i = 0; i < 100; ++i, s = 1LL * s * x % m)
    if (s == y) return i; // skip non-coprime transient
  if (s == y) return 100;
  int tail = DiscreteLog(s, x, y, m);
  long long ans = 100LL + tail;
  return tail < 0 || ans > INT_MAX ||
    fpow(x, (int)ans, m) != y ? -1 : (int)ans;
}
// Needs ModArithmetic for this verification helper.
int fpow(int x, int k, int m) {
  return (int)ModArithmetic(m).pow(x, k);
}
int k = DiscreteLog(2, 8, 13); // k=3; -1 if none
```

6.23 Berlekamp Massey [5b644c]

Finds recurrence coefficients in $O(n^2)$: $a_i = \sum_j c_j a_{i-j-1}$. Requires a caller-provided field type T with zero value-initialization, arithmetic, division, and comparison with zero. For modular arithmetic use a prime-modulus mint; composite-modulus division is invalid. Returns coefficients only.

```
// Field
    : 0 init, arithmetic, nonzero /, ==0; size<INT_MAX.
template<class T>
vector<T> BerlekampMassey(const vector<T> &a) {
  vector<T> me, he, old; T last = 0;
  for (int f = 0, i = 1; i <= SZ(a); ++i) {
    T d = -a[i - 1];
    for (
        int j = 0; j < SZ(me); ++j) d += a[i-j-2] * me[j];
    if (d == 0) continue;
    if (me.empty()) {
      me.resize(f = i), last = d; continue; }
    T k = -d / last; int s = i - f + SZ(he);
    bool grow = s >= SZ(me);
    if (grow) old.resize(SZ(me)),
      copy(me.begin(), me.end(), old.begin());
    me.resize(max(s, SZ(me))), me[i - f - 1] += -k;
    for (int j = 0; j < SZ(he); ++j)
      me[i - f + j] += he[j] * k;
    if (grow) he.swap(old), f = i, last = d;
  } return me; // a[i]=sum(c[j]*a[i-j-1]); O(n^2).
}
// mint: prime-modulus field with division.
vector<mint> seq = {0,1,1,2,3,5,8};
auto coef = BerlekampMassey(seq); // {1,1}
// a[i] = coef[0]*a[i-1] + coef[1]*a[i-2].
```

6.24 Primes

Handy primes for hashing and NTT moduli, from small values up to $2^{61}-1$ and near 2^{64}.

```
/* 12721 13331 14341 75577 123457 222557 556679 999983
 98789101
      100102021 987654361 987777733 999888733 999991231
 999991921 999997771 1000512343 1001010013 1010101333
 1010102101 1076767633 1097774749 1000000000039
 1000000000000037 2305843009213693951 4611686018427387847
 9223372036854775783 18446744073709551557 */
```

6.25 Theorem

Notation

$[P] \in \{0,1\}$ is the indicator of P; p is prime; $H = \min(N,M)$; $(f*g)(n) = \sum_{d|n} f(d)g(n/d)$. Quotient blocks over $\lfloor N/d \rfloor, \lfloor M/d \rfloor$: $O(\sqrt{N}+\sqrt{M})$. Linear sieve for μ, φ or any multiplicative function up to L: O(L).

Core identities

$\mu(1)=1$, $\mu(p)=-1$, $\mu(p^e)=0$ $(e \geq 2)$, multiplicative.
$\sum_{d|n}\mu(d) = [n=1]$ (the only identity the recipe uses).
$\sum_{d|n}\varphi(d) = n$, $\varphi(n) = \sum_{d|n}\mu(d)n/d = n\prod_{p|n}(1-1/p)$.
$f(n) = \sum_{d|n} g(d) \Leftrightarrow g(n) = \sum_{d|n}\mu(d)f(n/d)$.
$F(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d}\mu(d/n)F(d)$ (finite support).
f,g multiplicative $\rightarrow f*g$ multiplicative
$\rightarrow$ linear sieve. Any $c = f*g$ up to L by looping multiples: O(LlogL).

Recipe: sum of h(gcd)

1. Exact gcd $\rightarrow$ coprime.
$i = ga$, $j = gb$: $[\gcd(i,j)=g] = [\gcd(a,b)=1]$, bounds $\lfloor N/g \rfloor, \lfloor M/g \rfloor$.
2. Kill the bracket. $[\gcd(i,j)=1] = \sum_{d|i,\ d|j}\mu(d)$. Shortcut: $h(\gcd(i,j)) = \sum_{d|i,\ d|j} c(d)$ with $c = \mu*h$ (e.g. $h = \mathrm{id} \rightarrow c = \varphi$), skips step 1.
3. Swap
sums, d outermost. $\#\{1 \leq i \leq N : d \mid i\} = \lfloor N/d \rfloor$, independent per variable.
4. Merge $T = gd$. $\lfloor \lfloor N/g \rfloor / d \rfloor = \lfloor N/(gd) \rfloor$;
everything not in the floors becomes $c(T) = \sum_{g|T} h(g)\mu(T/g)$.
5. Evaluate.
Final shape $\sum_T c(T)\prod \lfloor N./T \rfloor$: sieve c, prefix sums, quotient blocks.

Example: $S(N,M) = \sum_{i \leq N}\sum_{j \leq M}\gcd(i,j)$
(1) $S = \sum_g g \sum_{a \leq \lfloor N/g \rfloor}\sum_{b \leq \lfloor M/g \rfloor}[\gcd(a,b)=1]$.
(2,3) $\rightarrow S = \sum_g g \sum_d \mu(d)\lfloor N/(gd) \rfloor \lfloor M/(gd) \rfloor$.
(4) $T = gd \rightarrow S = \sum_{T=1}^{H} \lfloor N/T \rfloor \lfloor M/T \rfloor \sum_{g|T} g\mu(T/g)$.
(5) $\sum_{g|T} g\mu(T/g) = \varphi(T) \rightarrow S = \sum_{T=1}^{H} \varphi(T)\lfloor N/T \rfloor \lfloor M/T \rfloor$.

Example: $\sum_{i=1}^{k}\gcd(i,n)$
(2) $\gcd(i,n) = \sum_{d|i,\ d|n}\varphi(d)$.
(3) $\rightarrow \sum_{i=1}^{k}\gcd(i,n) = \sum_{d|n}\varphi(d)\lfloor k/d \rfloor$. Given divisors of n: $O(\tau(n))$.

Results

$\sum_{i \leq N}\sum_{j \leq M}[\gcd(i,j)=1] = \sum_{d=1}^{H}\mu(d)\lfloor N/d \rfloor \lfloor M/d \rfloor$.
$\sum_{i \leq N}\sum_{j \leq M}[\gcd(i,j)=g]$: same with $N,M \rightarrow \lfloor N/g \rfloor, \lfloor M/g \rfloor$.
$\sum_{i \leq N}\sum_{j \leq M} h(\gcd(i,j)) = \sum_{d=1}^{H}(\mu*h)(d)\lfloor N/d \rfloor \lfloor M/d \rfloor$.
$h(n) = n^s \rightarrow \mu*h = J_s(d) = d^s\prod_{p|d}(1-p^{-s})$.
k variables:
$\sum_{i_1..i_k \leq N}[\gcd=1] = \sum_d \mu(d)\lfloor N/d \rfloor^k$, $\sum_{i_1..i_k \leq N}\gcd = \sum_T \varphi(T)\lfloor N/T \rfloor^k$.
$\sum_{a \leq x}[\gcd(a,m)=1] = \sum_{e|m}\mu(e)\lfloor x/e \rfloor$.

Prefix recurrences (Du Jiao)

$\mathcal{M}(N) = \sum_{d \leq N}\mu(d) = 1 - \sum_{i=2}^{N}\mathcal{M}(\lfloor N/i \rfloor)$.
$\Phi(N) = \sum_{d \leq N}\varphi(d) = N(N+1)/2 - \sum_{i=2}^{N}\Phi(\lfloor N/i \rfloor)$.
Why: $\sum_{i=1}^{N}\sum_{d \leq \lfloor N/i \rfloor} f(d) = \sum_{t \leq N}(f*1)(t)$.
Sieve up to $B = \Theta(N^{2/3})$ + memo: $O(N^{2/3})$.

Quotient blocks

$q_N = \lfloor N/l \rfloor$, $q_M = \lfloor M/l \rfloor$, $r = \min(\lfloor N/q_N \rfloor, \lfloor M/q_M \rfloor)$;
floors constant on [l,r].
$\rightarrow \sum_{d=l}^{r} c(d)\lfloor N/d \rfloor \lfloor M/d \rfloor = q_N q_M (P_c(r) - P_c(l-1))$; $l \leftarrow r+1$ while $l \leq H$.

Other theorems

Lucas: prime p, $n = \sum_i n_i p^i$,
$k = \sum_i k_i p^i \rightarrow \binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$. Factorial/inverse-factorial preprocessing: O(p) time/space; query: $O(1+\log_p(n+1))$.

CRT: compatible residues
determine one solution modulo the lcm of the moduli. For r congruences and intermediate values bounded by U: O(rlogU) arithmetic operations.

Kirchhoff: for
an undirected graph, $L_{ii}=\deg(i)$, $L_{ij}=-\#\{\text{edges between i,j}\}$ for $i\neq j$.
Delete row and column r to obtain $L^{(r)}\to\#\{\text{spanning trees}\}=\det L^{(r)}$.
For v vertices, Gaussian elimination: $O(v^3)$ time, $O(v^2)$ space.

Burnside: $|X/G|=|G|^{-1}\sum_{g\in G}|\mathrm{Fix}(g)|$.
Cost T_{fix} per fixed-point count $\to O(|G|T_{fix})$ total time.

6.26 Estimation

n	2	3	4	5	6	7	8	9	20	30	40	50	100	n	100	1e3	1e6	1e9	1e12	1e15	1e18
$p(n)$	2	3	5	7	11	15	22	30	627	5604	4e4	2e5	2e8	$d(i)$	12	32	240	1344	6720	26880	103680

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\binom{2n}{n}$	2	6	20	70	252	924	3432	12870	48620	184756	7e5	2e6	1e7	4e7	1.5e8

n	2	3	4	5	6	7	8	9	10	11	12	13
B_n	2	5	15	52	203	877	4140	21147	115975	7e5	4e6	3e7

6.27 General Purpose Numbers

- Bernoulli numbers
  $B_0=1,\quad B_1^-=-\frac{1}{2},\quad B_1^+=\frac{1}{2},\quad B_2=\frac{1}{6},\quad B_3=0$
  $\sum_{j=0}^{m}\binom{m+1}{j}B_j=0\,(m\geq 1)$ uses $B_1=B_1^-$; EGF is $B(x)=\frac{x}{e^x-1}=\sum_{n=0}^{\infty}B_n\frac{x^n}{n!}$.
  $S_m(n)=\sum_{k=1}^{n}k^m=\frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k^+n^{m+1-k}$
- Stirling numbers of the second kind
  Partitions of n distinct elements into exactly k groups.
  $S(n,k)=S(n-1,k-1)+kS(n-1,k), S(n,1)=S(n,n)=1$
  $S(n,k)=\frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i}i^n$
  $x^n=\sum_{i=0}^{n}S(n,i)(x)_i$
- Pentagonal number theorem
  $\prod_{n=1}^{\infty}(1-x^n)=1+\sum_{k=1}^{\infty}(-1)^k\left(x^{k(3k+1)/2}+x^{k(3k-1)/2}\right)$
- Catalan numbers
  $C_0=1,\quad C_{n+1}=C_n 2(2n+1)/(n+2)$
  $C_n^{(k)}=\frac{1}{(k-1)n+1}\binom{kn}{n}, C^{(k)}(x)=1+x[C^{(k)}(x)]^k$
- Eulerian numbers
  Number of permutations $\pi\in S_n$ with exactly k descents ($\pi(j)>\pi(j+1)$).
  $E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)$
  $E(n,0)=E(n,n-1)=1$
  $E(n,k)=\sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$

7 Polynomial

7.1 Fast Fourier Transform [2bea3f]

Iterative complex FFT and integer convolution via mult, $O(n\log n)$. fft requires a positive power-of-two length; mult returns empty if either coefficient vector is empty, otherwise exactly $|a|+|b|-1$ rounded coefficients. Double precision limits coefficient size.

```
using cd = complex<double>;
const double pi = acos(-1);
void fft(vector<cd> &vec, bool inv=false) {
  // The input length must be a positive power of two.
  int n = vec.size();
  for(int i=0, j=0; i<n; ++i) { // j = 反過來模擬加法
    if(i<j) swap(vec[i],vec[j]);
    for(int bit=n>>1; (j^=bit)<bit; bit>>=1);
  }
  for(int len=2; len<=n; len<<=1) { // 循環長度 = 2pi
    double ang = 2*pi/len*(inv?-1:1);
    cd wlen(cos(ang), sin(ang));
    for(int i=0; i<n; i+=len) {
      cd w=1;
      for(int j=0; j<len/2; ++j) {
      // i = 定範圍, j = 去跑 merge
        cd a = vec[i+j], b = w*vec[i+len/2+j];
        vec[i+j]       = a+b;
        vec[i+len/2+j] = a-b;
        w *= wlen;
      }
    }
  }
  if(inv) for(auto &i : vec) i/=n;
}
#define ifft(vec) fft(vec,true)
vector<int> mult(const vector<int>& a,
  const vector<int>& b) {
  if (a.empty() || b.empty()) return {};
  int sz = a.size()+b.size()-1, n=1; while(n<sz) n<<=1;
  vector<cd> ca(a.begin(),
    a.end()), cb(b.begin(),b.end());
  ca.resize(n), cb.resize(n), fft(ca), fft(cb);
  for(int i=0; i<n; ++i) ca[i] *= cb[i];
  ifft(ca);
  vector<int> res(sz);
  for(int i=0; i<sz; ++i) res[i] = round(ca[i].real());
  return res;
}
vector<int> c = mult(a, b); // exact output length
// rounded from double; keep coefficients moderate
```

Prime	Root	Prime	Root
7681	17	167772161	3
12289	11	104857601	3
40961	3	985661441	3
65537	3	998244353	3
786433	10	1107296257	10
5767169	3	2013265921	31
7340033	3	2810183681	11
23068673	3	2885681153	3
469762049	3	605028353	3

7.2 Number Theory Transform* [5c4186]

Iterative NTT and convolution via mult, $O(n\log n)$ modulo 998244353. Direct ntt input lengths must be positive powers of two up to 2^{23}; coefficients are residues modulo 998244353. mult returns empty if either vector is empty. The listing defines int as long long; contain or undefine the macro after use.

```
#define int long long
const int mod=998244353, g=3;
int pwr(int a, int b, int ret=1) {
  do ret = b&1?ret*a%mod:ret, a=a*a%mod;
  while(b>>=1); return ret;
}
void ntt(vector<int> &vec, bool inv=false) {
  // The input length
       must be a positive power of two (at most 2^23).
  int n = vec.size();
  for(int i=0, j=0; i<n; ++i) { // j = 反過來模擬加法
    if(i<j) swap(vec[i],vec[j]);
    for(int bit=n>>1; (j^=bit)<bit; bit>>=1);
  }
  for(int len=2; len<=n; len<<=1) { // 循環長度 = mod-1
    int wlen = pwr(g,(mod-1)/len);
    if(inv) wlen = pwr(wlen,mod-2);
    for(int i=0; i<n; i+=len) {
      int w=1;
      for(int j=0; j<len/2; ++j) {
      // i = 定範圍, j = 去跑 merge
        int a = vec[i+j], b = w*vec[i+len/2+j]%mod;
        vec[i+j]       = a+b<mod ? a+b : a+b-mod;
        vec[i+len/2+j] = a-b>=0  ? a-b : a-b+mod;
        w = w*wlen%mod;
      }
    }
  }
  if(inv) {
    int invn = pwr(n,mod-2);
    for(auto &i : vec) i = i*invn%mod;
  }
}
#define intt(vec) ntt(vec,true)
vector<int> mult(vector<int> a, vector<int> b) {
  if (a.empty() || b.empty()) return {};
  int sz = a.size()+b.size()-1, n=1; while(n<sz) n<<=1;
  a.resize(n), b.resize(n), ntt(a), ntt(b);
  for(int i=0; i<n; ++i) a[i] = a[i]*b[i] % mod;
  intt(a), a.resize(sz); return a;
}
vector
  <ll> c = mult(a, b); // vector<ll> a,b; mod 998244353
```

7.3 Fast Walsh Transform* [95c282]

or/and/xor convolution, $O(n\log n)$; file implements or. Subset convolution: $O(2^L L^2), L\leq 21$. $n=2^k, k\leq 29$; C++20; 528 MiB scratch. int arithmetic must fit.

```
// x=a[j], y=a[j+L/2]: OR y+=x*op; AND x+=y*op.
// XOR: (x,y)=(x+y,x-y); inverse divides both by 2.
// n=2^k, 0<=k<=29; op=1/-1; O(n log n); int ops must fit.
void fwt(int *a, int n, int op) { // OR
  for (int L = 2; L <= n; L <<= 1)
    for (int i = 0; i < n; i += L)
      for (int j = i; j < i + L / 2; ++j)
        a[j + L / 2] += a[j] * op;
}
const int N = 21;
int f[N + 1][1 << N], g[N + 1][1 << N], h[N + 1][1 << N];
void subset_convolution(int *a, int *b, int *c, int L) {
  // c[k]=sum(a[i]*b[j]) for i|j=k, i&j=0; 0<=L<=N.
  // a,b,c: 2^L ints outside f/g/h; c may alias a or b.
  // O(2^L L^2) time; three (N+1)*2^N int arrays.
  int n = 1 << L;
  for (int i = 0; i <= L; ++i)
    fill_n(f[i],n,0), fill_n(g[i],n,0), fill_n(h[i],n,0);
  for (unsigned i = 0; i < n; ++i) {
    int k = popcount(i); f[k][i] = a[i], g[k][i] = b[i];
  }
  for (int i = 0; i <= L; ++i)
    fwt(f[i], n, 1), fwt(g[i], n, 1);
  for (int i = 0; i <= L; ++i)
    for (int j = 0; j <= i; ++j)
      for (int x = 0; x < n; ++x)
```

```
      h[i][x] += f[j][x] * g[i - j][x];
  for (int i = 0; i <= L; ++i) fwt(h[i], n, -1);
  for (unsigned i = 0; i < n; ++i) c[i]=h[popcount(i)][i];
}
fwt(a, n, 1); fwt(b, n, 1); // n = 2^k
for (int i = 0; i < n; ++i) a[i] *= b[i];
fwt(a, n, -1); // inv: or/and -1, xor 1/2
subset_convolution(a, b, c, L); // O(2^L L^2)
```

7.4 Value Polynomial [57b515]

Polynomial held as consecutive point values; Lagrange eval and prefix-sum lift in $O(n)$. Requires prime-field mint with canonical .v, ifac, and inegfac precomputed through size $-1 < P$. Before raise, size $< P$.

```
struct Poly { // Prime field mint; canonical .v; 1<=n<=P.
  mint base; // f(base+i)=poly[i]; deg(f)<n=SZ(poly).
  vector<mint> poly; // Tables through n-1:
  // ifac[i]=1/i!, inegfac[i]=(-1)^i/i!
  Poly(mint b = 0, mint x = 0): base(b), poly(1, x) {}
  mint get_val(const mint &x) { // O(n) time/space
    int n = SZ(poly); mint t = x - base, rt = 0, suf = 1;
    if (t.v < n) return poly[t.v];
    vector<mint> pre(n, 1);
    for (int i = 1; i < n; ++i)
      pre[i] = pre[i - 1] * (t - (i - 1));
    for (int i = n - 1; i >= 0; --i) {
      rt += (poly[i] * ifac[i]) *
        (inegfac[n - 1 - i] * pre[i]) * suf;
      suf *= t - i;
    } return rt;
  }
  void raise() { // n<P; g(x)=sum_{k=base}^x f(k); O(n)
    if (SZ(poly) == 1 && poly[0] == 0) return;
    mint nw = get_val(base + SZ(poly)); poly.pb(nw);
    for (int i = 1; i < SZ(poly); ++i)
      poly[i] += poly[i - 1];
  }
};
Poly f(base, y0); // f(base+i) = poly[i]
f.poly = {y0, y1, y2}; // consecutive values
mint v = f.get_val(x); // any x, O(n)
f.raise(); // f becomes prefix sum, deg+1
```

7.5 Newton's Method

Given a formal power series $F(x)$ where

$$F(x)=\sum_{i=0}^{\infty}\alpha_i(x-\beta)^i$$

for a constant β. Assume $F(\beta)=0$ and $\alpha_1=F'(\beta)$ is a unit, and set $Q_0=\beta$. A formal-series root can be found iteratively: if $F(Q_k)=0 \pmod{x^{2^k}}$, then since $F'(Q_k)$ remains a unit modulo x^{2^k},

$$Q_{k+1}=Q_k-\frac{F(Q_k)}{F'(Q_k)} \pmod{x^{2^{k+1}}}.$$

8 Geometry

8.1 Basic [7486c6]

Point/line primitives the rest of this section depends on: * is dot, ^ is cross; project, reflect, segment inside, SegmentIntersect (endpoints included). Intersect **divides by zero on parallel lines.** Unit vectors and projection/reflection require nonzero directions.

```
// unit: nonzero; project/reflect/get_ratio: nonzero line.
// Intersect
    : nonparallel; segment tests include endpoints.
bool same(double a, double b) { return abs(a - b) < eps; }
// eps>0, compatible sign; finite, reliable arithmetic.
struct P {
  double x, y;
  P() : x(0), y(0) {}
  P(double x, double y) : x(x), y(y) {}
  P operator + (P b) const { return {x + b.x, y + b.y}; }
  P operator - (P b) const { return {x - b.x, y - b.y}; }
  P operator * (double b) const { return {x * b, y * b}; }
  P operator / (double b) const { return {x / b, y / b}; }
  double
       operator*(P b) const { return x * b.x + y * b.y; }
  double
       operator^(P b) const { return x * b.y - y * b.x; }
  double abs() const { return hypot(x, y); }
  P unit() const { return *this / abs(); }
  P rot(double o) const { double c = cos(o), s = sin(o);
    return {c * x - s * y, s * x + c * y};
  }
  double angle() const { return atan2(y, x); }
};
struct L { // ax + by + c = 0
  double a, b, c, o; P pa, pb;
  L() : a(0), b(0), c(0), o(0), pa(), pb() {}
  L(P pa, P pb) : a(pa.y - pb.y), b(pb.x - pa.x),
    c(pa ^ pb), o(atan2(-a, b)), pa(pa), pb(pb) {}
  P project(P p) { P d = pb - pa;
    return pa + d * (d * (p - pa) / (d * d)); }
  P reflect(P p) { return p + (project(p) - p) * 2; }
  double get_ratio(P p) { P d = pb - pa;
    return (p - pa) * d / (d * d); }
  bool inside(P p) { return
    min(pa.x, pb.x) <= p.x && p.x <= max(pa.x, pb.x) &&
    min(pa.y, pb.y) <= p.y && p.y <= max(pa.y, pb.y) &&
    same(a * p.x + b * p.y, -c); }
};
bool SegmentIntersect(P p1, P p2, P p3, P p4) {
  if (max(p1.x, p2.x) < min(p3.x, p4.x) ||
      max(p3.x, p4.x) < min(p1.x, p2.x) ||
      max(p1.y, p2.y) < min(p3.y, p4.y) ||
      max(p3.y, p4.y) < min(p1.y, p2.y)) return false;
  return sign((p3 - p1) ^ (p4 - p1)) *
      sign((p3 - p2) ^ (p4 - p2)) <= 0 &&
    sign((p1 - p3) ^ (p2 - p3)) *
      sign((p1 - p4) ^ (p2 - p4)) <= 0;
}
bool parallel(L x, L y) {
  return same(x.a * y.b, x.b * y.a); }
P Intersect(L x, L y) {
  return P(-x.b * y.c + x.c * y.b, x.a * y.c - x.c * y.a)
    / (-x.a * y.b + x.b * y.a);
}
```

8.2 Delaunay Triangulation [94d44a]

Divide-and-conquer quad-edge Delaunay, $O(n \log n)$. **Points must be distinct**, integer, $|x|, |y| \le 5 \cdot 10^8$ (__int128 in-circle test). Returns CCW triangles with no input point strictly inside any circumcircle; empty if fewer than 3 points or all collinear. Cocircular points give one valid triangulation.

```
// Distinct integer points, |x|,|y|<=5e8; O(n log n).
// CCW
    triangles; empty if n<3 or collinear. Sizes fit int.
namespace delaunay {
struct Point {
  ll x, y; Point(ll x = 0, ll y = 0) : x(x), y(y) {}
  Point operator-(Point b) const { return {x-b.x,y-b.y}; }
  ll operator
      ^(Point b) const { return x * b.y - y * b.x; }
  ll Norm() const { return x * x + y * y; }
  bool operator<(Point b) const {
    return tie(x,y) < tie(b.x,b.y); }
  bool operator==(Point b) const {
    return x==b.x && y==b.y; }
};
const Point kA(LLONG_MAX, LLONG_MAX); // Sentinel
bool InCircle(Point p, Point a, Point b, Point c) {
  a = a - p, b = b - p, c = c - p;
  __int128 x = __int128(a.Norm()) * (b ^ c) +
    __int128(b.Norm()) * (c ^ a) +
    __int128(c.Norm()) * (a ^ b);
  ll s = (b - a) ^ (c - a);
  return s > 0 ? x > 0 : s < 0 && x < 0;
}
struct Quad {
  bool mark = 0; Point p;
  Quad *o = 0, *rot = 0;
  Quad(Point p) : p(p) {}
  Point F() { return r()->p; }
  Quad* r() { return rot->rot; }
  Quad* prev() { return rot->o->rot; }
  Quad* next() { return r()->prev(); }
};
deque<vector<array<Quad, 4>>> edges; // Stable blocks.
Quad* MakeEdge(Point s, Point t) {
  if (edges.empty() || edges.back().size() == 1024)
    edges.emplace_back().reserve(1024);
  edges.back().push_back(
    {Quad(s), Quad(kA), Quad(t), Quad(kA)});
  auto &q = edges.back().back();
  for (int i = 0; i < 4; ++i)
    q[i].o = &q[-i & 3], q[i].rot = &q[(i + 1) & 3];
  return &q[0];
}
void Splice(Quad* a, Quad* b) {
  swap(a->o->rot->o, b->o->rot->o), swap(a->o, b->o); }
Quad* Connect(Quad* a, Quad* b) {
  Quad* q = MakeEdge(a->F(), b->p);
  Splice(q, a->next()), Splice(q->r(), b); return q;
}
void Kill(Quad* e) {
  Splice(e, e->prev()), Splice(e->r(), e->r()->prev()); }
pair<
    Quad*,Quad*> Dfs(const vector<Point>& s,int l,int r) {
  if (r - l <= 3) {
    Quad* a = MakeEdge(s[l], s[l + 1]);
    if (r - l == 2) return {a, a->r()};
    Quad* b = MakeEdge(s[l+1], s[l+2]); Splice(a->r(), b);
```

```
    ll side = (s[l + 1] - s[l]) ^ (s[l + 2] - s[l]);
    Quad* c = side ? Connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r()};
  }
  int m = (l + r) / 2; auto [ra, a] = Dfs(s, l, m);
  auto [b, rb] = Dfs(s, m, r);
  while ((((a->F()-b->p)^(a->p-b->p)) < 0 &&
          (a = a->next())) ||
         (((b->F()-a->p)^(b->p-a->p)) > 0 &&
          (b = b->r()->o)));
  Quad* base = Connect(b->r(), a);
  auto Valid = [&](Quad* e) {
    return ((base->F()-e->F())^(base->p-e->F())) > 0; };
  if (a->p == ra->p) ra = base->r();
  if (b->p == rb->p) rb = base;
  while (1) {
    Quad *lc = base->r()->o, *rc;
    if (Valid(lc))
      while (InCircle(lc->o->F(),
                      base->F(), base->p, lc->F()))
        { Quad* t = lc->o; Kill(lc); lc = t; }
    rc = base->prev();
    if (Valid(rc))
      while (InCircle(rc->prev()->F(),
                      base->F(), base->p, rc->F()))
        { Quad* t = rc->prev(); Kill(rc); rc = t; }
    bool lv = Valid(lc), rv = Valid(rc);
    if (!lv && !rv) break;
    if (!lv || (rv &&
        InCircle(rc->F(), rc->p, lc->F(), lc->p)))
      base = Connect(rc, base->r());
    else base = Connect(base->r(), lc->r());
  }
  return {ra, rb}; }
vector<array<Point, 3>> Triangulate(vector<Point> pts) {
  if (SZ(pts) < 3) return {};
  if (all_of(pts.begin() + 2, pts.end(), [&](Point p) {
    return ((pts[1]-pts[0])^(p-pts[0]))==0; })) return {};
  sort(ALL(pts));
  size_t start = edges.size();
  size_t used = start ? edges.back().size() : 0;
  Quad* e = Dfs(pts, 0, SZ(pts)).first;
  vector<Quad*> q = {e}; q.reserve(6 * pts.size());
  vector<array<Point,3>> res; res.reserve(2 * pts.size());
  while (((e->F()-e->o->F())^(e->p-e->o->F())) < 0)
    e = e->o;
  auto Add = [&]() { Quad* c = e;
    do c->mark = 1, q.pb(c->r()), c = c->next();
    while
        (c != e); }; Add(); // Mark the outer face first.
  for (int i = 0; i < SZ(q); ++i) if (!(e = q[i])->mark)
    res.pb({e->p, e->F(), e->next()->F()}), Add();
  while (edges.size() > start) edges.pop_back();
  while (start && edges.back().size() > used)
    edges.back().pop_back();
  return res; }
} // namespace delaunay
vector<delaunay::Point> p(n);  // distinct integer points
auto tri = delaunay::Triangulate(p); // CCW triangles
// Delaunay contains the Euclidean MST; Voronoi vertices
// are the circumcentres of these triangles.
```

8.3 Sector Area [a291d2]

Area of the sector between two nonzero vectors, $O(1)$; $r \geq 0$. **Returns the minor arc**—subtract from the full disc for the major one.

```
// Minor sector; a,b!=0; r>=0; finite arithmetic; O(1).
double SectorArea(P a, P b, double r) {
  return r * r *
    abs(remainder(a.angle() - b.angle(), 2 * pi)) / 2;
}
```

8.4 Half Plane Intersection [8d0ad8]

Intersection polygon of half planes in $O(n \log n)$; feasible side is *left* of each nonzero line. Requires a bounded positive-area region (add a box); equal directions need exactly equal o. Sorts input in place.

```
bool jizz(L l1, L l2, L l3) { P p = Intersect(l2, l3);
  return ((l1.pb - l1.pa) ^ (p - l1.pa)) < -eps; }
// Equal directions need equal o; finite, reliable tests.
bool cmp(const L &a, const L &b) { return a.o == b.o ?
  a.c/hypot(a.a, a.b) < b.c/hypot(b.a, b.b) : a.o < b.o; }
// Nonzero
    lines; feasible left; bounded area>0. O(n log n)
vector<P> HPI(vector<L> &ls) {
  sort(ls.begin(), ls.end(), cmp); deque<L> dq;
  for (L l : ls) {
    if (!dq.empty() && l.o == dq.back().o) continue;
    while (dq.size() > 1 &&
        jizz(l, dq.back(), dq[dq.size() - 2]))
      dq.pop_back();
    while (dq.size() > 1 && jizz(l, dq[0], dq[1]))
      dq.pop_front();
    dq.push_back(l);
  }
  while (dq.size() > 1 && jizz(dq.front(), dq.back(),
      dq[dq.size() - 2])) dq.pop_back();
  while (dq.size() > 1 && jizz(dq.back(), dq[0], dq[1]))
    dq.pop_front();
  if (dq.size() < 3) return {};
  vector<P> rt(dq.size());
  for (int i = 0; i < (int)dq.size(); ++i)
    rt[i] = Intersect(dq[i], dq[(i + 1) % dq.size()]);
  return rt; }
vector<L> hs = {L({0,0},{1,0}), L({1,0},{1,1}),
                L({1,1},{0,1}), L({0,1},{0,0})};
auto poly = HPI(hs); // unit square; left side is feasible
```

8.5 Rotating Sweep Line [c9e1fd]

Sweeps all $\binom{n}{2}$ directions in $O(n^2 \log n)$; points must be distinct with no three collinear. rotatingSweepLineOrder returns the final order, while the legacy void wrapper preserves the old call shape.

```
// Distinct points, no 3 collinear; n fits int.
// O(n^2 log n) time, O(n^2) event storage.
vector
    <int> rotatingSweepLineOrder(const vector<pii> &ps) {
  int n = ps.size(); vector<int> id(n), pos(n);
  struct Event { int a, b; ll x, y; };
  vector<Event> line; line.reserve(1LL * n * (n - 1) / 2);
  for (int i = 0; i < n; ++i)
    for (int j = i + 1; j < n; ++j) {
      ll x = ll(ps[i].X) - ps[j].X,
        y = ll(ps[i].Y) - ps[j].Y;
      // x>0 or x=0,y>0; vertical events last.
      if (x < 0 || (x == 0 && y < 0)) x = -x, y = -y;
      line.push_back({i, j, x, y});
    }
  sort(ALL(line), [](const Event &a, const Event &b) {
    return (__int128)a.y * b.x < (__int128)b.y * a.x;
  });
  iota(ALL(id), 0);
  sort(ALL(id), [&](int a, int b) {
    return ps[a] < ps[b]; });
  for (int i = 0; i < n; ++i) pos[id[i]] = i;
  for (auto e : line)
    swap(id[pos[e.a]], id[pos[e.b]]),
      swap(pos[e.a], pos[e.b]);
  return id;
}
void rotatingSweepLine(vector<pii> &ps) {
  rotatingSweepLineOrder(ps); }
vector<pair<int,int>> p={{0,0},{2,0},{1,1}};
vector<int> order=rotatingSweepLineOrder(p); // point ids
```

8.6 Triangle Center [19fc6a]

Circumcentre, centroid, orthocentre (via the Euler line) and incentre, each $O(1)$. **Degenerate (collinear) input divides by zero.**

```
// Noncollinear; finite arithmetic; computed d!=0; O(1).
Point TriangleCircumCenter(Point a, Point b, Point c) {
  Point u = b - a, v = c - a, w = c - b;
  double x = u.x*u.x + u.y*u.y, y = v.x*v.x + v.y*v.y,
    z = w.x * w.x + w.y * w.y;
  // Anchor on a shortest side.
  if (z < min(x, y)) a = b, u = u * -1, v = w, y = z;
  double d = 2 * Cross(u, v);
  return a + Point((v.y * x - u.y * y) / d,
    (u.x * y - v.x * x) / d);
}
Point TriangleMassCenter(Point a, Point b, Point c) {
  return (a + b + c) / 3.0; }
Point TriangleOrthoCenter(Point a, Point b, Point c) {
  return
      (a + b + c) - TriangleCircumCenter(a, b, c) * 2.0;
}
Point TriangleInnerCenter(Point a, Point b, Point c) {
  double
      la = len(b - c), lb = len(a - c), lc = len(a - b);
  return (a * la + b * lb + c * lc) / (la + lb + lc);
}
```

8.7 Polygon Center [20363a]

Area centroid of a nonzero-area simple polygon in $O(|p|)$ by signed-area weighting, so concave shapes are fine. Processes the whole vector; parameter n is unused.

```
// Simple CW/CCW polygon; nonzero area, finite arithmetic.
// |p| fits int; n unused; reliable doubles; O(|p|).
Point BaryCenter(vector<Point> &p, int n) {
  Point res(0, 0), u = p[1] - p[0]; double s = 0;
  for (int i = 2; i < (int)p.size(); ++i) {
    Point v = p[i] - p[0]; double t = Cross(u, v);
    s += t, res.x += (u.x + v.x) * t,
      res.y += (u.y + v.y) * t, u = v;
  } return Point(p[0].x + res.x / (3 * s),
    p[0].y + res.y / (3 * s));
}
```

8.8 Maximum Triangle [dec5cb]

Largest triangle inscribed in a convex polygon in $O(n^2)$: fix one vertex and advance the other two monotonically. Hull indices must be in cyclic order (CW or CCW).

```
// Cyclic CW/CCW convex; finite arithmetic; O(chnum^2).
double ConvexHullMaxTriangleArea(
    Point p[], int res[], int chnum) {
  double area = 0;
  auto twice_area = [&](int i, int j, int k) {
    return fabs(Cross(p[res[j]] - p[res[i]],
      p[res[k]] - p[res[i]]));
  };
  for (int i = 0; i + 2 < chnum; ++i) {
    int k = i + 2;
    for (int j = i + 1; j + 1 < chnum; ++j) {
      if (p[res[i]].x == p[res[j]].x &&
          p[res[i]].y == p[res[j]].y) continue;
      k = max(k, j + 1);
      double cur = twice_area(i, j, k), next;
      while (k + 1 < chnum &&
          (next = twice_area(i, j, k + 1)) >= cur)
        ++k, cur = next;
      area = max(area, cur);
    }
  } return area / 2;
}
```

8.9 Point in Convex Polygon [489d59]

Fan binary search, $O(\log K)$ per query; integer IPoint with `__int128` cross products. Returns 0 outside, 1 on the boundary, 2 strictly inside. Input is CCW, without repeated or collinear intermediate vertices (do not repeat the first vertex). Empty/point/segment inputs supported; all coordinates, including queries, have absolute value $\le 10^{18}$.

```
// CCW
    convex; no repeated/intermediate collinear vertices.
// No closing duplicate. Queries too: |x|,|y|<=1e18.
// O(log n): 0 outside, 1 boundary, 2 strictly inside.
struct IPoint { long long x, y; };
__int128 cross(IPoint a, IPoint b, IPoint c) {
  return ((__int128)b.x - a.x) * ((__int128)c.y - a.y)
       - ((__int128)b.y - a.y) * ((__int128)c.x - a.x);
}
bool onSegment(IPoint a, IPoint b, IPoint p) {
  return cross(a, b, p) == 0 &&
    min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
    min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
int pointInConvex(const vector<IPoint>& ch, IPoint p) {
  int n = ch.size();
  if (n < 3) return n && onSegment(ch[0], ch[n - 1], p);
  auto a = cross(ch[0], ch[1], p),
       b = cross(ch[0], ch[n - 1], p);
  if (a < 0 || b > 0) return 0;
  if (!a) return onSegment(ch[0], ch[1], p);
  if (!b) return onSegment(ch[0], ch[n - 1], p);
  auto it = partition_point(ch.begin() + 1, ch.end(),
    [&](IPoint q) { return cross(ch[0], q, p) >= 0; });
  auto c = cross(*prev(it), *it, p);
  return c < 0 ? 0 : (c == 0 ? 1 : 2);
}
vector<IPoint> ch = {{0,0},{4,0},{4,4},{0,4}}; // CCW
int inside = pointInConvex(ch, {2,2}); // 2
int boundary = pointInConvex(ch, {4,2}); // 1
int outside = pointInConvex(ch, {5,2}); // 0
```

8.10 Circle [8390b2]

Nonnegative-radius circle intersection points and area, circle-segment crossing, and circle∩triangle area. All $O(1)$. Coincident circles return an empty finite-point list; a zero-length segment is treated as one point. Triangle circle is at the origin; vector takes 3 vertices.

```
struct C { // r>=0; finite arithmetic; O(1) operations
  P c; double r;
  C(P c = P(0, 0), double r = 0) : c(c), r(r) {}
};
vector<P> Intersect(C a, C b) {
  if (a.r > b.r) swap(a, b);
  double d = (a.c - b.c).abs();
  if (same
      (d, 0)) return {}; // Coincident: no finite list.
  if (same(a.r + b.r, d))
    return {a.c + (b.c - a.c) / d * a.r};
  if (same(d + a.r, b.r))
    return {a.c - (b.c - a.c) / d * a.r};
  if (a.r + b.r <= d || d + a.r < b.r) return {};
  double z = d < b.r / 2
    ? sq(d) + (a.r - b.r) * (a.r + b.r)
    : sq(a.r) + (d - b.r) * (d + b.r);
  double h = z / (2 * a.r * d);
  P u = (b.c - a.c) / d, v = u * (a.r * h),
    w = P(-u.y, u.x) * (a.r * sqrt(max(0., 1 - h * h)));
  return {a.c + v + w, a.c + v - w};
}
double IntersectArea(C a, C b) {
  if (a.r > b.r) swap(a, b);
  double d = (a.c - b.c).abs();
  if (d >= a.r + b.r - eps) return 0;
  if (d + a.r <= b.r + eps) return sq(a.r) * acos(-1);
  double z = d < b.r / 2
    ? sq(d) + (a.r - b.r) * (a.r + b.r)
    : sq(a.r) + (d - b.r) * (d + b.r), x = z / (2 * d),
    y = sqrt(max(0., sq(a.r) - sq(x))); // Chord (x,+/-y).
  auto cap = [](double r, double t) { // Small-angle cap
    return sq(r) * (t < 1e-3
      ? t * t * t * (2. / 3 - 2. / 15 * t * t)
      : t - sin(2 * t)) / 2);
  }; return cap(a.r, atan2(y, x))
         + cap(b.r, atan2(y, d - x));
}
// r>=0; endpoint tolerance eps in coordinate length.
// Drop parameter-range checks for an infinite line.
vector<P> CircleCrossLine(P a, P b, P o, double r) {
  P u = b - a, v = a - o; double A = u * u;
  if (!A) // Zero-length segment is a point.
    return same((a - o).abs(), r)
      ? vector<P>{a} : vector<P>{};
  // Compensated
       cross preserves small tilted-line offsets.
  double t0 = u.y * v.x,
    z = fma(u.x, v.y, -t0) - fma(u.y, v.x, -t0);
  double mid = -(u * v) / A, d = sq(r) - sq(z) / A,
    tol = eps / sqrt(A);
  P p = P(-u.y, u.x) * (z / A);
  auto within = [&](double t) {
    return -tol <= t && t <= 1 + tol; };
  vector<P> t;
  if (d >= -eps) {
    double root = sqrt(max(0., d) / A),
      i = mid - root, j = mid + root;
    if (within(i))
      t.reserve(2), t.push_back(o + p - u * root);
    if (root > 0 && within(j))
      t.push_back(o + p + u * root);
  } return t;
}
// r>=0; unsigned disk cap OAB; last root exits from a.
double AreaOfCircleTriangle(P a, P b, double r) {
  bool ina = a.abs() < r, inb = b.abs() < r;
  if (ina && inb) return abs(a ^ b) / 2;
  auto p = CircleCrossLine(a, b, P(0, 0), r);
  if (ina) return SectorArea(b, p.back(), r)
                + abs(a ^ p.back()) / 2;
  if (inb) return SectorArea(p[0], a, r)
                + abs(p[0] ^ b) / 2;
  if (p.size() == 2) return SectorArea(a, p[0], r) +
    SectorArea(p[1], b, r) + abs(p[0] ^ p[1]) / 2;
  return SectorArea(a, b, r);
}
// r>=0; any triangle, exactly 3 vertices; disk at origin.
double AreaOfCircleTriangle(
  const vector<P> &ps, double r) {
  double ans = 0;
  for (int i = 0; i < 3; ++i) {
    int j = (i + 1) % 3; double z = ps[i] ^ ps[j];
    if (z) ans += AreaOfCircleTriangle(ps[i], ps[j], r) *
      (z > 0 ? 1 : -1);
  } return abs(ans);
}
```

8.11 Tangent of Circles and Points to Circle [9783a0]

Common tangents of two circles (0/1/2/3/4 by configuration) and tangents from a point, $O(1)$. Positive radii; eps-close centres return an empty finite-line list.

```
// eps>0, radii>0; finite math, reliable distance tests.
// Resolved contacts/directions; eps-close centers: empty.
vector<L> tangent(C a, C b) {
  if (a.r < b.r) swap(a, b);
  vector<L> z; double d = (a.c - b.c).abs();
  if (same(d, 0) || d + b.r < a.r) return z;
  P u = (b.c - a.c) / d; z.reserve(4);
  for (double side : {1., -1.}) { // External first.
    bool touch = side == 1 ? same(d + b.r, a.r) :
      same(d, a.r + b.r);
    if (touch) { P i = u * a.r, j(i.y, -i.x);
      z.emplace_back(a.c + i, a.c + i + j);
      if (side == 1) break;
    } else if (side == 1 || d > a.r + b.r) {
      double h = (a.r - side * b.r) / d;
      double t = sqrt(max(0., 1 - h * h));
      for (double k : {t, -t}) {
        P v = u * h + P(-u.y, u.x) * k;
```

```
        z.emplace_back(a.c + v * a.r,
                       b.c + v * (side * b.r));
      }
    }
  } return z;
}
vector<L> tangent(C c, P p) {
  vector<L> z; double d = (p - c.c).abs();
  if (d > 0 && same(d, c.r))
    z.emplace_back(p, p + (p - c.c).rot(pi / 2));
  else if (d > c.r) {
    double h = c.r / d;
    double t = sqrt(max(0., (1 - h) * (1 + h)));
    P u = (p - c.c) / d; z.reserve(2);
    for (double k : {t, -t})
      z.emplace_back(
        c.c + (u * h + P(-u.y, u.x) * k) * c.r, p);
  } return z;
}
```

8.12 Area of Union of Circles [7d0bbd]

Union area by integrating uncovered arcs with Green's theorem, $O(n^2 \log n)$. Nonnegative radii; **eps-coincident circles must be removed first**: no pair with both centre distance and radius difference $\le$ eps.

```
void AppendCoverSegment(C &a, C &b,
    vector<pair<double, double>> &res) {
  if (max(abs(a.c.x - b.c.x), abs(a.c.y - b.c.y)) >
      a.r + b.r + eps) return;
  double d = (a.c - b.c).abs();
  if (same(a.r + b.r, d)) return;
  if (d <= abs(a.r - b.r) + eps) {
    if (a.r < b.r) res.emplace_back(0, 2 * pi);
  } else if (d < abs(a.r + b.r) - eps) {
    double o = acos((sq(a.r) + sq(d) - sq(b.r)) /
      (2 * a.r * d)), z = (b.c - a.c).angle();
    if (z < 0) z += 2 * pi; double l = z - o, r = z + o;
    if (l < 0) l += 2 * pi; if (r > 2 * pi) r -= 2 * pi;
    if (l > r) res.emplace_back(l, 2 * pi),
      res.emplace_back(0, r);
    else res.emplace_back(l, r);
  }
}
vector<pair<double, double>> CoverSegment(C &a, C &b) {
  vector<pair<double, double>> v;
  AppendCoverSegment(a, b, v); return v; }
// r>=0, n fits int; no pair with both d, |dr|<=eps.
// Finite arithmetic; geometry resolved to tolerance.
// O(n^2 log n) worst case; zero radii skipped.
double CircleUnionArea(vector<C> c) {
  if (!c.empty()) { P o = c[0].c;
    for (auto &v : c) v.c = v.c - o; }
  sort(ALL(c), [](C a, C b) { return a.r > b.r; });
  int n = c.size(); while (n && c[n - 1].r == 0) --n;
  double a = 0; vector<pair<double, double>> s;
  for (int i = 0; i < n; ++i) {
    double w = 0; s = {{2 * pi, 2 * pi}};
    for (int j = 0; j < n; ++j) if (i != j) {
      AppendCoverSegment(c[i], c[j], s);
      if (s.back() == make_pair(0., 2 * pi)) {
        s.clear(); break; }
    }
    sort(ALL(s));
    auto F = [&](double t) { return c[i].r * (c[i].r * t +
      c[i].c.x * sin(t) - c[i].c.y * cos(t)); };
    for (auto [l, r] : s) {
      if (l > w) a += F(l) - F(w);
      w = max(w, r);
    }
  } return a * 0.5; }
```

8.13 Minimun Distance of 2 Polygons [830a94]

Minimum distance of nonempty convex polygons in cyclic CW/CCW order, $O(n+m)$ time/space via Minkowski difference. Returns 0 for overlap or touching; point/segment and repeated/collinear boundary vertices supported.

```
double _point_seg_dist(Point p, Point a, Point b) {
  double dx = b.x - a.x, dy = b.y - a.y;
  double z = dx * dx + dy * dy;
  double t = z ? clamp(((p.x - a.x) * dx +
    (p.y - a.y) * dy) / z, 0., 1.) : 0;
  return hypot(p.x - a.x - t * dx, p.y - a.y - t * dy); }
double _cross(Point a, Point b, Point c) {
  return (b.x - a.x) * (c.y - a.y) -
    (b.y - a.y) * (c.x - a.x); }
bool _on_segment(Point a, Point b, Point p) {
  return fabs(_cross(a, b, p)) <= 1e-10 &&
    min(a.x, b.x) - 1e-10 <= p.x &&
    p.x <= max(a.x, b.x) + 1e-10 &&
    min(a.y, b.y) - 1e-10 <= p.y &&
    p.y <= max(a.y, b.y) + 1e-10; }
bool _segments_intersect(Point a, Point b,
    Point c, Point d) {
  double x1 = _cross(a, b, c), x2 = _cross(a, b, d),
    x3 = _cross(c, d, a), x4 = _cross(c, d, b);
  return _on_segment(a, b, c) || _on_segment(a, b, d) ||
    _on_segment(c, d, a) || _on_segment(c, d, b) ||
    ((x1 > 0) != (x2 > 0) && (x3 > 0) != (x4 > 0)); }
bool _inside_convex(Point p, Point poly[], int n) {
  // Nonzero area.
  int sign = 0; for (int i = 0; i < n; ++i) {
    double c = _cross(poly[i], poly[(i + 1) % n], p);
    if (fabs(c) > 1e-10) { int s = c > 0 ? 1 : -1;
      if (sign && sign != s) return false;
      sign = s; }
  } return true;
}
// Nonempty cyclic convex CW/CCW; repeats OK.
// Finite arithmetic; reliable signs and 1e-10 tests.
double TwoConvexHullMinDist(Point P[], Point Q[],
    int n, int m) {
  auto prepare = [&](Point p[], int n, int s) {
    vector<Point> a; a.reserve(n + 1); a.assign(p, p + n);
    for (auto &v : a) v.x *= s, v.y *= s;
    double area = 0;
    for (int i = 1; i + 1 < (int)a.size(); ++i)
      area += _cross(a[0], a[i], a[i + 1]);
    if (area < 0) reverse(ALL(a));
    auto less = [](Point u, Point v) { return
      tie(u.y, u.x) < tie(v.y, v.x); };
    if (!
        area) { auto [l, r] = minmax_element(ALL(a),less);
      a = {*l, *r}; } // Collinear: segment.
    rotate(a.begin(), min_element(ALL(a), less), a.end());
    a.push_back(a[0]); return a; }; // Close for indexing.
  auto a = prepare(P, n, 1), b = prepare(Q, m, -1);
  n = a.size() - 1, m = b.size() - 1;
  auto sum = [&](int i, int j) { return Point{
    a[i].x + b[j].x, a[i].y + b[j].y}; };
  double ans = numeric_limits<double>::infinity();
  double area = 0; bool inside = true;
  auto half = [](Point p) { return // Opposite edges.
    p.y < 0 || (!p.y && p.x < 0); };
  for (int i = 0, j = 0; i < n || j < m;) {
    Point p = sum(i, j), u = a[min(i + 1, n)],
      v = b[min(j + 1, m)];
    u.x -= a[i].x, u.y -= a[i].y,
      v.x -= b[j].x, v.y -= b[j].y;
    double z = _cross({0, 0}, u, v);
    bool az = !u.x && !u.y, bz = !v.x && !v.y;
    if (!az && !bz && half(u) != half(v))
      z = half(v) - half(u);
    bool x = i < n && (j == m || az || (!bz && z >= 0)),
      y = j < m && (i == n || bz || (!az && z <= 0));
    i += x, j += y; Point q = sum(i, j);
    inside &= _cross(p, q, {0, 0}) >= -1e-10;
    area += _cross({0, 0}, p, q);
    ans = min(ans, _point_seg_dist({0, 0}, p, q));
  } // Minkowski difference; O(n+m) time/space.
  return inside && area > 0 ? 0 : ans;
}
```

8.14 2D Convex Hull [64eaa3]

Monotone chain in $O(n \log n)$ (collinear points dropped), plus CH for $O(\log n)$ extreme-point, line-intersection, containment and tangent queries. CH needs a strictly convex CCW hull, at least 3 vertices.

```
bool operator<(const P &a, const P &b) {
  return a.x == b.x ? a.y < b.y : a.x < b.x; }
bool operator>(const P &a, const P &b) { return b < a; }
double crx(P a, P b, P c) { return (b - a) ^ (c - a); }
// CCW, drops collinear; O(n log n); reliable cross/dot.
vector<P> convex(vector<P> ps) {
  if (SZ(ps) < 2) return ps;
  sort(ALL(ps));
  if (!(ps.front() < ps.back())) return {ps[0]};
  vector<P> p; p.reserve(ps.size() + 1);
  auto add = [&](P q, int t) {
    while (SZ(p) > t &&
           crx(p
               [SZ(p)-2], q, p.back()) >= 0) p.pop_back();
    p.pb(q);
  };
  for (P q : ps) add(q, 1);
  int t = SZ(p);
  for (int i = SZ(ps) - 2; i >= 0; --i) add(ps[i], t);
  p.pop_back(); return p;
}
int sgn(double x) {
  return same(x, 0) ? 0 : x > 0 ? 1 : -1; }
P isLL(P p1, P p2, P q1, P q2) { // Nonparallel lines.
  double a = crx(q1, q2, p1), b = -crx(q1, q2, p2);
  return (p1 * b + p2 * a) / (a + b);
```

```
}
struct CH { // Strict convex CCW; n>=3; reliable eps.
  // O(log n) queries; initialize before use.
  int n;
  vector<P> p, u, d;
  CH() {}
  CH(vector<P> ps) : n(SZ(ps)), p(move(ps)) {
    rotate(p.begin(), min_element(ALL(p)), p.end());
    auto t = max_element(ALL(p));
    d = vector<P>(p.begin(), next(t));
    u = vector<P>(t, p.end()); u.pb(p[0]);
  }
  int find(vector<P> &v, P d) { // Peak of chosen chain.
    int l = 0, r = SZ(v);
    while (l + 1 < r) {
      int m = (l + r) / 2;
      if (v[m - 1] * d >= v[m] * d) r = m;
      else l = m;
    } return l;
  }
  int findFarest(P v) {
    if (v.y > 0 || (v.y == 0 && v.x > 0))
      return (SZ(d) - 1 + find(u, v)) % n;
    return find(d, v);
  }
  P get(int l, int r, P a, P b) {
    int s = sgn(crx(a, b, p[l % n]));
    while (l + 1 < r) {
      int m = (l + r) >> 1;
      if (sgn(crx(a, b, p[m % n])) == s) l = m;
      else r = m;
    } return isLL(a, b, p[l % n], p[(l + 1) % n]);
  }
  vector<P> getLineIntersect(P a, P b) { // a != b
    P v = b - a;
    int x = findFarest(P(-v.y, v.x)),
        y = findFarest(P(v.y, -v.x));
    if (x > y) swap(x, y);
    if (sgn(crx(a, b, p[x])) * sgn(crx(a, b, p[y])) < 0)
      return {get(x, y, a, b), get(y, x + n, a, b)};
    return {}; // tangent case falls here
  }
  void update_tangent(P q, int i, int &a, int &b) {
    if (sgn(crx(q, p[a], p[i])) > 0) a = i;
    if (sgn(crx(q, p[b], p[i])) < 0) b = i;
  }
  void bs(int l, int r, P q, int &a, int &b) {
    if (l == r) return;
    update_tangent(q, l % n, a, b);
    int s = sgn(crx(q, p[l % n], p[(l + 1) % n]));
    while (l + 1 < r) {
      int m = (l + r) >> 1;
      if (sgn(crx(q, p[m % n], p[(m+1) % n])) == s) l = m;
      else r = m;
    } update_tangent(q, r % n, a, b);
  }
  bool contain(P p) {
    if (p.x < d[0].x || p.x > d.back().x) return 0;
    auto it = lower_bound(ALL(d), p.x,
      [](P a, double x) { return a.x < x; });
    if (it->x == p.x && it->y > p.y) return 0;
    if (it->x != p.x && crx(*prev(it), *it, p) < -eps)
      return 0;
    it = lower_bound(ALL(u), p.x,
      [](P a, double x) { return a.x > x; });
    if (it->x == p.x && it->y < p.y) return 0;
    if (it->x != p.x && crx(*prev(it), *it, p) < -eps)
      return 0;
    return 1;
  }
  bool get_tangent(P p, int &a, int &b) { // b -> a
    if (contain(p)) return 0;
    a = b = 0;
    int i = lower_bound(ALL(d), p) - d.begin();
    bs(0, i, p, a, b);
    bs(i, SZ(d), p, a, b);
    i = lower_bound(ALL(u), p, greater<P>()) - u.begin();
    bs(SZ(d) - 1, SZ(d) - 1 + i, p, a, b);
    bs(SZ(d) - 1 + i, SZ(d) - 1 + SZ(u), p, a, b);
    return 1;
  }
};
vector<P> hull = convex(points); // CCW, drops collinear
CH ch(hull); bool inside = ch.contain(q); // SZ(hull)>=3
```

8.15 3D Convex Hull [3109f0]

Incremental 3D hull in $O(n^2)$; faces() counts coplanar face groups in $O(m)$. **Input must not contain duplicate points.** Set $0 \le n \le 1010, m = 0$; input is permuted.

```
double absvol(P a, P b, P c, P d) {
  return abs(((b-a)^(c-a))*(d-a))/6; }
// 0<=n<=1010, m=0; distinct p; finite arithmetic.
// O(n^2); permutes p; outward f[0..m); reliable eps.
struct convex3D { // Degenerate input: m=0.
  static const int maxn=1010;
  struct T{ int a,b,c; bool res; T(){}
    T(int a,int b,int c,bool res=1):
      a(a),b(b),c(c),res(res){}
  };
  int n,m; P p[maxn];
  T f[maxn*8]; int id[maxn][maxn];
  bool on(T &t,P &q) {
    return ((p[t.c]-p[t.b])^(p[t.a]-p[t.b]))
      *(q-p[t.a])>eps; }
  void add(T t){
    id[t.a][t.b]=id[t.b][t.c]=id[t.c][t.a]=m; f[m++]=t; }
  void meow(int q,int a,int b){
    int g=id[a][b]; if(f[g].res){
      if(on(f[g],p[q]))dfs(q,g);
      else add(T(b,a,q));
    }
  }
  void dfs(int p,int i){ f[i].res=0;
    meow(p,f[i].b,f[i].a);
    meow(p,f[i].c,f[i].b); meow(p,f[i].a,f[i].c); }
  void operator()(){
    if(n<4)return;
    auto choose = [&](int k, auto test) {
      for(int i=k;i<n;++i)if(test(p[i])){
        swap(p[k],p[i]); return true;
      } return false;
    };
    if(!choose(1,[&](P q){return abs(p[0]-q)>eps;}) ||
       !choose(2,[&](P q){
         return abs((p[0]-q)^(p[1]-q))>eps;}) ||
       !choose(3,[&](P q){
         return abs(((p[1]-p[0])^(p[2]-p[0]))
           *(q-p[0]))>eps;
       }))return;
    for(int i=0;i<4;++i){
      T t((i+1)%4,(i+2)%4,(i+3)%4,1);
      if(on(t,p[i]))swap(t.b,t.c);
      add(t);
    } for(int i=4;i<n;++i)
      for(int j=0;j<m;++j)
        if(on(f[j],p[i])){
          dfs(i,j); int mm=m; m=0;
          for(int k=0;k<mm;++k)if(f[k].res)add(f[k]);
          break; }
  }
  bool same(int i,int j){
    for(int k : {f[j].a,f[j].b,f[j].c})
      if(absvol(p[f[i].a],p[f[i].b],p[f[i].c],p[k])>eps)
        return false;
    return true;
  }
  int faces(){ // O(m); reliable coplanarity at eps.
    int r=0; vector<int> seen(m), q;
    for(int i=0;i<m;++i)if(!seen[i]){
      ++r; seen[i]=1; q={i};
      for(int j=0;j<(int)q.size();++j){ T t=f[q[j]];
        for(int
            k:{id[t.b][t.a],id[t.c][t.b],id[t.a][t.c]})
          if(!seen[k] && same(i,k))
            seen[k]=1,q.push_back(k);
      }
    } return r;
  }
} tb;
```

8.16 Minimum Enclosing Circle [b26f68]

Welzl randomised incremental with shuffle, expected $O(n)$. pt() must be origin; empty: radius 0. Recenter ill-scaled data.

```
// center requires noncollinear a,b,c.
pt center(const pt &a, const pt &b, const pt &c) {
  pt p0 = b - a, p1 = c - a;
  double d = p0 ^ p1;
  double c1 = norm2(p0)*0.5/d, c2 = norm2(p1)*0.5/d;
  return a + pt(c1 * p1.y - c2 * p0.y,
                c2 * p0.x - c1 * p1.x);
}
// pt
    ()=origin; no overflow/underflow; recenter ill-scaled.
// Squared-distance roundoff < relative slack 1e-12.
// |p| fits int; shuffles p; expected O(n), worst O(n^3).
circle min_enclosing(vector<pt> &p) {
  static mt19937 rng(712367821);
  shuffle(ALL(p), rng);
  double r = 0; pt cent;
  for (int i = 0; i < SZ(p); ++i) {
```

```
    if (norm2(cent - p[i]) <= r * (1 + 1e-12)) continue;
    cent = p[i], r = 0;
    for (int j = 0; j < i; ++j) {
      if (norm2(cent - p[j]) <= r * (1 + 1e-12)) continue;
      cent = (p[i] + p[j]) / 2, r = norm2(p[j] - cent);
      for (int k = 0; k < j; ++k) {
        if (norm2(cent-p[k]) <= r*(1+1e-12)) continue;
        cent = center(p[i], p[j], p[k]);
        r = norm2(p[k] - cent);
      }
    }
  } return circle(cent, sqrt(r));
}
vector<pt> points = {{0,0},{2,0},{0,2}};
circle c = min_enclosing(points); // points are shuffled
```

8.17 Closest Pair [85b7f3]

Sweep line in $O(n \log n)$, with $O(n)$ auxiliary space. p[] **must already be sorted by** x; the array is preserved. Inclusive 0-base $[l,r]$; $l \geq r$ returns 1e9.

```
double closest_pair(int l, int r) {
  // Inclusive [l,r]; valid finite x-sorted p if l<r.
  // Euclidean dist; finite math; p unchanged; l>=r: 1e9.
  // O(n log n) time, O(n) space.
  if (l >= r) return 1e9;
  double d = dist(p[l], p[l + 1]);
  set<pair<double, int>> active; // (y,index), x-window
  for (int i = l, j = l; i <= r; ++i) {
    while (j < i && p[i].x - p[j].x >= d)
      active.erase({p[j].y, j}), ++j;
    auto it = active.lower_bound({p[i].y - d, l});
    for (; it != active.end() && it->first <= p[i].y + d;
         ++it)
      d = min(d, dist(p[i], p[it->second]));
    if (d == 0) return 0;
    if (i < r && p[i + 1].x - p[i].x < d)
      active.insert({p[i].y, i});
  } return d;
}
sort(p, p+n, [](auto a, auto b) { return a.x < b.x; });
// Inclusive; preserves p; n<2 returns 1e9.
double d = closest_pair(0, n-1);
```

9 Else

9.1 Cyclic Ternary Search* [59423b]

Minimum of a **cyclically shifted** unimodal array in $O(\log n)$; pred(a,b) means $f(a) < f(b)$; returns an index. The cyclic U-shape must have pairwise-distinct values (equivalently, strict comparisons); $n \geq 1$.

```
// n>=1; cyclic-shift U-function with distinct values.
// pred(a,b): f(a)<f(b); minimum index, O(log n).
int cyc_tsearch(int n, auto pred) {
  if (n == 1) return 0;
  int l = 0, r = n; bool rv = pred(1, 0);
  while (r - l > 1) {
    int m = l + (r - l) / 2;
    if (pred(0, m) ? rv : pred(m, m + 1 == n ? 0 : m + 1))
      r = m;
    else l = m;
  } return pred(l, r % n) ? l : r % n;
}
vector<int> a={3,4,5,1,2};
int at=cyc_tsearch(a.size(),
  [&](int i,int j){return a[i]<a[j];}); // at=3
```

9.2 CDQ: k-D Dominance Count [da7384]

For each point, counts other points $\leq$ it in every coordinate (ties and duplicates counted). Equal-length rows, $k \geq 0$ dims. Lexicographic sort removes dim 0, each CDQ layer removes one more, the last uses a BIT: $O(n \log^{\max(1,k-1)} n)$ for fixed $k \geq 1$. $k=0$ gives $n-1$. Requires C++20 (std::span).

```
// Other points <= in all dims; O(n log^max(1,k-1) n).
// Fixed k; equal-size rows; n,k fit int; enough stack.
vector<ll> dominance(vector<vector<int>> a) {
  int n = SZ(a), k = n ? SZ(a[0]) : 0, m = n;
  if (k > 1) { // Compress last dim for BIT.
    vector<int> v(n);
    for (int i = 0; i < n; ++i) v[i] = a[i].back();
    sort(ALL(v)); v.erase(unique(ALL(v)), v.end());
    m = SZ(v);
    for (auto &p : a)
      p.back
          () = lower_bound(ALL(v), p.back())-v.begin()+1;
  }
  vector<int> ord(n), grp(n), rep, w, bit(m + 1LL);
  iota(ALL(ord), 0);
  sort(ALL(ord), [&](int i, int j) {
    return a[i] < a[j]; });
  for (int i = 0; i < n; ++i) { // Merge duplicates.
    if (!i || a[ord[i]] != a[ord[i - 1]])
      rep.pb(ord[i]), w.pb(0);
    ++w.back(), grp[ord[i]] = SZ(rep) - 1;
  }
  vector<ll> ans(SZ(rep)), res(n);
  if (k < 2) exclusive_scan(ALL(w), ans.begin(), 0LL);
  vector
      <int> coord; coord.reserve(rep.size() * size_t(k));
  for (int i : rep) coord.insert(coord.end(), ALL(a[i]));
  struct It { int g; bool up, qu; };
  auto val = [&](It x, int d) {
    return coord[size_t(x.g) * k + d]; };
  // For each query, count earlier updates in dims d..k-1.
  auto rec = [&](auto &&self, int d, span<It> v) -> void {
    if (SZ(v) < 2) return;
    if (d == k - 1) {
      for (auto x : v) {
        if (x.qu) for (int i = val(x, d); i; i -= i & -i)
          ans[x.g] += bit[i];
        if (x.up)
          for (ll i = val(x, d); i <= m; i += i & -i)
            bit[i] += w[x.g];
      }
      for (auto x : v) if (x.up)
        for (ll i = val(x, d); i <= m; i += i & -i)
          bit[i] = 0;
      return;
    } int mid = SZ(v) / 2;
    self(self, d, v.first(mid));
    self(self, d, v.subspan(mid));
    vector<It> c; c.reserve(v.size());
    for (auto x : v.first(mid))
      if (x.up) c.pb({x.g, 1, 0});
    for (auto x : v.subspan(mid))
      if (x.qu) c.pb({x.g, 0, 1});
    sort(ALL(c), [&](It x, It y) { // Ties: updates first.
      return pair(val(x, d), !x.up)
          < pair(val(y, d), !y.up); });
    self(self, d + 1, c);
  };
  vector<It> v(SZ(rep));
  for (int i = 0; i < SZ(v); ++i) v[i] = {i, 1, 1};
  if (k > 1) rec(rec, 1, v);
  for (int i = 0; i < n; ++i)
    res[i] = ans[grp[i]] + w[grp[i]] - 1;
  return res; }
vector<vector<int>> a(n, vector<int>(3)); // n 3D points
auto c = dominance(a); // Other points <= in all dims.
vector<int> level(n); // Histogram of dominance counts
for (int i = 0; i < n; ++i) ++level[c[i]];
```

9.3 Mo's Algorithm(With modification) [6a768a]

Mo's with point updates, common block $N^{2/3}$, $O(N^{5/3})$ for $O(N)$ queries/updates. 0-base inclusive ranges; start empty at $T = 0$. Callbacks apply/undo 1-based updates and report queries. C++20.

```
// Common block>0; 0<=L<=R<n; 0<=T<=U (#updates); C++20.
// add_time/sub_time: apply/undo t>=1; [L,R] may be empty.
// Start
    empty at T=0; add/sub positions; answer(q): state.
// Block ~ N^(2/3), O(N^(5/3)) for O(N) queries/updates.
struct Query {
  int L, R, LBid, RBid, T, id;
  Query(int l, int r, int t, int block = 1,
        int query_id = -1) : L(l), R(r), LBid(l / block),
        RBid(r / block), T(t), id(query_id) {}
  bool operator<(const Query &q) const {
    if (LBid != q.LBid) return LBid < q.LBid;
    if (RBid != q.RBid)
      return LBid & 1 ? RBid > q.RBid : RBid < q.RBid;
    return (LBid ^ RBid) & 1 ? T > q.T : T < q.T;
  }
};
void solve(vector<Query> query, auto add_time,
           auto sub_time, auto add, auto sub,
           auto answer) {
  sort(query.begin(), query.end());
  int L = 0, R = -1, T = 0;
  for (const Query &q : query) {
    while (T < q.T) add_time(++T, L, R);
    while (T > q.T) sub_time(T--, L, R);
    while (R < q.R) add(++R);
    while (L > q.L) add(--L);
    while (R > q.R) sub(R--);
    while (L < q.L) sub(L++);
    answer(q);
  }
}
int B = max(1, (int)pow(n, 2.0/3));
qs.emplace_back(l, r, updates_seen, B, id);
solve(qs,
  [&](int t,int L,int R){ apply_update(t,L,R); },
  [&](int t,int L,int R){ rollback_update(t,L,R); },
  [&](int i){ add_pos(i); }, [&](int i){ remove_pos(i); },
  [&](const Query &q){ ans[q.id] = current_answer(); });
```

9.4 Mo's Algorithm On Tree [f3ac5b]

Path queries via a 0-base $2n$ Euler bracket sequence; sorting/ flips $O(q \log q + n^2/B + qB)$ with common block $B > 0$. Constructor takes caller's LCA; solve handles the extra LCA. Start empty; flip toggles vertices.

```
// Common block>0; valid 0-based 2n tour/LCA; 2n fits int.
// Entry/exit tour; flip toggles; start empty.
// block~sqrt(n), q=O(n): O(n sqrt(n)).
struct Query {
  int L, R, LBid, lca, id;
  Query(int u, int v, int block, const vector<int> &in,
        const vector<int> &out, auto get_lca,
        int query_id = -1)
    : id(query_id) {
    if (in[u] > in[v]) swap(u, v);
    int c = get_lca(u, v);
    if (c == u) L = in[u], R = in[v], lca = -1;
    else L = out[u], R = in[v], lca = c;
    LBid = L / block;
  }
  bool operator<(const Query &q) const {
    if (LBid != q.LBid) return LBid < q.LBid;
    return LBid & 1 ? R > q.R : R < q.R;
  }
};
void solve(vector<Query> query, const vector<int> &ord,
           auto flip, auto answer) {
  sort(query.begin(), query.end());
  int L = 0, R = -1;
  for (const Query &q : query) {
    while (R < q.R) flip(ord[++R]);
    while (L > q.L) flip(ord[--L]);
    while (R > q.R) flip(ord[R--]);
    while (L < q.L) flip(ord[L++]);
    if (q.lca != -1) flip(q.lca);
    answer(q);
    if (q.lca != -1) flip(q.lca);
  }
}
int B = max(1, (int)sqrt(2*n));
qs.emplace_back(u, v, B, in, out, get_lca, id);
solve(qs, ord, [&](int v){ toggle(v); },
  [&](const Query &q){ ans[q.id] = current_answer(); });
```

9.5 Additional Mo's Algorithm Trick

- Mo's Algorithm With Addition Only
  - Sort querys same as the normal Mo's algorithm.
  - For each query $[l,r]$:
  - If $l/blk = r/blk$, brute-force.
  - If $l/blk \neq curL/blk$, initialize $curL := (l/blk+1)\cdot blk$, $curR := curL-1$
  - If $r > curR$, increase $curR$
  - decrease $curL$ to fit l, and then undo after answering
- Mo's Algorithm With Offline Second Time
  - Require: Changing answer $\equiv$ adding $f([l,r],r+1)$.
  - Require: $f([l,r],r+1) = f([1,r],r+1) - f([1,l),r+1)$.
  - Part1: Answer all $f([1,r],r+1)$ first.
  - Part2: Store $curR \to R$ for $curL$ (reduce the space to $O(N)$), and then answer them by the second offline algorithm.
  - Note: You must do the above symmetrically for the left boundaries.

9.6 Hilbert Curve [e9c3c2]

Hilbert index of (x,y) in $O(\log n)$, positive int $n = 2^k, 0 \le x,y < n$. Use it to order Mo's queries instead of block sorting.

```
ll hilbert(int n, int x, int y) {
  ll res = 0;
  for (int s = n / 2; s; s >>= 1) {
    int rx = (x & s) > 0, ry = (y & s) > 0;
    res += s * 1ll * s * ((3 * rx) ^ ry);
    if (!ry && rx) x = s - 1 - x, y = s - 1 - y;
    if (!ry) swap(x, y);
  } return res;
} // n: positive int power of 2; 0<=x,y<n; O(log n).
```

9.7 DynamicConvexTrick* [673ffd]

Line container (CHT) with arbitrary insert order, amortised $O(\log n)$, keeps the **maximum**. **Integer coordinates only.** Nonempty hull; $-10^{18} < x \le 10^{18}$; ll arithmetic must fit.

```
// Max hull; integer -kInf<x<=kInf; query nonempty hull.
// All math fits ll; O(log n) add amortised / query.
struct Line {
  mutable ll a, b, p;
  bool
      operator<(const Line &rhs) const { return a<rhs.a; }
  bool operator<(ll x) const { return p < x; }
};
struct DynamicHull : multiset<Line, less<>> {
  static const ll kInf = 1e18;
  ll Div(ll a, ll b) { return a/b - ((a^b) < 0 && a%b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = kInf; return 0; }
    if (x->a == y->a) x->p = x->b > y->b ? kInf : -kInf;
    else x->p = Div(y->b - x->b, x->a - y->a);
    return x->p >= y->p;
  }
  void addline(ll a, ll b) {
    auto z = insert({a, b, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y))
      isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) { auto l = *lower_bound(x);
    return l.a * x + l.b; }
};
DynamicHull hull;
hull.addline(2, 3); hull.addline(-1, 10);
ll best = hull.query(x);
```

9.8 All LCS* [ce3f17]

Returns every distinct longest common subsequence in lexicographic order. A zero-length LCS gives one empty string; output may be exponential. Uses the standard $O(|s||t|)$ DP plus memoised reconstruction.

```
// Sorted distinct LCS; DP O(nm), O(n) if s==t.
// Exponential output; merges linear in string data.
// Lengths < INT_MAX; cells fit size_t; O(n+m) stack.
vector<string> all_lcs(const string &s, const string &t) {
  if (s == t) return {s};
  int n = (int)s.size(), m = (int)t.size();
  vector<vector<int>> dp(n + 1, vector<int>(m + 1));
  for (int i = n - 1; i >= 0; --i)
    for (int j = m - 1; j >= 0; --j)
      dp[i][j] = s[i] == t[j] ? 1 + dp[i + 1][j + 1]
        : max(dp[i + 1][j], dp[i][j + 1]);
  if (!dp[0][0]) return {""};
  vector<vector<string>> memo(size_t(n + 1) * (m + 1));
  auto go = [&](auto &&self, int i, int j)
      -> vector<string> & {
    auto &ret = memo[size_t(i) * (m + 1) + j];
    if (!ret.empty()) return ret; // Empty LCS gives {""}.
    if (!dp[i][j]) return ret.push_back(""), ret;
    if (s[i] == t[j]) {
      for (const string &x : self(self, i + 1, j + 1))
        ret.push_back(s[i] + x);
    } else if (dp[i + 1][j] > dp[i][j + 1])
      ret = self(self, i + 1, j);
    else if (dp[i + 1][j] < dp[i][j + 1])
      ret = self(self, i, j + 1);
    else {
      auto &a = self(self, i + 1, j);
      auto &b = self(self, i, j + 1);
      set_union(ALL(a), ALL(b), back_inserter(ret));
    } return ret;
  }; return move(go(go, 0, 0));
}
```

9.9 AdaptiveSimpson* [100d34]

Adaptive Simpson integration with Richardson extrapolation. Use eval2 to pre-split multi-peak integrands. Needs smooth deterministic f, attainable $\varepsilon > 0$, and $k > 0$.

```
template<class Func, class d = double>
struct Simpson {
  using pdd = pair<d, d>; Func f;
  pdd mix(pdd l, pdd r, optional<d> fm = {}) { // {x,f(x)}
    d h = (r.first - l.first) / 2;
    d v = fm ? *fm : f(l.first + h);
    return {v, h / 3 * (l.second + 4 * v + r.second)};
  }
  d eval(pdd l, pdd r, d fm, d eps) { // fm=f(midpoint)
    pdd m(l.first + (r.first - l.first) / 2, fm);
    d s = mix(l, r, fm).second;
    auto [flm, sl] = mix(l, m);
    auto [fmr, sr] = mix(m, r);
    d delta = sl + sr - s;
    if (abs(delta) <= 15 * eps)
      return sl + sr + delta / 15;
    return eval(l, m, flm, eps / 2)
         + eval(m, r, fmr, eps / 2);
  }
  // Attainable
       eps>0; smooth deterministic f; finite math.
  // Enough stack; error estimate, not a rigorous bound.
  d eval(d l, d r, d eps) {
    return eval({l, f(l)}, {r, f(r)},
                f(l + (r - l) / 2), eps);
  }
  // k>0; pre-split peaks, reuse endpoints.
  d eval2(d l, d r, d eps, int k = 997) {
    d h = (r - l) / k, s = 0; pdd a(l, f(l));
    for (int i = 0; i < k; ++i) {
      d x = i+1 == k ? r : l + h*(i+1); pdd b(x, f(x));
      s += eval(a, b, f(a.first + (x-a.first)/2), eps/k);
      a = b;
    } return s;
```

```
  }
};
template<class Func> Simpson<Func> make_simpson(Func f) {
  return {f}; }
auto f=make_simpson([](double x){return sin(x);});
double area=f.eval(0,acos(-1.0),1e-10); // area ~= 2
```

9.10 Simulated Annealing [d78052]

Generic minimising annealing routine: caller supplies a score and neighbour proposal, and receives the best state found. Score must be pure; proposal preserves its input; state supports copying.

```
// Minimise pure score; State needs copy ctor/assignment.
// next_state preserves input; finite double scores/math.
// O(iterations * (proposal + scoring + state-copy cost)).
template<class State, class Score, class NextState>
State simulated_annealing(State current, Score score,
  NextState next_state,
  mt19937_64 &rng, int iterations = 100000,
  double temperature=100000.0, double cooling=0.99995) {
  State best = current;
  double cur = score(current), best_score = cur;
  uniform_real_distribution<double> unit(0.0, 1.0);
  for (int it = 0; it < iterations; ++it) {
    State candidate = next_state(current, rng);
    double cand = score(candidate);
    if (cand < cur ||
        exp((cur - cand) / max(temperature, 1e-15))
        >= unit(rng)) current = candidate, cur = cand;
    if (cur
         < best_score) best = current, best_score = cur;
    temperature *= cooling;
  } return State(best); // Copy; move ctor may be deleted.
}
mt19937_64 rng(7122);
int best
    =simulated_annealing(0,[](int x){return abs(x-3);},
  [](int x,auto&){return x<3?x+1:x-1;},rng); // best=3
```

9.11 Tree Hash* [f75ff7]

Order-independent rooted tree hash in $O(n)$ through rooted_tree_hash; for unrooted trees hash at each centroid and take the minimum.

```
// O(n); rooted, order-independent; collisions possible.
// Nonempty tree, valid root/adjacency; O(height) stack.
using tree_hash_u64 = unsigned long long;
constexpr tree_hash_u64 tree_hash_default_seed =
  0x9e3779b97f4a7c15ULL;
tree_hash_u64 tree_hash_shift(tree_hash_u64 x) {
  x ^= x << 13, x ^= x >> 7, x ^= x << 17;
  return x; }
tree_hash_u64 tree_hash_dfs(const vector<vector<int>> &g,
  int u, int parent, tree_hash_u64 seed) {
  if (!seed) seed = tree_hash_default_seed;
  tree_hash_u64 sum = seed;
  for (int v : g[u]) if (v != parent)
    sum += tree_hash_shift(tree_hash_dfs(g, v, u, seed));
  return sum; }
tree_hash_u64 rooted_tree_hash(
  const vector<vector<int>> &g, int root = 0,
  tree_hash_u64 seed = tree_hash_default_seed) {
  return tree_hash_dfs(g, root, -1, seed); }
```

9.12 Binary Search On Fraction [7e5f4f]

Smallest fraction p/q, $0 \le p,q \le N$, via Stern-Brocot descent with doubling; frac_bs(N,pred). Require $N \ge 1$, increasing pred, true at 1/0(infinity); arithmetic fits ll.

```
struct Q { ll p, q;
  Q go(Q b, ll d) { return {p + b.p*d, q + b.q*d}; }
};
// Increasing bool pred; true at 1/0; search fits ll.
// Min feasible p/q, 0<=p,q<=N, N>=1; q=0 is infinity.
template<class Pred>
Q frac_bs(ll N, Pred pred) {
  Q lo{0, 1}, hi{1, 0};
  if (pred(lo)) return lo; assert(pred(hi));
  bool dir = 1, L = 1, H = 1;
  for (; L || H; dir = !dir) {
    ll len = 0, step = 1;
    for (int t = 0; t < 2 && (t ? step/=2 : step*=2);)
      if (Q mid = hi.go(lo, len + step);
          mid.p > N || mid.q > N || dir ^ pred(mid)) t++;
      else len += step;
    swap(lo, hi = hi.go(lo, len));
    (dir ? L : H) = !!len;
  } return dir ? hi : lo;
}
Q x=frac_bs(100,[](Q v){
  return v.q==0 || 3*v.p>=2*v.q;
}); // x=2/3
```

9.13 Min Plus Convolution* [f5a92e]

$c_k = \min_{i+j=k}(a_i + b_j)$ in $O((n+m)\log(n+m))$ by divide and conquer on monotone decisions. **Requires** a **convex**, b arbitrary; supports empty inputs and exact additive arithmetic whose sums are representable.

```
// Convex a: a[i+1]-a[i] <= a[i+2]-a[i+1]; b arbitrary.
// O((n+m)log(n+m)); empty -> {}; exact sums fit T.
template<class T> vector<T> min_plus_convolution(
  const vector<T> &a, const vector<T> &b) {
  int n = (int)a.size(), m = (int)b.size(); // n+m fit int
  if (!n || !m) return {};
  vector<T> c(n + m - 1);
  auto dc = [&](auto &&self, int l, int r,
                int jl, int jr) -> void {
    int mid
        = l + (r - l) / 2, from = max(jl, mid - n + 1);
    T best = a[mid - from] + b[from];
    for (int j = from + 1; j <= min(jr, mid); ++j)
      if (T v = a[mid - j] + b[j]; v < best)
        best = v, from = j;
    c[mid] = best; // Leftmost argmin is monotone.
    if (l < mid) self(self, l, mid - 1, jl, from);
    if (mid < r) self(self, mid + 1, r, from, jr);
  };
  dc(dc, 0, n + m - 2, 0, m - 1); return c;
}
vector<int> a={0,1,4}, b={3,0}; // a must be convex
auto c=min_plus_convolution(a,b); // c={3,0,1,4}
```

9.14 Bitset LCS [a17444]

LCS via 64-bit subtract-and-borrow; bitset_lcs(a,b). Expected $O(m+(n+D)\lceil m/64\rceil)$, D: distinct values in b. C++20; m fits int.

```
// Expected O(m+(n+D)ceil(m/64)); D distinct values in b.
// b.size() fits int; either empty input returns 0.
int bitset_lcs(const vector<int>& a,
               const vector<int>& b) {
  if (a.empty() || b.empty()) return 0;
  int m = (int)b.size(), W = (m + 63LL) >> 6;
  unordered_map<int, vector<unsigned long long>> mask;
  for (int j = 0; j < m; ++j) {
    auto &v = mask.try_emplace(b[j], W).first->second;
    v[j >> 6] |= 1ULL << (j & 63);
  }
  vector<unsigned long long> f(W);
  for (int value : a) {
    auto it = mask.find(value);
    if (it == mask.end()) continue;
    unsigned long long carry = 1, borrow = 0;
    for (int i = 0; i < W; ++i) {
      auto x = f[i] | it->second[i];
      auto y = (f[i] << 1) | carry;
      carry = f[i] >> 63;
      auto rhs = y + borrow;
      borrow = (rhs < y) || (x < rhs);
      f[i] = x & ~(x - rhs);
    }
  } // Padding stays zero: new bits are a subset of x.
  int ans = 0; for (auto word : f) ans += popcount(word);
  return ans;
}
```

9.15 N Queens Problem [6f75f9]

Appends one solution's 1-based columns in $O(n)$ by cases on n mod 6 — no search; $n \ge 0$. **No solution for** $n = 2,3$; appends nothing.

```
// n>=0; appends 1-based columns; O(n).
void solve(vector<int> &ret, int n) {
  if (n == 2 || n == 3) return; // No solution.
  auto offset = ret.size(); if (!offset) ret.reserve(n);
  for (ll i = 2; i <= n; i += 2) ret.pb(i); // Evens.
  for (ll i = 1; i <= n; i += 2) ret.pb(i);
  auto first = ret.begin() + offset, odd = first + n / 2;
  if (n % 6 == 2) {
    swap(odd[0], odd[1]);
    rotate(odd + 2, odd + 3, ret.end());
  } else if (n % 6 == 3) {
    rotate(first, first + 1, odd);
    rotate(odd, odd + 2, ret.end());
  }
}
vector<int> column;
solve
    (column,8); // column[row] is 1-based; empty for n=2,3
```

9.16 Digit DP [35e1ac]

Memoized binary digit DP over (pos,pre,lead,limit,cnt); as written calc(x) $= \prod_{v=1}^{x} \text{popcount}(v) \bmod \text{mod}$, calc(0)=0. Needs globals s, dp[pos][pre][cnt], mod, macro sz; int is long long.

```
int dfs(int pos, int pre, int lead, int limit, int cnt){
    if(pos == -1)return max(cnt,1ll);
    if(!lead && !limit &&
         dp[pos][pre][cnt] != -1)return dp[pos][pre][cnt];
    int lb = 0, rb = (limit ? s[pos]-'0' : 1), ret = 1;
```

```
    for(int i=lb;i<=rb;i++){
        ret *= dfs(pos - 1, i, lead
             & (i == 0), limit & (i == rb), cnt + (i==1));
        ret %= mod;
    }
    if(!lead && !limit)dp[pos][pre][cnt] = ret;
    return ret;
}
int calc(int x){
    s.clear();
    if(x==0)return 0;
    while(x){s += (x%2 + '0');x/=2;}
    memset(dp,-1,sizeof dp);
    return dfs(sz(s)-1,0,1,1,0);
}
int answer=calc(R)-calc(L-1); // range [L,R]
```

10 Python

10.1 Misc

Importable Python fallbacks for exact decimals/ fractions, matrix and fast-array parsing, modular powers, seeded random helpers, and formatting. Huge integer input raises Python 3.11+'s digit limit.

```
from decimal import (Decimal, Context, localcontext,
                     MAX_PREC, MAX_EMAX, ROUND_FLOOR)
from fractions import Fraction
import sys
# Exact decimals; caller's context unchanged.
def decimal_multiply(a, b):
    with
        localcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX,
                             rounding=ROUND_FLOOR)):
        return Decimal(a) * Decimal(b)
def limit_fraction(s, bound):
    return Fraction(s).limit_denominator(bound)
def read_matrix(lines, n, m):  # n,m>=0; at least n rows.
    it = iter(lines)
    a = [list(map(int, next(it).split()))
         for _ in range(n)]
    if any(len(row) != m for row in a):
        raise ValueError("row length")
    return a
def modular_power(a, b, m): return pow(a, b, m)
# rng = random.Random(seed); choices uses keyword k.
def random_examples(rng, values):
    if not values: raise ValueError("empty choices")
    a = list(values); rng.shuffle(a)
    return rng.choice(values), rng.choices(values, k=2), a
# rng
    .randrange(L,R,step),rng.randint(L,R),rng.uniform(L,R)
def format_example(num, pi, text):
    return f"num: {num}, pi: {pi:.2f}, str: {text}"
def parse_int_array(data):  # sys.stdin.buffer.read()
    a = list(map(int, data.split()))
    if not a or a[0] < 0 or len(a) != a[0]+1:
        raise ValueError("length")
    return a.pop(0), a
# with open("in") as f: data = f.read()
# with open("out", "w") as f: print(1,2,sep=" | ",file=f)
# print("debug", file=sys.stderr); sys.stdin = open("in")
# sys.set_int_max_str_digits(5000000)  # 3.11+, huge ints
```